מדעי הבינה המלאכותית

חוברת היסוד — מהדורת תלמיד

ארבעת פרקי המבוא למסלול למידת מכונה ולמידה עמוקה

Python | OOP | Data Science & Machine Learning | EDA

85 שעות לימוד לפי מפת תכנית ההתמחות

ספר לימוד לתיכון • הסברים, דוגמאות, קוד, גרפים ותרגול עצמאי

מבוסס על חוברות הפרקים המקוריות שנבנו עבור הספר המאוחד. קובצי Word ומחברות Colab מצורפים בנפרד לעריכה ולהוראה.

מהדורת נובמבר 2026 — ספר תלמיד

תוכן העניינים ומפת הלמידה

הסדר מיועד להוביל מהבנת קוד ומבני תוכנה לעבודה עצמאית עם נתונים. כל פרק נפתח מחדש, אך כל מושג ממשיך לשרת את המשימות בפרקים שאחריו.

פרק	נושא	שעות	עמוד בספר
1	יסודות Python למפתח AI	30	3
2	תכנות מונחה עצמים ב־Python	30	21
3	מבוא למדעי הנתונים ולמידת מכונה	10	39
4	חקר נתונים (EDA)	15	56

כיצד משתמשים בחוברת?

ראשית קוראים את ההסבר ואת הדוגמה הפתורה. לאחר מכן מריצים את קוד ההדגמה ומשווים לחישוב או לטבלה. לבסוף פותרים את משימות Colab העצמאיות ומנסחים במילים את הממצא ואת מגבלותיו.

שימו לב: ארבעת הפרקים מהווים את חוברת היסוד של מסלול ההתמחות (85 שעות). הם אינם מחליפים את שאר פרקי הלמידה העמוקה או את ההוראות הרשמיות העדכניות לבחינות ולפרויקטים.

חוברת היסוד לספר הקורס המאוחד

פרק 1 — היכרות חוזרת עם Python למפתח AI

מהקוד הראשון ועד לעיבוד נתונים, פונקציות ומערכים מספריים

מסלול התמחות למידת מכונה ולמידה עמוקה | כיתות י״א–י״ב | מסגרת סילבוס: 30 שעות

ספר תלמיד עצמאי • הסברים מלאים • דוגמאות מקוריות • קוד מודרך • תרגול עצמי



תהליך הלמידה בספר: לומדים לכתוב קוד כדי לשאול שאלות על נתונים — ורק בהמשך לאמן מודלים.

שאלה פותחת איך נכתוב תוכנית שמקבלת מדידות של מחשבים, מזהה מצב חריג ומכינה נתונים למחקר — בלי לאבד את היכולת להסביר כל שורה?

בפרק הזה דודו אחראי על מעבדת מחשבים בבית הספר. הוא אוסף טמפרטורות, שעות שימוש ונתוני תקלות. בתחילה הוא מחשב ממוצע ומסמן מדידות גבוהות. בהמשך הוא מעביר את הפעולות לפונקציות ומייצג את הנתונים במערכים. כל פעולה נועדה לפתור בעיה אמיתית, לא רק להדגים תחביר.

הפרק מבוסס על מפת תכנית ההתמחות שבמאגר הקורס שלנו: פרק 1 — היכרות חוזרת עם שפת תכנות, 30 שעות. חלוקת השעות הפנימית ומקרי החקר הם תכנון פדגוגי מקורי; אין בכך טענה שהפרטים מופיעים מילה במילה בסילבוס הרשמי.

מפת הדרך של הפרק

יחידת לימוד	מוקד	שעות מוצעות
1.1	סביבת Google Colab, קוד ונתונים	3
1.2	משתנים, טיפוסים, פעולות וחישובים	3
1.3	מחרוזות, קלט והדפסה קריאה	3
1.4	תנאים, ביטויים בוליאניים ולוגיקה	3
1.5	לולאות, סכימה וספירה	3
1.6	רשימות, אינדקסים והעתקת נתונים	3
1.7	מילונים ומבנה של רשומה	3
1.8	פונקציות, תכנון קוד ובדיקות	3
1.9	שגיאות, מודולים וקובצי נתונים	3
1.10	NumPy ופרויקט מסכם	3

בכל יחידה: בעיה קצרה, הסבר מושגי, דוגמה פתורה, קוד, טעויות נפוצות, משימות העברה וכרטיס יציאה. חלק מהמיומנויות מוכרות מיחידה 3; כאן מעמיקים בהן כדי להכין קוד יציב וקריא לפרויקט למידה עמוקה.

מפת הקשרים בין שלוש תכניות הלימוד

לאן ממשיכים	מה כבר נלמד / מה נלמד כאן	מסלול
הפעלת פונקציות וקוד עצמאי שאפשר להסביר בעל־פה	היכרות עם Colab, משתנים, רשימות, Pandas וניתוח ראשוני	יחידה 3 — כיתה י׳
הבנה מתמטית של פעולות NumPy ומרחקים	וקטורים, מטריצת מאפיינים, תיאור נתונים ואלגוריתמים	יחידה 5 — נושאי ליבה
OOP, EDA, טנזורים ומערכת פרויקט גמר	השלמת יסודות Python, קבצים, פונקציות ובדיקות	למידה עמוקה — פרק 1

מפת הקשרים היא תכנון דידקטי המתבסס על המסלולים במאגר; אין לראות בכל שורה דרישת חובה זהה בשלוש תכניות הלימודים.

מה נדרש לדעת לפני שמתחילים?

נדרשת היכרות בסיסית עם מושגי קלט ופלט, משתנה ופעולת חשבון. מי שכבר למד Python יוכל לבצע את מבדק הפתיחה ולעבוד מהר יותר בחלקי החזרה, אבל חשוב שלא לדלג על תרגול של פונקציות, רשימות ומבני נתונים.

מבדק פתיחה — ללא מחשב

1. מה ההבדל בין המספר 7 לבין המחרוזת "7"? איזו פעולה אפשר לבצע על כל אחד מהם?
2. אם ברשימה יש חמישה איברים, מהו האינדקס של האיבר האחרון?
3. מה פירוש הביטוי "הפונקציה מחזירה ערך"? במה הוא שונה מהדפסת ערך?
4. דודו קיבל 4 מדידות: 70, 82, 75, 89. כיצד יחשב ממוצע ומספר מדידות של 80 ומעלה?

מילון סימנים מוקדם

סימן	משמעות
=	השמה של ערך למשתנה
==	השוואת שני ערכים
:	תחילת בלוק כמו if, for או def
[]	רשימה, אינדקס או חיתוך
()	קריאה לפונקציה או קיבוץ ביטוי
None	היעדר ערך שימושי, ולא המספר אפס
shape	ממדי מערך NumPy; לא מופיע ברשימת Python רגילה

1.1 — סביבת עבודה, תאים וסדר הרצה

1 עובדים במחברת המורכבת מתאים. תא טקסט מאפשר לתאר את שאלת המחקר; תא Google Colab קוד מריץ הוראות Python ומציג פלט. היתרון הוא שניתן לשלב באותו מסמך הסבר, חישוב, טבלה וגרף. החיסרון הוא שתאים אינם תמיד רצים לפי סדר הופעתם: אם מריצים את תא 6 ואז חוזרים לתא 2, המשתנים בזיכרון עלולים להכיל ערכים שלא רואים כעת על המסך.

לכן הרגל מקצועי ראשון הוא הרצה מחדש מתחילת המחברת בסדר, ללא שימוש במשתנים שנוצרו במקרה בהרצה קודמת. בפרויקט גמר התלמיד צריך להוכיח שעמיתו או בוחן יכול לפתוח את המחברת ולשחזר את התוצאה.

דוגמה 1 — התא הראשון: שמירת נתונים והצגתם

```
# Notebook cell 1 – measurements
computer_name = "Dudu-PC-01"
temperature = 72.5
print(computer_name)
print(temperature)
```

בפקודה הראשונה הגדרנו משתנה שמחזיק טקסט. בשנייה הגדרנו משתנה שמחזיק מספר עשרוני. `print` אינו מאמן מודל ואינו משנה את הערכים: הוא רק מדפיס אותם כדי שנוכל לבדוק מה קרה.

לשאול לפני שמריצים איזה פלט יופיע? מה יקרה אם תא זה לא הורץ אבל תא אחר מנסה להשתמש ב־temperature? במקרה כזה תיתכן שגיאת NameError.

כיצד קוראים הודעת שגיאה?

כאשר שורה אינה פועלת, אין להעתיק מיד פתרון בלי להבין מה קרה. מתחילים מקריאת שורת השגיאה האחרונה: היא מתארת את סוג הבעיה. אחר כך מאתרים את השורה שאליה ההודעה מפנה ושואלים מה Python ציפה לקבל. שגיאה היא מידע על התנהגות התוכנית, לא רק כישלון של המתכנת.

קטע לדיון בלבד — מדוע סדר הרצה חשוב?

```
# This will fail if temperature was never defined:
# print(temperature + 5)
```

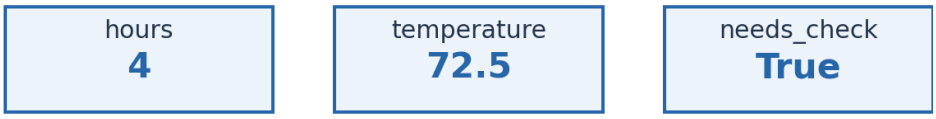
בדיקת הבנה

1. תארו שני מצבים שבהם מחברת נותנת פלט שונה בגלל סדר הרצת תאים.

2. כתבו תא שמגדיר שם מחשב, טמפרטורה ושעות שימוש; הדפיסו כל ערך בשורה משלו.

1.2 — משתנים, טיפוסים וחשבון מספרי

משתנה הוא שם שמאפשר לפנות לערך. ב־Python אין צורך להצהיר מראש על טיפוס המשתנה, אך לכל ערך יש טיפוס, והטיפוס קובע אילו פעולות אפשר לבצע עליו. מספר שלם (`int`) מתאים למשל למספר התקלות; מספר עשרוני (`float`) מתאים לטמפרטורה; מחרוזת (`str`) מתאימה לשם מחשב; ערך בוליאני (`bool`) מתאים לשאלה כן או לא.



Names refer to values; the value type matters

שלושה שמות משתנים והערכים שהם מפנים אליהם. הציור מדגיש שטיפוס הערך קובע את משמעותו.

טיפוס	דוגמה	משמעות בהקשר שלנו
int	faults = 3	מספר תקלות
float	temp = 72.5	טמפרטורה
str	name = "PC-01"	מזהה מחשב
bool	urgent = True	האם נדרש טיפול דחוף

דוגמה 2 — איך בודקים טיפוס?

```
faults = 3
temp = 72.5
name = "PC-01"
urgent = True

print(type(faults))
print(type(temp))
print(type(name))
print(type(urgent))
```

הסימן `=` מבצע השמה: לוקחים את התוצאה בצד ימין ומקשרים אותה לשם בצד שמאל. הוא אינו טענה מתמטית של שוויון. הביטוי `faults = faults + 1` תקין בתכנות: קודם מחשבים את מספר התקלות הקיים ועוד אחת, ואז מעדכנים את המשתנה.

דוגמה 3 — עדכון מצב

```
faults = 3
faults = faults + 1
print(faults)  # 4
```

אילו פעולות חשבון קיימות?

פעולה	דוגמה	תוצאה
+	7 + 2	9
-	7 - 2	5

פעולה	דוגמה	תוצאה
*	7 * 2	14
/	7 / 2	3.5
//	7 // 2	3
%	7 % 2	1
**	2 ** 3	8

שימו לב: `/` היא חלוקה רגילה ומחזירה מספר עשרוני גם כאשר החלוקה מדויקת, ואילו `//` היא חלוקת רצפה. עבור מספרים שליליים אין לפרש אותה תמיד כ״מחיקת החלק העשרוני״: למשל `-7 // 2` שווה -4. האופרטור `%` מספק שארית בהתאם לכללי Python ומשמש, בין היתר, לבדיקת זוגיות.

דוגמה פתורה: ממוצע ושליטה בסדר פעולות

דודו מדד שלוש טמפרטורות: 68, 82, 75. כדי לחשב ממוצע מחברים את הערכים ומחלקים במספרם. טעות נפוצה היא לכתוב `68 + 82 + 75 / 3`: לפי קדימות הפעולות רק המספר האחרון יחולק בשלוש.

$$\text{mean} = (68 + 82 + 75) / 3 = 75.0$$

דוגמה 4 — סוגריים מגנים על משמעות הנוסחה

```
t1, t2, t3 = 68, 82, 75
mean = (t1 + t2 + t3) / 3
print(mean)
```

למה זה חשוב ב־AI? אלגוריתם יוכל להשתמש בטמפרטורה ממוצעת בתור תכונה, אך תחילה עלינו לוודא שהחישוב מייצג את התכונה שהתכוונו אליה. מספר שהודפס ללא הודעת שגיאה עדיין עשוי להיות שגוי מבחינה מדעית.

תרגול מדורג

1. חשבו ביד ואז ב־Python את הממוצע של 60, 75, 90, 75.
2. הסבירו מה ההבדל בין `"5" + "2"` לבין `5 + 2`.
3. כתבו ביטוי המחזיר את מספר הדקות שנותרו לאחר חלוקת 137 דקות לשעות שלמות.
4. בדקו מה מחזיר `type(True)` ומדוע אין להשתמש ב־`True` כמדידה מספרית רגילה.

1.3 — מחרוזות, קלט והצגת תוצאות

הנתונים במערכת אינם רק מספרים. לפעמים צריך לייצג מזהה מחשב, שם כיתה, הודעת מצב או תאריך. כל אלה הם מחרוזות (`str`). מחרוזת היא רצף תווים: גם הספרות שבתוכה הן תווים ולא בהכרח מספרים שניתן לחבר ולממוצע. למשל `"72.5"` היא מחרוזת, בעוד `72.5` הוא מספר.

פונקציות כמו `float()` ו־`int()` מאפשרות להמיר טקסט לערכים מספריים כאשר הטקסט תקין. אם הוא מכיל מילה במקום מספר, ההמרה תיכשל; בהמשך נלמד כיצד לזהות ולטפל בכך.

דוגמה 5 — המרת טקסט ובדיקת תבנית

```
reading = "72.5"
value = float(reading)
print(value + 2)   # 74.5

computer = "PC-04"
print(computer.lower())
print(computer.startswith("PC-"))
```

פלט שקריא לאדם

הדפסת רשימת מספרים לבדה אינה מספרת לקורא מה המשמעות שלהם. לכן משתמשים במחרוזות מעוצבות (`f-string`), המאפשרות לשלב ערכים והסברים. הביטוי שבתוך הסוגריים המסולסלים מחושב לפני ההדפסה. ניתן לבקש שתי ספרות אחרי הנקודה כדי להציג ממוצע בצורה נוחה.

דוגמה 6 — תצוגה שאפשר להכניס לדוח

```
computer = "PC-04"
average = 75.333333
print(f"Computer: {computer}; average temperature: {average:.2f} C")
```

נבחין בין עיגול לצורך הצגה לבין שינוי הנתונים עצמם: הדפסת `75.33` אינה מוחקת את הדיוק של המשתנה `average`. חשוב לשמור את ערכי המקור, בייחוד לפני חישובי סטטיסטיקה או אימון מודל.

קלט מהמשתמש — מתי כן ומתי לא?

הפונקציה `input()` מאפשרת למשתמש להקליד ערך, אבל היא מחזירה מחרוזת. לכן לשם חישוב יש להמיר את הקלט. במחברות AI משתדלים לרוב להפריד בין קוד החישוב לבין קלט אינטראקטיבי, כדי שניתן יהיה להריץ את אותה מחברת באופן אוטומטי בלי להמתין למשתמש.

קוד לדיון — אינו מופעל אוטומטית במחברת

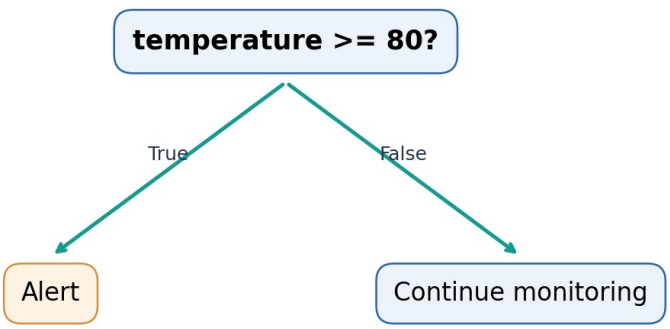
```
# Interactive example – run when you want to enter data:
# text = input("Temperature: ")
# temperature = float(text)
# print(temperature + 1)
```

בדיקת הבנה

1. כתבו שורת `f-string` המציגה מחשב וטמפרטורה עם ספרה אחת אחרי הנקודה.
2. הסבירו מדוע `int("4.5")` אינו מבצע אותה פעולה כמו `int(float("4.5"))`.
3. כתבו בדיקה שמזהה האם מזהה מחשב מתחיל באותיות `PC-`.

1.4 — תנאים ולוגיקה: מתיאור נתון להחלטה

מערכת לניטור מחשבים צריכה לעיתים לפעול אחרת לפי מצב המדידה. תנאי הוא ביטוי שמחזיר `True` או `False`. מבנה `if` מאפשר להריץ קוד רק כאשר התנאי מתקיים. החלק שמוזח פנימה שייך לבלוק התנאי; הזחה שגויה משנה את משמעות התוכנית או גורמת לשגיאה.



עץ החלטה פשוט: מדידה של 80 מעלות ומעלה מסומנת להתייחסות.

דוגמה 7 — החלטה בעזרת *if/else*

```
temperature = 82
if temperature >= 80:
    print("Temperature alert")
else:
    print("Temperature normal")
```

נוסחת ההחלטה אינה ״למידת מכונה״. היא כלל מפורש שנכתב על ידי המתכנת. ההבדל חשוב: בהמשך נלמד מודל שמנסה ללמוד כלל מנתוני עבר, אבל בשלב זה אנחנו עדיין כותבים את הכלל בעצמנו.

השוואות וצירופי תנאים

סימן	פירוש	דוגמה
==	שווה	temperature == 80
!=	שונה	faults != 0
>, >=	גדול / גדול או שווה	temperature >= 80
<, <=	קטן / קטן או שווה	faults <= 2
and	שני התנאים מתקיימים	temp > 80 and faults > 0
or	לפחות אחד מתקיים	temp > 90 or faults > 4
not	שלילת תנאי	not urgent

הביטוי `and` מחזיר אמת כאשר שני התנאים נכונים; `or` מחזיר אמת כאשר לפחות אחד נכון. בדוגמאות פשוטות הערכים הם בוליאניים, אך יש לזכור שב־Python אופרטורים אלה יכולים להחזיר גם אחד מהאופרנדים. כדי לא להקשות בתחילה נשתמש בביטויים שהשוואותיהם בוליאניות מפורשות.

דוגמה 8 — שלושה מצבים ולא רק שניים

```
temperature = 86
faults = 2
if temperature >= 80 and faults >= 2:
    status = "urgent"
elif temperature >= 80:
    status = "check temperature"
else:
    status = "normal"
print(status)
```

מקרה גבול דורש תשומת לב: האם 80 נחשב חריג או שהכלל חל רק מעל 80? תשובה כזו חייבת להופיע בהגדרת הבעיה. בקוד ההבדל הוא בין `>` לבין `>=`, אבל במערכת אמיתית מדובר בהחלטה עסקית או מדעית.

תרגול

1. עבור טמפרטורה 80, מה התוצאה של `temperature > 80` ומה של `temperature >= 80`?
2. כתבו תנאי שמזהה מחשב עם יותר משלוש תקלות או טמפרטורה של 90 ומעלה.
3. הסבירו מדוע כלל ידני בתנאי `if` אינו נחשב מודל שאומן על נתונים.

1.5 — לולאות, סכימה וספירה של נתונים

כאשר מתקבלות מאות מדידות, אי אפשר לכתוב פקודת חיבור נפרדת לכל מדידה. לולאה מאפשרת לבצע פעולה חוזרת לפי סדר על אוסף נתונים. נתחיל בלולאת `for` שמבקרת בכל מדידה, ולאחר מכן נכיר את `range` ואת לולאת `while` המתאימה למקרים שבהם ממשיכים עד שמתקיים תנאי עצירה.



Loop: visit each item and update a count or total

הלולאה עוברת לפי הסדר על ארבע מדידות; ערכים של 80 ומעלה מסומנים בצבע שונה.

דוגמה פתורה: סכום ומונה

דודו רוצה לחשב טמפרטורה ממוצעת וגם לדעת כמה מדידות הגיעו לסף 80. נצטרך שני משתנים: `total` יצבור את הסכום; `high_count` יספור מדידות חריגות. לפני תחילת הלולאה מאתחלים את שניהם באפס, כי עדיין לא עיבדנו אף מדידה.

דוגמה 9 — צבירה וספירה

```
temperatures = [68, 82, 76, 91]
total = 0
high_count = 0

for temperature in temperatures:
    total += temperature
    if temperature >= 80:
        high_count += 1

average = total / len(temperatures)
print(average)     # 79.25
print(high_count)  # 2
```

שלב	temperature	total לאחר עדכון	high_count
אתחול	—	0	0
1	6 8	6 8	0
2	8 2	150	1
3	76	226	1
4	9 1	3 1 7	2

טבלת המעקב חשובה יותר משינון התחביר: היא עוזרת להסביר מה נשאר נכון אחרי כל איטרציה. לאחר שני ערכים, למשל, `total` הוא סכום שני הערכים הראשונים ו־`high_count` הוא מספר הערכים שחצו את הסף מביניהם. הרעיון הזה יופיע בהמשך גם באלגוריתמים הלומדים מנתונים.

לולאה לפי אינדקס

דוגמה 10 — כאשר צריך גם את מיקום התצפית

```
for i in range(len(temperatures)):
    print(i, temperatures[i])
```

`range(len(...))` מחזיר אינדקסים מ־0 ועד אחד פחות ממספר האיברים. בהמשך, כאשר צריך לעבור על כמה מערכים מקבילים, חשוב לדעת מה מייצג האינדקס. אם לא צריך מיקום, נעדיף בדרך כלל מעבר ישיר על הערכים כדי לשמור על קוד פשוט.

מתי נשתמש ב־while?

דוגמה 11 — עם תנאי עצירה ברור *while*

```
index = 0
while index < len(temperatures):
    print(temperatures[index])
    index += 1
```

בלולאת `while` חובה לוודא שמשהו משתנה וכך מקדם אותנו לעבר תנאי העצירה. אם שכחנו `index += 1`, נקבל לולאה שאינה מסתיימת. ב־Colab אפשר לעצור את ההרצה, אבל עדיף ללמוד לזהות את הבעיה לפני שהיא מתרחשת.

זהירות: רשימה ריקה אם אין מדידות, `len(temperatures)` הוא אפס. חלוקה ב־0 תגרום ל־ZeroDivisionError. בהמשך נלמד טיפול בקלט חסר; כבר עכשיו חשוב להגדיר כיצד התוכנית צריכה להגיב.

תרגול העברה

1. חשבו ביד את הסכום ואת מספר המדידות מעל 85 ברשימה `[65, 86, 80, 92, 79]`.
2. כתבו לולאה המדפיסה רק מדידות בטווח 70–85 (כולל גבולות).
3. הסבירו מתי תעדיפו `for temperature in temperatures` ומתי תצטרכו `range(len(...))`.
4. תכננו שינוי שימנע חלוקה באפס במקרה של רשימה ריקה.

1.6 — רשימות, אינדקסים והעתקת נתונים

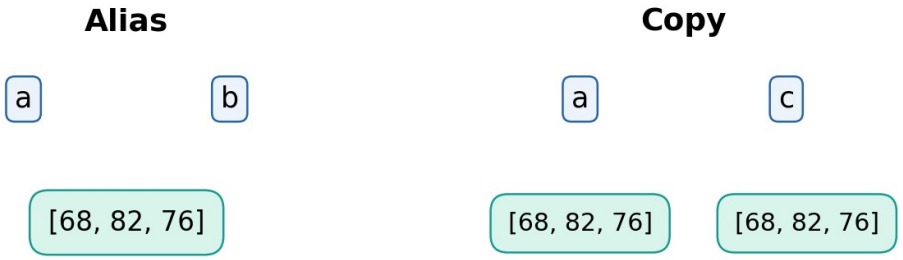
רשימה (`list`) מאפשרת לשמור כמה ערכים באוסף מסודר. האינדקסים מתחילים באפס, ולכן האיבר הראשון נמצא ב־`temperatures[0]`. רשימה ניתנת לשינוי: אפשר להוסיף איבר, להחליף איבר ולמחוק איבר. תכונה זו שימושית, אך גם דורשת זהירות כדי לא לשנות נתוני מקור בלי כוונה.

דוגמה 12 — גישה, חיתוך והוספה

```
temperatures = [68, 82, 76, 91]
print(temperatures[0])    # first value
print(temperatures[-1])   # last value
print(temperatures[1:3])  # values at indexes 1 and 2

temperatures.append(74)
print(len(temperatures))  # 5
```

בביטוי `[1:3]` הגבול השמאלי כלול והגבול הימני אינו כלול. לכן נקבל את האיברים במקומות 1 ו־2. חיתוך של רשימה יוצר רשימה חדשה; לעומת זאת, השמה `b = a` יוצרת שם נוסף לאותו אובייקט רשימה. ההבדל יכול ליצור טעויות שקשה לאתר בניסויי AI.



שני שמות יכולים להפנות לאותה רשימה; העתקה יוצרת רשימה עצמאית ברמה הראשונה.

דוגמה 13 — כינוי לעומת העתקה

```
original = [68, 82, 76]
alias = original
copy_of_values = original.copy()

alias[0] = 999
print(original)       # [999, 82, 76]
print(copy_of_values) # [68, 82, 76]
```

`copy()` יוצרת העתקה שטחית. עבור רשימה של מספרים זה מספיק בדרך כלל. אם הרשימה כוללת רשימות פנימיות, השינויים ברשימות הפנימיות עדיין עשויים להשפיע על שני המבנים. נרחיב על מבנים מקוננים רק לאחר שההבדל הבסיסי ברור.

משמעות ל־AI: לשמור נתוני מקור

נניח שתלמיד מחליף ברשימה את כל המדידות שמעל 80 בערך 80. זו פעולה שמתקנת לכאורה קלט חריג, אבל היא גם מוחקת מידע. אם לא נשמר עותק של נתוני המקור, קשה להבין מה בוצע וייתכן שנאמן בהמשך מודל על נתונים מעוותים. כל שלב בעיבוד נתונים צריך להיות מתועד ובעל הצדקה.

תרגול

1. הסבירו מה תחזיר הפקודה `temperatures[-2]` כאשר הרשימה כוללת ארבעה איברים.
2. כתבו קוד היוצר עותק של רשימת מדידות וממיר רק בעותק ערכים שליליים ל־0.
3. מדוע השורה `b = a` אינה מגינה על רשימת המקור?
4. מה יקרה אם ננסה לגשת ל־`temperatures[100]`? איך נמנעים מכך?

1.7 — מילונים, רשומות ומערכי נתונים פשוטים

כאשר לכל מחשב יש יותר ממאפיין אחד, לא נוח לשמור כמה רשימות מקבילות בלי הסבר. מילון (`dict`) שומר קשרים בין מפתחות לערכים, למשל `"name"` למזהה המחשב ו־`"temperature"` למדידה. רשימת מילונים מאפשרת לייצג אוסף רשומות, שכל אחת מהן מתארת תצפית אחת.

דוגמה 14 — תצפית כמילון

```
computer = {
    "name": "PC-01",
    "temperature": 72.5,
    "hours": 4,
    "faults": 1
}
print(computer["temperature"])
print(computer.get("location", "unknown"))
```

הגישה `computer["temperature"]` מניחה שהמפתח קיים; אם אינו קיים נקבל `KeyError`. לעומת זאת, `get` מאפשר להחזיר ערך חלופי כאשר המפתח חסר. אין לפרש ערך חסר כאפס באופן אוטומטי: אפס תקלות הוא נתון, ואילו ״אין לנו מידע על מספר תקלות״ הוא מצב אחר לגמרי.

דוגמה 15 — רשימה של תצפיות

```
computers = [
    {"name": "PC-01", "temperature": 72, "faults": 1},
    {"name": "PC-02", "temperature": 87, "faults": 3},
    {"name": "PC-03", "temperature": 76, "faults": 0}
]
```

```
for pc in computers:
    print(pc["name"], pc["temperature"])
```

מה ההבדל בין מפתח, מאפיין ותווית?

המפתח במילון הוא שם של שדה, למשל `temperature`. המאפיין (Feature) הוא ערך שמשתמשים בו כדי לתאר את התצפית בזמן החיזוי. התווית או משתנה המטרה (Target) היא התשובה שהמודל יתבקש לחזות, אם קיימת. בחירת תכונה אינה רק החלטה טכנית: אסור לכלול בין תכונות החיזוי מידע שלא יהיה זמין כאשר המערכת תידרש לפעול.

לדוגמה, אם רוצים לחזות האם מחשב יתקלקל מחר, מספר התקלות שיתרחשו בשבוע הבא אינו מאפיין לגיטימי של היום. בהמשך נקרא לבעיה של שימוש במידע עתידי בשם "זליגת נתונים" (Data Leakage).

היכרות שימושית — Set־ו Tuple

בנוסף לרשימה ולמילון יש שני מבנים נפוצים: `tuple` הוא רצף מסודר שאינו ניתן לשינוי לאחר יצירתו (אם כי אובייקטים משתנים בתוכו עשויים להשתנות), ו־`set` הוא אוסף ללא סדר אינדקסים קבוע וללא כפילויות. לא נעמיס כאן ניתוח מעמיק; התלמיד צריך לזהות אותם בקוד ולבחור מבנה נתונים מתאים.

דוגמה 16 — *tuple* לממדים ו־*set* לזיהוי ערכים ייחודיים

```
resolution = (8, 8)  # two dimensions
labels = ["ok", "alert", "ok", "check"]
unique_labels = set(labels)
print(resolution)
print(unique_labels)
```

תרגול

1. תכננו רשומת מחשב עם שדות `id`, `hours`, `temperature`, `status`.
2. הסבירו מדוע מידע חסר על תקלות אינו שקול לערך 0.
3. מה תהיה משמעות תווית `needs_repair` בפרויקט סיווג? מתי היא נודעת?
4. כתבו לולאה שמדפיסה את שמות המחשבים שהטמפרטורה שלהם 80 ומעלה.

1.8 — פונקציות: להפוך חישוב לפעולה ברורה ואמינה

תוכנית אמיתית מורכבת מכמה פעולות החוזרות במקומות שונים: בדיקת מדידות, חישוב ממוצע, יצירת סיכום, שמירת קובץ. אם נעתיק את אותן שורות שוב ושוב, כל תיקון יחייב אותנו למצוא את כל העותקים. פונקציה מאפשרת לתת לפעולה שם, להגדיר קלטים ולהחזיר תוצאה. זהו אחד המעברים החשובים מכתיבת פקודות בודדות לכתיבת מערכת שניתן לתחזק.



פונקציה מקבלת קלט ומחזירה תוצאה: אותו חישוב ניתן להפעלה על מדידות שונות.

דוגמה 17 — קלט, שם פעולה וערך מוחזר

```
def average_temperature(values):
    """Return the average of a non-empty sequence."""
    return sum(values) / len(values)

readings = [68, 82, 76]
result = average_temperature(readings)
print(result)
```

המילה `def` מגדירה פונקציה; `values` הוא פרמטר מקומי, שמקבל בזמן הקריאה את הרשימה שהעברנו; `return` מחזיר את התוצאה למקום שממנו קראנו לפונקציה. אם היינו כותבים `print` במקום `return`, הפונקציה הייתה מציגה ערך למסך אבל לא מאפשרת להשתמש בו בנוחות בחישובים הבאים.

הפרדה בין חישוב להצגה

דוגמה 18 — פונקציה בוליאנית וערך ברירת מחדל

```
def is_hot(temperature, threshold=80):
    return temperature >= threshold

print(is_hot(82))
print(is_hot(82, threshold=90))
```

הדוגמה גם מלמדת על ערך ברירת מחדל: אם המשתמש אינו מעביר סף, משתמשים ב־80; אם מעביר סף אחר, נקבל החלטה בהתאם אליו. חשוב לא לשנות ערכי סף סתם כדי להתאים תוצאה רצויה, אלא לקבוע אותם לפי הגדרת הבעיה ולתעד את ההחלטה.

בדיקות וקלט לא תקין

הפונקציה הקודמת לחלוקה בממוצע הניחה שהרשימה אינה ריקה. אפשר להפוך את ההנחה למפורשת ולשלוח הודעת שגיאה ברורה כאשר אין נתונים. לא תמיד צריך ״לתקן״ קלט לא תקין במקום; לעיתים נכון לעצור את החישוב כדי למנוע הפקת תוצאה מטעה.

דוגמה 19 — הגדרת תנאי תקינות לפונקציה

```
def average_temperature(values):
    if len(values) == 0:
        raise ValueError("Expected at least one temperature")
    return sum(values) / len(values)
```

משתנים שנוצרים בתוך הפונקציה אינם בהכרח זמינים מחוץ לה. זהו רעיון תחום ההכרה של משתנים (Scope): הוא מונע תלות מיותרת בין חלקי תוכנית. פרק התכנות מונחה העצמים יתבסס על היכולת הזאת להפריד אחריות בין רכיבים.

בדיקות עם assert

דוגמה 20 — שלוש בדיקות אוטומטיות קטנות

```
assert average_temperature([60, 80]) == 70
assert is_hot(80) is True
assert is_hot(79) is False
```

בדיקה טובה כוללת גם מקרה רגיל וגם מקרה גבול. עם זאת, `assert` איננו מנגנון אימות קלט למערכת ייצור: ניתן לבטל אותו בהרצות Python מסוימות. כאן הוא כלי לימודי מהיר לבדיקת נכונות של פונקציות. בהמשך נלמד לכתוב בדיקות מסודרות.

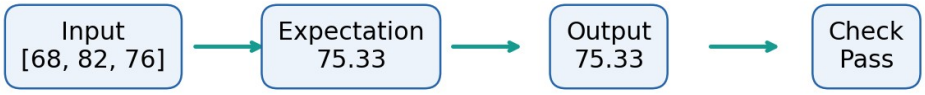
תרגול

1. כתבו פונקציה `count_hot(values, threshold=80)` המחזירה את מספר המדידות שעומדות בסף.
2. הסבירו מדוע פונקציה שמחזירה מספר עדיפה לעיתים על פונקציה שרק מדפיסה אותו.
3. הגדירו שלוש בדיקות: רשימה רגילה, רשימה שבה אף מדידה אינה חריגה ומקרה גבול בדיוק בסף.
4. תכננו מה הפונקציה אמורה לעשות כשהרשימה ריקה; נמקו את הבחירה.

1.9 — שגיאות, מודולים, קבצים ושחזור

תלמיד שמפתח מערכת AI חייב ללמוד לא רק לכתוב תוכנית שעובדת בדוגמה אחת, אלא גם לברר מדוע תוכנית אינה עובדת או מחזירה תוצאה חשודה. נהוג להבחין בין שגיאת תחביר (`SyntaxError`), שגיאת זמן ריצה כגון `IndexError` או `ValueError`, ושגיאת לוגיקה שבה הקוד רץ אך מחשב דבר אחר מהנדרש.

דרך בדיקה	דוגמה	סוג הבעיה
קריאת שורת השגיאה ותיקון הסימן	חסר : אחרי if	תחביר
בדיקת אורך הרשימה והאינדקס	גישה לאינדקס לא קיים	זמן ריצה
חישוב ידני והשוואה לפלט	ממוצע ללא סוגריים	לוגיקה
בירור מדיניות ערכים חסרים	טמפרטורה חסרה	נתונים



כל בדיקה משווה פלט שהתקבל לפלט צפוי, ולא רק בודקת שהקוד סיים לרוץ.

למה צריך import?

Python כוללת ספרייה תקנית עשירה, לצד ספריות חיצוניות. באמצעות `import` ניתן להשתמש בפונקציות שהוגדרו במודולים אחרים. `math` מספקת פעולות מתמטיות, `csv` קריאה של קבצי CSV, ו־`pathlib` טיפול בנתיבי קבצים. מודולים כגון `numpy` ו־`pandas` הם ספריות חיצוניות שמשמשות במיוחד במדעי הנתונים.

דוגמה 21 — מודולים מספרייה תקנית

```
from pathlib import Path
from math import sqrt

folder = Path(".")
print(folder.resolve())
print(sqrt(81))
```

קריאת CSV — לא להניח שכל קובץ זהה

קובץ CSV מייצג נתונים טבלאיים באמצעות טקסט. שורה אחת עשויה להכיל כותרות, והשורות הבאות ערכי תצפיות. אך אין להניח שהנתונים תקינים: ייתכנו שדות ריקים, מפרידים שונים ומספרים שנשמרו כטקסט. בפרק EDA נעמיק בבדיקות ובניקוי; כאן נלמד לזהות את המבנה ולקרוא קובץ בצורה בסיסית.

דוגמה 22 — יצירה וקריאה של *CSV* קטן

```
import csv
from pathlib import Path

path = Path("computers_demo.csv")
with path.open("w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["name", "temperature", "hours"])
    writer.writerow(["PC-01", 72, 4])
    writer.writerow(["PC-02", 87, 8])

with path.open("r", newline="", encoding="utf-8") as f:
    rows = list(csv.DictReader(f))
print(rows)
```

הערכים ש־`csv.DictReader` קוראת הם בדרך כלל מחרוזות, ולכן לפני חישוב ממוצע יש לבצע המרות ולבדוק תקינות. זוהי דוגמה נוספת להבחנה בין סוג הנתון כפי שנשמר בקובץ לבין המשמעות המספרית שלו. בפרויקט גמר יש לשמור מקור וגרסת קובץ ולא לדרוס אותו ללא צורך.

בדיקה מול מקרה מוכר

דוגמה 23 — התאמה בין תוצאה ידנית למימוש

```
def calc_average(values):
    if not values:
        raise ValueError("No values")
    return sum(values) / len(values)

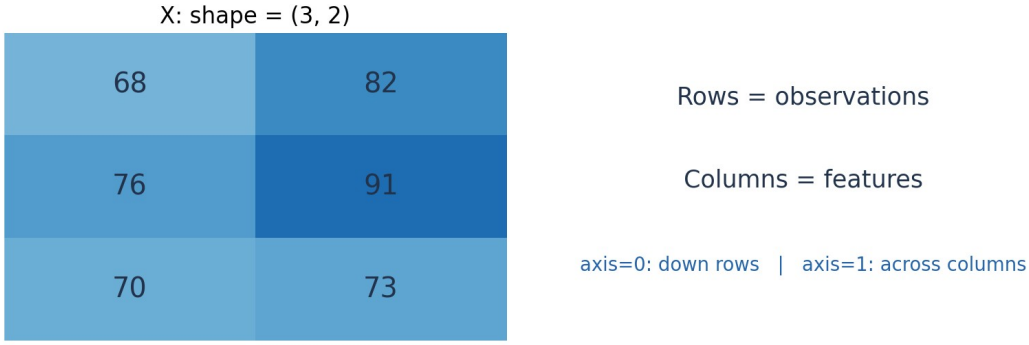
assert calc_average([68, 82, 76]) == 226 / 3
```

תרגול

1. תארו שגיאת לוגיקה שעלולה לעוות ממוצע גם כאשר אין הודעת שגיאה.
2. שנו את דוגמת ה־CSV כך שתכיל שלושה מחשבים ובדקו מה סוג הערכים שנקראו.
3. מדוע חשוב לרשום את מקור קובץ הנתונים ואת קוד ההכנה שלו בפרויקט AI?
4. תכננו בדיקה למקרה שבו קובץ אינו נמצא בתיקייה הרצויה.

1.10 — NumPy: מעבר מרשימות למערכים מספריים

רשימות Python נוחות לצעדים הראשונים, אך בעיבוד נתונים מספריים אנו עובדים לעיתים על אלפי ערכים, מטריצות ותמונות. NumPy מאפשרת לייצג מערכים רב־ממדיים בעלי טיפוס אחיד ולהפעיל עליהם פעולות וקטוריות. המשמעות היא לא רק כתיבה קצרה יותר: הצורה (`shape`) של המערך אומרת כמה תצפיות וכמה מאפיינים יש לנו.



מטריצה X בעלת שלוש שורות ושני מאפיינים. הצורה קובעת מה מייצג כל ציר.

דוגמה 24 — שלוש תצפיות ושני מאפיינים

```
import numpy as np

X = np.array([
    [68, 4],
    [82, 7],
    [76, 5]
], dtype=float)

print(X.shape)         # (3, 2)
print(X[:, 0])        # first feature for all computers
print(X.mean(axis=0)) # feature means
```

כאשר `axis=0` מפעילים חישוב לאורך שורות, כך שמתקבלת תוצאה לכל עמודה; ממוצע שתי העמודות בדוגמה הוא 75.33 לטמפרטורה ו־5.33 לשעות שימוש. כאשר `axis=1`, מתקבלת תוצאה לכל שורה. לא כדאי לחשב ממוצע של טמפרטורות ושעות יחד: היחידות שלהן שונות, והתוצאה אינה מדד בעל משמעות מוגדרת.

פעולות איבר־איבר ו־Broadcasting

דוגמה 25 — אותה פעולה על כל איבר

```
temperatures = np.array([68, 82, 76], dtype=float)
shifted = temperatures + 2
scaled = temperatures / 100
print(shifted)
print(scaled)
```

כאשר מוסיפים 2 למערך, NumPy מחילה את החיבור על כל איבר. לכן אפשר לחשוב על ההוראה כעל לולאה נסתרת, אך בביצוע המקצועי זו פעולה מערכית. Broadcasting מערכים בעלי צורות מתאימות יכולים להשתתף באותה פעולה בלי ליצור במפורש את כל העותקים. בפרקי הווקטורים ו־KNN שבספר הגדול נעמיק במנגנון ובתנאי ההתאמה בין ממדים.

דוגמה: הכנת מאפיינים לפני מודל

דוגמה 26 — מהו וקטור תכונות?

```
X = np.array([
    [68, 4],
    [82, 7],
    [76, 5]
], dtype=float)

feature_means = X.mean(axis=0)
print(feature_means)
print(X[0])
```

כל שורה ב־X מייצגת מחשב אחד וכל עמודה מאפיין. בעתיד נוסיף מערך `y` שבו נשמרות התוויות הידועות, אם מדובר בלמידה מונחית. חשוב לשמור תמיד על התאמה בין סדר השורות ב־X לבין סדר התוויות ב־y; ערבוב סדר אחד מהם ישבש את הלמידה גם אם הקוד רץ.

מיני־פרויקט מסכם: ניטור מעבדת המחשבים

דודו רוצה לקבל דוח קצר על מצב המעבדה. הוא צריך לוודא שיש נתונים, לחשב טמפרטורה ממוצעת, לספור מדידות שעברו סף, לכתוב פונקציה נפרדת לקבלת החלטה ולהכין מטריצת מאפיינים. זהו תהליך עיבוד נתונים מבוסס כללים; הוא עדיין אינו מודל למידת מכונה, משום שהמחשב לא למד את הכלל מתוך דוגמאות מסומנות.



Rule-based analysis is not yet a trained machine-learning model

תהליך מיני־הפרויקט: איסוף, בדיקה, סיכום, סימון והסבר התוצאה.

דוגמה 27 — מימוש מודרך של שלבי סיכום הנתונים

```
import numpy as np

readings = [68, 82, 76, 91]

def summarize(values, threshold=80):
    if len(values) == 0:
        raise ValueError("At least one reading is required")
    average = sum(values) / len(values)
    high_count = sum(value >= threshold for value in values)
    return average, high_count

average, high_count = summarize(readings)
print(f"Average: {average:.2f}; high readings: {high_count}")

X = np.array(readings, dtype=float).reshape(-1, 1)
print("Feature matrix shape:", X.shape)
```

`reshape(-1, 1)` משנה רשימה חד־ממדית למטריצה בעלת עמודה אחת. הסימון -1 מבקש מ־NumPy לחשב את מספר השורות הנדרש על פי כמות האיברים. עיבוד הנתונים כאן פשוט בכוונה, כדי שכל תלמיד יוכל להסביר אותו. בשלב מתקדם יותר נחליף את סף הכלל במודל שמאומן מנתונים, ונבדוק אותו על קלט שלא שימש לאימון.

משימת סיכום עצמאית — בלי קוד פתרון

במעבדת Colab המלווה את הפרק תבנו דוח למערכת ספרייה או למערכת ניטור אנרגיה על בסיס אותו רעיון, אך עם נתונים ושאלות אחרות. המחברת כוללת תאים להשלמת קוד, מקרים לבדיקה ושאלות לנימוק. אל תעתיקו את דוגמת דודו כפי שהיא: מטרת המשימה היא להוכיח שאתם יכולים להעביר את המושגים להקשר חדש.

שאלות חשיבה בסגנון בחינת ידע והגנה

1. דודו אומר ש־`X.shape == (4, 1)` פירושו ארבעה מאפיינים ותצפית אחת. האם הוא צודק? נמקו.

2. הסבירו כיצד ניתן לקבל תוכנית שרצה ללא שגיאות אך מפיקה תוצאה בלתי תקפה מבחינה מדעית.
3. מה ההבדל בין בדיקת כלל סף שנכתב ידנית לבין אימון מודל שלומד סף או גבול החלטה מנתונים?
4. כיצד הייתם מתעדים שינוי באופן חישוב הממוצע כדי לאפשר שחזור של ניסוי?

תכנית הוראה מוצעת ל־30 שעות

מפגש	מטרת ביצוע	תוצר אישי
1	Colab ומשתנים	מחברת נפתחת מתחילתה ומפיקה אותו פלט
2	טיפוסים וחישובים	ממוצע ויחידות מדידה מוסברים
3	מחרוזות וקלט	הצגת דוח קריא והמרת סוגי נתונים
4	תנאים	כלל החלטה ומקרי גבול
5	לולאות	סכימה וספירה עם טבלת מעקב
6	רשימות	גישה, חיתוך, העתקה ושמירת מקור
7	מילונים	רשומות בעלות שדות ברורים
8	פונקציות	פעולות עם return ובדיקות
9	שגיאות וקבצים	נקרא ונבדק CSV
10	NumPy ופרויקט	דוח מסכם ומטריצת מאפיינים

מילון מונחים מסכם

מונח	הסבר לתלמיד
Variable	שם שמתייחס לערך
Data type	הטיפוס שמגדיר אילו פעולות אפשריות על ערך
Condition	ביטוי שמוביל לענף פעולה
Loop	ביצוע חוזר של הוראות
List	אוסף מסודר וניתן לשינוי
Dictionary	אוסף של זוגות מפתח–ערך
Function	פעולה בעלת שם, פרמטרים ותוצאה אפשרית
Return	החזרת ערך אל קוד שקרא לפונקציה
Exception	אירוע שגיאה בזמן ריצה
Module	קובץ/יחידה המכילים קוד שניתן לייבא
NumPy array	מבנה מערך מספרי רב־ממדי
Shape	מספר האיברים לאורך כל ציר
Feature	מאפיין המשמש לתיאור תצפית
Target	משתנה המטרה שמנסים לחזות
Data leakage	שימוש במידע שאינו לגיטימי בזמן האימון או הבחינה

מקורות והקשר לספר הגדול

עוגן תכנית: מאגר Data-Science-ML-Course, מסלול ההתמחות, פרק 1 — היכרות חוזרת עם שפת תכנות (30 שעות). מקורות ההשראה להסברים הם ספרי יסוד מוכרים, אך הניסוח, הדוגמאות, האיורים וקוד התרגול בחוברת מקוריים.

Eric Matthes, Python Crash Course, 3rd ed., No Starch Press (2022) — https://nostarch.com/Python-crash-course-3rd-edition

Al Sweigart, Automate the Boring Stuff with Python, 3rd ed., No Starch Press (2025) — https://nostarch.com/automate-boring-stuff-python-3rd-edition

/Python Software Foundation, The Python Tutorial — https://docs.python.org/3/tutorial

מיפוי ההשלמות במסלול שלנו — https://github.com/samiabo75/Data-Science-ML-Course/tree/main/01-Python-Basics/03-Deep-Learning-Python

לאחר חוברת זו נמשיך ל־פרק 2: תכנות מונחה עצמים (30 שעות), לפרק 3: מבוא למדעי הנתונים וללמידת מכונה (10 שעות) ולפרק 4: חקר נתונים — EDA (15 שעות). לאחר השלמת ארבעתם נחבר אותם לחוברת יסוד אחת ונמקם אותה בתחילת ספר הקורס המאוחד. אין כאן כפילות של לימוד האלגוריתמים והמתמטיקה שכבר פותחו בספר הגדול.

בדיקת עצמאות לפני המשך האם אתם מסוגלים להסביר מה מייצגת כל שורה בטבלת הנתונים, להראות איך חושב כל מספר, לאתר טעות בקוד, ולהריץ את המחברת מחדש ללא התערבות? אם כן — מוכנים לפרק התכנות מונחה העצמים.

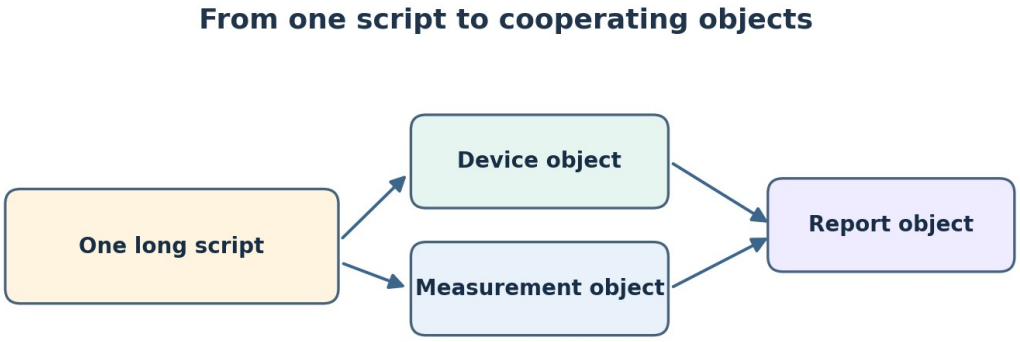
חוברת היסוד לספר הקורס המאוחד

פרק 2 — תכנות מונחה עצמים ב־Python

ממחלקה ועצם ועד למבנה של מערכת AI: הבנה, תכנון ומימוש

מסלול התמחות למידת מכונה ולמידה עמוקה | כיתות י״א–י״ב | מסגרת סילבוס: 30 שעות

מהדורת תלמיד • הסברים מדורגים • קוד עצמאי • המחשות מקוריות • תרגול והכנה לפרויקט



איור פתיחה: מעבר מסקריפט יחיד למספר עצמים בעלי תפקידים ברורים.

לפני שמתחילים — הסיפור שלנו

דודו אחראי על מעבדת המחשבים בבית הספר. עד היום הוא שמר את נתוני כל המחשבים ברשימות נפרדות: רשימת שמות, רשימת טמפרטורות ורשימת תקלות. כאשר נוספו מחשבים הוא התחיל להתבלבל: האם הטמפרטורה במקום השלישי שייכת למחשב השלישי בכל הרשימות? מה יקרה אם ימחק מחשב רק מרשימה אחת? כעת הוא רוצה לבנות תוכנה שאפשר להרחיב, לבדוק ולשלב בעתיד במערכת חיזוי תקלות מבוססת AI.

תכנות מונחה עצמים אינו אלגוריתם של למידת מכונה. זהו כלי לארגון תוכנה. הוא עוזר להפריד בין נתונים ופעולות, ליצור רכיבים שאפשר לבדוק בנפרד ולמנוע תלות מיותרת בין חלקי הפרויקט. בהמשך נשתמש באותן יכולות לכתיבת רכיבי קדם־עיבוד, מודלים ומערכות חיזוי.

מה תלמדו בפרק?

- להגדיר מחלקה, ליצור עצמים ולהבחין בין תכונה של עצם למשתנה רגיל.
- להסביר את התפקיד של self ושל __init__, ולכתוב פעולות שמבצעות חישוב אמיתי.
- להשתמש בהכלה ובהרכבה, ולהבחין ביניהן לבין ירושה ופולימורפיזם.
- להבין הפניה כפולה, העתקת עצמים, גישה מבוקרת לשדות וטיפול בקלט חריג.
- לבנות מערכת קטנה שבה רכיב נתונים ורכיב חיזוי פועלים יחד, ולבדוק את פעולתם.

תכנית הוראה מוצעת ל־30 שעות

תוצר תלמיד	נושאים ומיומנויות	מפגש (3 שעות)
מחלקת Device ראשונה	מהבעיה למחלקה, מצב והתנהגות	1
עצם עם חישוב ומצב	init__, self, get/set__ ופעולות	2
דוח על מספר מחשבים	פעולות פנימיות, רשימות עצמים	3
טבלת מעקב של עצמים	הפניות, העתקה וזהות	4
מערכת עצמים מקושרים	הכלה: Lab מכיל Device	5

תוצר תלמיד	נושאים ומיומנויות	מפגש (3 שעות)
	Measurement־ו	
מחלקה שמונעת מצב שגוי	כימוס, property ואימות נתונים	6
סוגי מחשבים ומימושים	ירושה, super ופולימורפיזם	7
קוד מאורגן וקובצי CSV	מודולים, קבצים וחריגות	8
Pipeline לימודי	רכיבי fit/transform/predict ובדיקות	9
SmartLab OOP	מיני־פרויקט, תיעוד והגנה בעל־פה	10

הקשר לשלוש תכניות הלימוד

דוגמה לשימוש	רמת הידע המכוונת	מסלול
קריאת נתוני מחשבים, סיכום והדפסתם	פונקציות, נתונים ומחברות Python בסיסיות	יחידה 3
עצם נתונים נושא וקטור מאפיינים	חישובים מתמטיים ואלגוריתמים (לא דרישת OOP עצמאית מזוהה)	נושאי ליבה — יחידה 5
Device, DataScaler, Predictor, Report	מחלקות, אחריות רכיבים, קוד מודלים ויישום תוכנה	התמחות למידה עמוקה

עוגן: במפת תכנית ההתמחות שבמאגר הקורס, פרק 2 — תכנות מונחה עצמים, 30 שעות. החלוקה למפגשים, דוגמת המעבדה וההקשרים ל־AI הם התאמה פדגוגית שלנו ולא חלוקה רשמית מאומתת של הסילבוס.

2.1 מדוע צריך תכנות מונחה עצמים?

נתאר שלושה מחשבים באמצעות שלוש רשימות: שמות, טמפרטורות ונתוני תקלות. כדי להפיק דוח על המחשב הראשון ניגש לאינדקס 0 בשלוש הרשימות. כל עוד הרשימות תמיד באותו אורך ובאותו סדר, התוכנית עובדת. אבל בפועל שינוי קטן — הוספה, מחיקה או מיון — עלול לשבור את ההתאמה ביניהן. הבעיה אינה שהרשימות הן כלי גרוע; הבעיה היא שאנחנו מייצגים ישות אחת באמצעות נתונים מפוזרים ללא הגדרה אחת שמאגדת אותם.

```
names = ["PC-01", "PC-02", "PC-03"]
temperatures = [58, 73, 64]
failures = [0, 2, 1]
for i in range(len(names)):
    print(names[i], temperatures[i], failures[i])
```

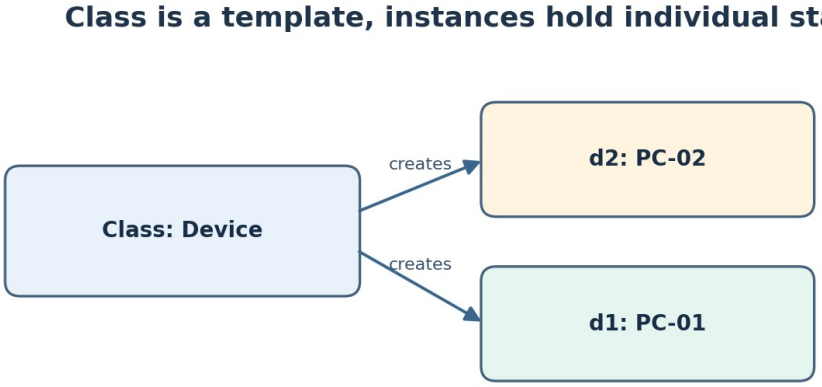
אפשר להחליף את שלוש הרשימות ברשימה של מילונים, וזה כבר שיפור. בתכנות מונחה עצמים עושים צעד נוסף: מגדירים מחלקה שמכילה את נתוני המחשב וגם פעולות שקשורות אליו. לדוגמה, במקום שקוד חיצוני יחליט בכל מקום אם הטמפרטורה חריגה, המחלקה עצמה יכולה להציע פעולה בשם needs_check. כך הכלל יוגדר פעם אחת ויוכל לשמש כמה חלקים בתוכנית.

☐ בדיקת הבנה: איזו בעיה עלולה להתרחש אם נוסיף שם של מחשב חדש אך נשכח להוסיף ערך לרשימת הטמפרטורות?

2.2 מחלקה, עצם, תכונות והתנהגות

מחלקה (Class) היא תיאור כללי של סוג עצמים; עצם (Object, או Instance) הוא מופע מסוים שנוצר על פי התיאור. אפשר לחשוב על מחלקה כתבנית להכנת כרטיסי מחשב: כל כרטיס מכיל אותם סוגי פרטים,

אבל הערכים בכל כרטיס שונים. התכונות (Attributes) מתארות את המצב של העצם; המתודות (Methods) מתארות מה הוא יודע לעשות.



איור 2.1: מחלקה אחת משמשת ליצירת כמה עצמים, ולכל עצם נתונים משלו.

```
class Device:
    def __init__(self, name, temperature):
        self.name = name
        self.temperature = temperature

    def needs_check(self):
        return self.temperature >= 70

computer_a = Device("PC-01", 58)
computer_b = Device("PC-02", 73)
print(computer_a.name, computer_a.needs_check())
print(computer_b.name, computer_b.needs_check())
```

כאשר מופיעה ההוראה Device("PC-01", 58), פייתון יוצרת מופע חדש ומפעילה את __init__ שלו. המופע שמתקבל נשמר במשתנה computer_a. שתי ההפניות computer_a ו־computer_b מצביעות על שני עצמים שונים, ולכן שינוי הטמפרטורה של אחד מהם אינו משנה את האחר.

דוגמה מהמעבדה	משמעות	רכיב בקוד
תבנית של כרטיס מחשב	הגדרה כללית של סוג העצם	class Device
מחשב מסוים	משתנה המחזיק הפניה לעצם	computer_a
שם המחשב	תכונה השייכת למופע	self.name
בדיקת סף טמפרטורה	פעולה של המופע	needs_check()

חשוב — הפעולה needs_check בדוגמה היא כלל תכנותי קבוע, ולא מודל שלמד מנתונים. בפרקים מתקדמים נחליף את הכלל הפשוט במודל שנאמן על נתוני אמת.

2.3 מה עושה __init__ ומה פירוש self?

הביטוי __init__ נקרא לעיתים "בנאי" בשפת ההוראה. ליתר דיוק, זוהי מתודת אתחול: היא מופעלת לאחר יצירת העצם ומגדירה את מצבו הראשוני. הפרמטר self מציין את המופע הנוכחי. כאשר קוראים computer_a.needs_check(), פייתון מעבירה את המופע בתור הפרמטר הראשון של המתודה — גם אם איננו כותבים אותו במפורש בעת הקריאה.

השוו בין name בלי self לבין self.name. השם name הוא פרמטר מקומי שקיים בזמן הפעלת __init__. לעומתו self.name נשמר כתכונה של המופע, ולכן אפשר לגשת אליו גם בהמשך, לאחר שהפעולה הסתיימה. ההבדל הוא כמו ההבדל בין פתק זמני לבין נתון ששמור בכרטיס של אותו מחשב.

```
class Device:
    def __init__(self, name, temperature=25):
        self.name = name
        self.temperature = temperature

    def description(self):
        return f"{self.name}: {self.temperature} C"

first = Device("PC-01")
second = Device("PC-02", 65)
print(first.description())
print(second.description())
```

בדוגמה הוגדר ערך ברירת מחדל temperature=25. לכן אפשר ליצור מחשב בלי למסור טמפרטורה, אך בכל זאת חייבים למסור שם. ערך ברירת מחדל צריך להתאים למשמעות הנתון; במערכת אמיתית עדיף לעיתים לציין שהמדידה אינה ידועה מאשר להמציא מדידה של 25 מעלות.

טבלת מעקב — יצירת שני עצמים

שלב	הוראה	התוצאה במצב העצמים
1	first = Device("PC-01")	first.name=PC-01, first.temperature=25
2	second = Device("PC-02",65)	second.name=PC-02, second.temperature=65
3	second.temperature = 72	רק second.temperature משתנה ל-72
4	first.description()	מחזירה את תיאור העצם הראשון

□ שאלת העברה: מה יקרה אם בתוך description נכתוב temperature במקום self.temperature? הסבירו לפי תחום ההכרה של המשתנים.

2.4 פעולות פנימיות: חישוב שמשתמש במצב העצם

מתודה אינה חייבת רק להדפיס נתון. היא יכולה לחשב תוצאה על סמך המצב של העצם או לעדכן אותו. נרצה למשל להוסיף קריאה חדשה של הטמפרטורה ולחשב את ההפרש מן המדידה הקודמת. כדי לעשות זאת נגדיר פעולה עם פרמטר חדש ונבחין בין ערך שהפעולה מחזירה לבין שינוי שהיא מבצעת במופע.

```
class Device:
    def __init__(self, name, temperature):
        self.name = name
        self.temperature = temperature

    def update_temperature(self, new_value):
        change = new_value - self.temperature
        self.temperature = new_value
        return change

pc = Device("PC-01", 60)
print(pc.update_temperature(68))
print(pc.temperature)
print(pc.update_temperature(65))
```

בקריאה הראשונה change הוא 8, והטמפרטורה נשמרת כ־68. בקריאה השנייה change הוא 3 −, כי הערך החדש קטן מן הקודם. חשוב לשים לב לסדר הפעולות: מחשבים את ההפרש לפני עדכון התכונה. אם נעדכן תחילה, החישוב ייתן תמיד אפס.

כיצד מתכננים מתודה?

לפני כתיבת כל מתודה מומלץ לענות על ארבע שאלות: מה היא מקבלת? מה היא משנה? מה היא מחזירה? ומה יקרה בקלט לא תקין? תכנון מראש מונע מתודות שמבצעות יותר מדי תפקידים ומקשה פחות על בדיקות היחידה.

שאלה	עבור **update_temperature**
קלט	טמפרטורה חדשה — new_value
מצב לפני	self.temperature
שינוי	עדכון self.temperature
פלט	ההפרש מן המדידה הקודמת

2.5 מספר עצמים — רשימת מחשבים ולולאות

מעבדה כוללת יותר ממחשב אחד. לכן ניצור רשימה של עצמי Device, נעבור עליהם בלולאה ונבנה דוח. המושג המרכזי הוא שהרשימה מחזיקה הפניות לעצמים; כל עצם אחראי לחשב האם הוא דורש בדיקה, ואילו הדוח אחראי לאסוף את התוצאות של כולם.

```
class Device:
    def __init__(self, name, temperature):
        self.name = name
        self.temperature = temperature

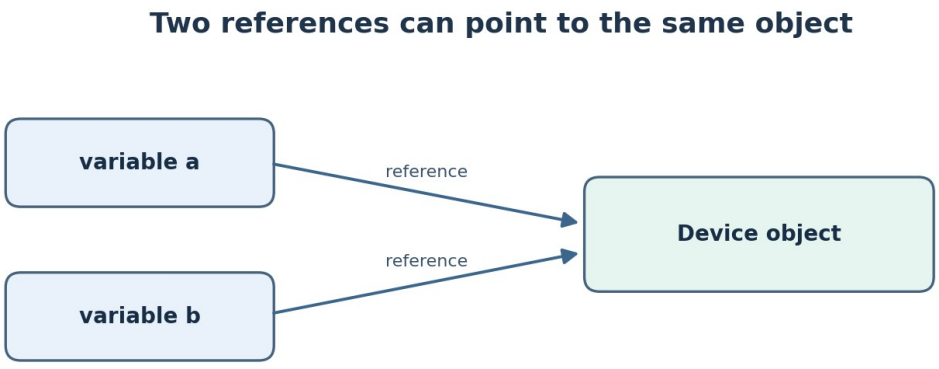
    def needs_check(self):
        return self.temperature >= 70

computers = [Device("PC-01", 61), Device("PC-02", 77),
             Device("PC-03", 72)]
for computer in computers:
    if computer.needs_check():
        print("CHECK:", computer.name)
```

בדוגמה הפעולה needs_check שייכת למחלקה Device, והלולאה רק מפעילה אותה. בעתיד נוכל להחליף את הכלל הפנימי בלי לשנות את לולאת הדוח. זו דוגמה ליתרון חשוב של OOP: מקומיות של שינוי.

2.6 זהות, הפניה כפולה והעתקה של עצמים

כאשר מבצעים b = a לא נוצרת אוטומטית העתקה של העצם. מתקבלת הפניה נוספת לאותו עצם, ולכן שינוי באמצעות b ייראה גם דרך a. זה שונה מעבודה עם מספרים פשוטים, שבהם המשתנה מקבל ערך, ולא עצם שניתן לשנות את שדותיו. כדי לבנות מערכות אמינות חשוב להבחין בין אותו עצם לבין עצם אחר בעל נתונים זהים.



איור 2.2: שני משתנים שונים מצביעים לאותו עצם בזיכרון.

```
class Device:
    def __init__(self, name):
        self.name = name

a = Device("PC-01")
b = a
b.name = "PC-99"
print(a.name)
print(a is b)

c = Device(a.name)
print(a is c)
print(a.name == c.name)
```

התוצאה a.name היא PC-99, מפני ששינוי דרך b שינה את אותו המופע. הביטוי a is b מחזיר True, כי זו אותה זהות. לעומת זאת c הוא מופע חדש, ולכן a is c מחזיר False גם אם לשניהם אותו שם. השוואת התכונות שלהם, לעומת זאת, יכולה להחזיר True. זהות אינה זהה לשוויון ערכים.

העתקה שטחית לעומת העתקה עמוקה

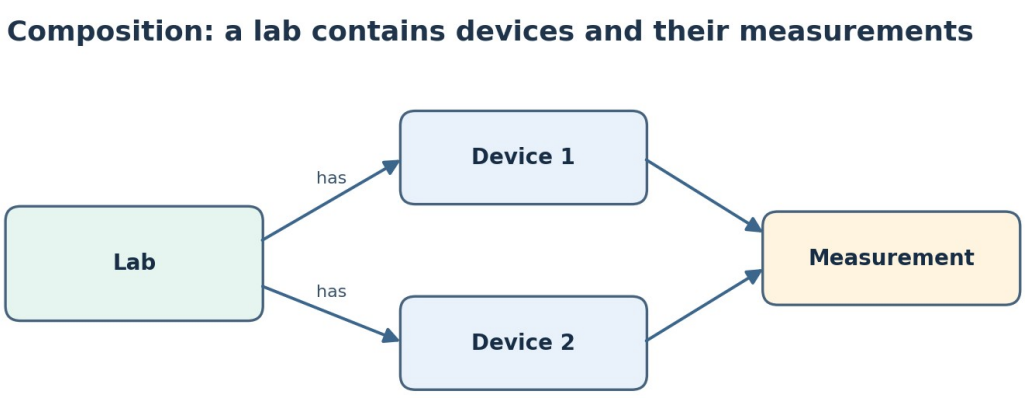
כאשר לעצם יש שדות פשוטים כמו מספר ומחרוזת, אפשר לעיתים ליצור עצם חדש באמצעות בנאי שמקבל את אותן תכונות. כאשר עצם מכיל רשימות או עצמים נוספים, העתקה מורכבת יותר: copy.copy מעתיק את האובייקט החיצוני, אבל בדרך כלל משאיר הפניות משותפות לעצמים פנימיים; copy.deepcopy מנסה להעתיק גם את הרכיבים המקוננים. אין צורך לשנן ספרייה זו, אך חשוב לדעת לשאול אם שני עצמים חולקים נתונים משותפים.

```
import copy
records = {"device": "PC-01", "temps": [60, 65]}
shallow = copy.copy(records)
deep = copy.deepcopy(records)
shallow["temps"].append(72)
print(records["temps"])
print(deep["temps"])
```

חשוב — עצם שנראה כעותק במסך אינו בהכרח בלתי תלוי. בפרויקט AI בעיות כאלה מופיעות בעת שינוי רשימות דוגמאות, הגדרות ניסוי או אובייקטים המשותפים למספר רכיבים.

2.7 הכלה: עצם שמכיל עצמים אחרים

בעולם האמיתי מחשב נמדד כמה פעמים, ומעבדה כוללת כמה מחשבים. לכן נכון ליצור מחלקת Measurement שמתארת מדידה, מחלקת Device שמחזיקה מדידות, ומחלקת Lab שמחזיקה מחשבים. זהו קשר של ״יש לו״ (has-a). הכלה מאפשרת לפרק מערכת ליחידות בעלות אחריות מוגדרת: מדידה שומרת תוצאה, מחשב אוסף מדידות, ומעבדה מפיקה דוח כללי.



איור 2.3: קשרי הכלה במערכת לניטור מעבדת מחשבים.

```
class Measurement:
    def __init__(self, value, minute):
        self.value = value
        self.minute = minute

class Device:
    def __init__(self, name):
        self.name = name
        self.measurements = []

    def add_measurement(self, value, minute):
        self.measurements.append(Measurement(value, minute))

    def average_temperature(self):
        if len(self.measurements) == 0:
            return None
        total = sum(m.value for m in self.measurements)
        return total / len(self.measurements)

a = Device("PC-01")
a.add_measurement(60, 0)
a.add_measurement(70, 10)
print(a.average_temperature())
```

נשים לב מדוע הרשימה נוצרת בתוך __init__ באמצעות self.measurements = []. כך כל מופע מקבל רשימה חדשה משלו. אם נגדיר רשימה משתנה ברמת המחלקה, או נשתמש ברשימה משתנה כערך ברירת מחדל, עלול להיווצר שיתוף לא רצוי בין עצמים. זו טעות נפוצה שכדאי לזהות כבר בשלב הלמידה.

הכלה לעומת ירושה

ביטוי עזר: Lab has Devices — המעבדה מכילה מחשבים, ולכן זה קשר הכלה. לעומת זאת Laptop is a Device — מחשב נייד הוא סוג של מחשב, ולכן ייתכן קשר ירושה. לפני כל קשר בין מחלקות שאלו: האם זהו רכיב שמצוי בתוך אחר, או סוג מיוחד של אותו דבר?

☐ שאלת חשיבה: האם רשת עצבית היא סוג של Dataset, או שהיא משתמשת ב־Dataset? נמקו לפי קשר is-a לעומת has-a.

2.8 כימוס, אימות נתונים ו־property

אם כל חלק בתוכנית יוכל להציב ערך שרירותי לטמפרטורה, אפשר לקבל נתונים לא הגיוניים. לכן נרצה שמחלקת Device תגדיר מקום אחד שבו בודקים קלט חדש. בפייתון נהוג לסמן שדות פנימיים בקו תחתון בתחילתם, למשל _temperature. זהו מוסכם בין מתכנתים ולא מנגנון אבטחה מוחלט; שמות עם קו תחתון הם עדיין נגישים מבחינה טכנית. המטרה היא שהקוד ישתמש בממשק ברור במקום לשנות את ייצוג הנתונים ישירות.

```
class Device:
    def __init__(self, name, temperature):
        self.name = name
        self.temperature = temperature

    @property
    def temperature(self):
        return self._temperature

    @temperature.setter
    def temperature(self, value):
        if not -30 <= value <= 120:
            raise ValueError("Temperature outside allowed range")
        self._temperature = value

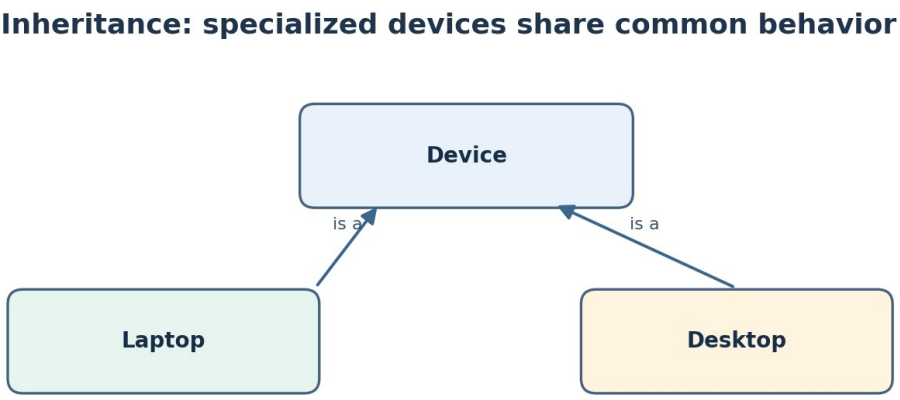
pc = Device("PC-01", 65)
pc.temperature = 71
print(pc.temperature)
```

הקוד בודק שהטמפרטורה בטווח שנבחר לדוגמה. מדובר בכלל אימות לצורכי הוראה, לא בטווח תקין רפואי או תקן של יצרן מחשבים. אם ננסה להציב 999 תיזרק חריגה מסוג ValueError. עדיף שהנתונים יידחו בשלב הקליטה מאשר שהתוכנית תמשיך לבצע חיזויים על ערכים שאין להם משמעות.

דגש — @property מאפשרת לקרוא תכונה כאילו הייתה שדה רגיל, אבל לבצע מאחורי הקלעים פעולה. זו אפשרות שימושית, לא חובה לכל שדה; מחלקה פשוטה אינה צריכה להפוך כל נתון ל־property ללא סיבה.

2.9 ירושה, super ופולימורפיזם

ירושה מאפשרת ליצור מחלקה מיוחדת שמבוססת על מחלקה כללית. אם לכל המחשבים יש שם וטמפרטורה, אבל למחשב נייד יש גם אחוז סוללה, אפשר לתאר Laptop כ־Device עם מידע נוסף. המחלקה היורשת מקבלת את המתודות והמאפיינים של מחלקת הבסיס, ויכולה להוסיף או לשנות התנהגות. חשוב לא להשתמש בירושה רק כדי לקצר כמה שורות קוד: קשר הירושה צריך להיות הגיוני מבחינת משמעות העצמים.



איור 2.4: *Laptop* ו־*Desktop* הם שני סוגים של *Device*.

```
class Device:
    def __init__(self, name):
        self.name = name

    def status(self):
        return "Active"

class Laptop(Device):
    def __init__(self, name, battery):
        super().__init__(name)
        self.battery = battery

    def status(self):
        if self.battery < 20:
            return "Charge needed"
        return super().status()

computers = [Device("PC-01"), Laptop("LAP-02", 12)]
for computer in computers:
    print(computer.name, computer.status())
```

הקריאה super().__init__(name) מפעילה את מתודת האתחול של מחלקת הבסיס. כך נשמרת הגדרת השם במקום אחד. ב־Laptop דרסנו (override) את המתודה status: בקריאה על מחשב נייד בעל סוללה חלשה יוחזר מסר אחר; בקריאה על Device רגיל יוחזר Active. זו דוגמה לפולימורפיזם: אותו שם מתודה מוביל להתנהגות מתאימה למופע המסוים.

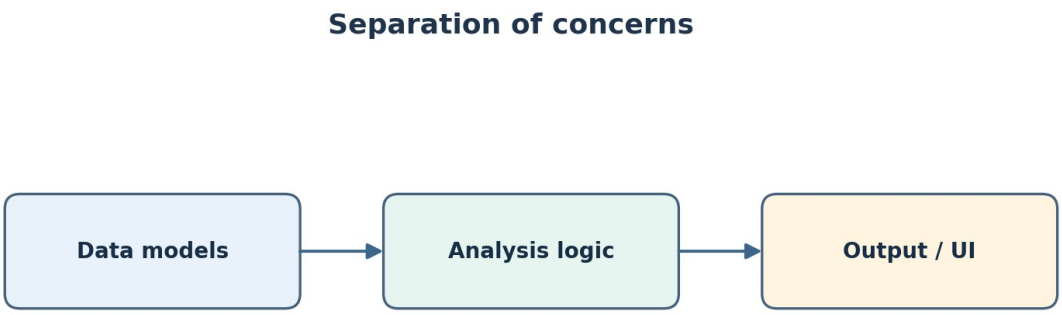
פולימורפיזם ללא קשר ירושה ישיר

פייתון מאפשרת גם כתיבת קוד שעובד עם כל עצם שמציע מתודה מתאימה, ללא חובה ששניהם יירשו ממחלקת בסיס. דרך חשיבה זו מכונה לעיתים duck typing. בספר שלנו נשתמש תחילה בירושה כי היא קלה להמחשה, ובהמשך נכיר ממשק של רכיבי AI שממממשים fit או predict בדרכים שונות.

☐ שאלת הבנה: מדוע שינוי של המתודה status במחלקת Laptop אינו מחייב שינוי בלולאה שעוברת על computers?

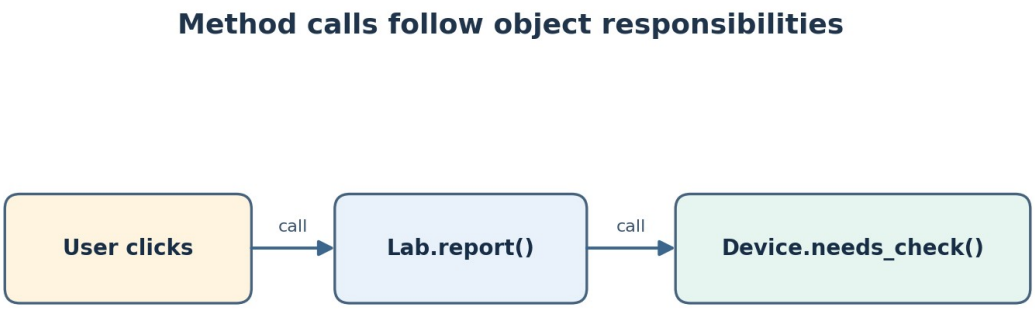
2.10 תרשים מחלקות ותכנון אחריות

לפני שמתחילים לכתוב פרויקט גדול, כדאי לשרטט אילו מחלקות דרושות, מי מחזיקה נתונים, מי מבצעת חישוב ומי מציגה תוצאה. כלל פשוט הוא "אחריות אחת ברורה לכל מחלקה". אין הכוונה שלמחלקה מותרת רק מתודה אחת; הכוונה שאפשר לתאר את מטרתה במשפט ברור. אם מחלקת Device גם קוראת CSV, גם מאמנת רשת וגם מציירת גרף, היא מבצעת תפקידים שונים מאוד וקשה לבדוק אותה.



איור 2.5: מומלץ להפריד בין מודלים של נתונים, לוגיקת ניתוח וממשק משתמש.

מחלקה	אחריות	פעולה אפשרית
Device	שמירת נתוני מחשב	add_measurement
Lab	ניהול אוסף מחשבים	add_device
DataScaler	לימוד טווחי מאפיינים והמרתם	fit / transform
Predictor	הפקת תחזית מדוגמה	predict
Report	הצגת תוצאות ומסקנות	build_summary



איור 2.6: כל מתודה מסייעת לאחרת בלי לבצע את כל העבודה בעצמה.

2.11 מודולים, קבצים ו־import

במחברת אחת אפשר להחזיק את כל הקוד, אבל בפרויקט אמיתי זו בדרך כלל בחירה לא נוחה. מודול בפייתון הוא קובץ py. המכיל הגדרות ומשפטי קוד. חלוקת התוכנית לקבצים מאפשרת לתקן רכיב בלי לחפש אותו בתוך מאות שורות ולבצע שימוש חוזר במחלקות שונות. בסביבת Colab אפשר להתחיל בתאים נפרדים, ובהמשך לעבור לקובצי .py.

שם קובץ	מה יכיל
device.py	המחלקות Device ו־Measurement
lab.py	המחלקה Lab והפעולות לניהול מחשבים
analysis.py	פעולות חישוב וסיכום נתונים
main.py	יצירת עצמים, הפעלה והצגת תוצאות

```
# file: device.py
class Device:
    def __init__(self, name):
        self.name = name

# file: main.py (save as a separate file)
# from device import Device
# computer = Device("PC-01")
# print(computer.name)
```

הדוגמה מציגה שני קבצים לצורכי תכנון, ולכן קטע main.py מובא כהערות כדי שניתן יהיה להריץ את התא בפני עצמו במחברת. כאשר עובדים בקבצים אמיתיים יש להסיר את סימני ההערה ולהריץ את main.py בסביבה שמכילה גם device.py. חשוב לא ליצור קובץ אישי בשם זהה לספרייה סטנדרטית כמו csv.py או math.py, כדי להימנע מהתנגשות בייבוא.

2.12 טיפול בטעויות — לא מתעלמים מקלט בעייתי

קוד אמין מבדיל בין מצב שאפשר לטפל בו לבין שגיאת תכנות שיש לתקן. לדוגמה, משתמש עשוי להקליד מחרוזת במקום טמפרטורה; אפשר לבקש ממנו להזין שוב. לעומת זאת, אם קראנו למתודה שאינה קיימת, אין טעם להסתיר את התקלה ולהמשיך כאילו דבר לא קרה. המנגנון try/except נועד לטפל במצבים צפויים ומוגדרים, ולא לכסות כל חריגה באופן גורף.

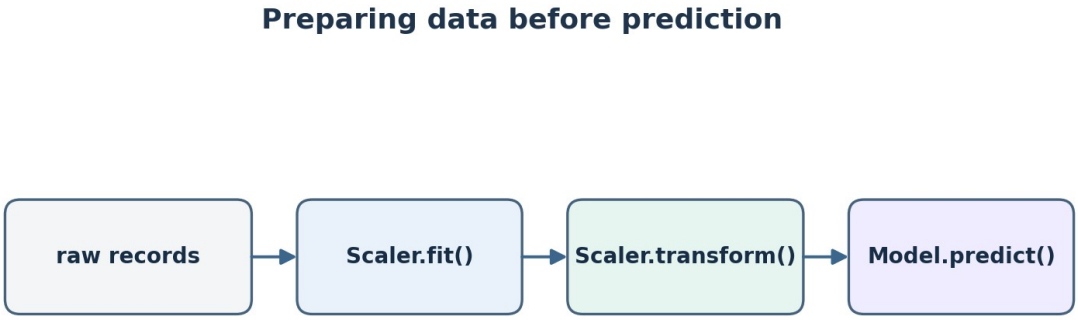
```
raw = "71"
try:
    temperature = float(raw)
    if not -30 <= temperature <= 120:
        print("Value outside the chosen range")
    else:
        print("Accepted:", temperature)
except ValueError:
    print("Enter a numeric temperature")
```

במערכת למידת מכונה יש לבצע בדיקות גם על אורך וקטור המאפיינים, על סדר העמודות ועל ערכים חסרים. מודל שמקבל ארבע תכונות בסדר מסוים אינו יכול לקבל בשקט שלוש תכונות או סדר אחר. עקרון זה נבנה בפרקים הבאים על הכנת נתונים ובדיקת מודלים.

2.13 OOP במערכת AI: הפרדה בין fit, transform ו־predict

נרצה כעת לראות את הקשר הישיר בין תכנות מונחה עצמים למערכות AI. בשפת העבודה של scikit-learn מקובל שרכיבי עיבוד נתונים מציעים פעולת fit הלומדת נתונים הדרושים להמרה, ופעולת transform הממירה דוגמאות באמצעות הערכים שנלמדו. מסווג מציע fit לאימון ו־predict לחיזוי. כאן נכתוב מחלקת MinMaxScaler לימודית קטנה כדי להבין את הרעיון, אך בפרויקט אמיתי נעדיף רכיבים מקצועיים שנבדקו היטב.

- מהדורת תלמיד • Python ב־OOP



איור 2.7: הרכיבים השונים משתתפים בשלבי הכנת הנתונים והחיזוי.

```
class SimpleMinMax:
    def __init__(self):
        self.minimum = None
        self.maximum = None

    def fit(self, values):
        if len(values) == 0:
            raise ValueError("Empty training data")
        self.minimum = min(values)
        self.maximum = max(values)
        return self

    def transform(self, values):
        if self.minimum is None:
            raise ValueError("Call fit before transform")
        span = self.maximum - self.minimum
        if span == 0:
            return [0.0 for _ in values]
        return [(v-self.minimum)/span for v in values]

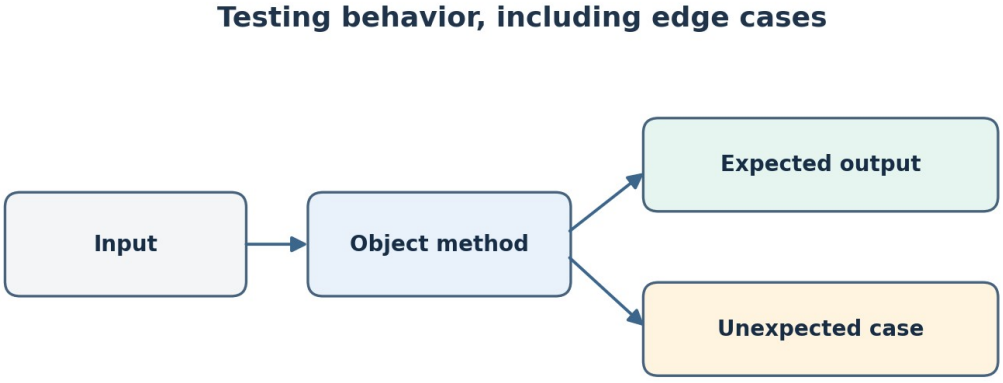
scaler = SimpleMinMax()
scaler.fit([40, 60, 80])
print(scaler.transform([50, 90]))
```

הפעולה fit לומדת מן האימון מינימום 40 ומקסימום 80. לאחר מכן transform ממירה 50 לערך 0.25 ואת 90 לערך 1.25. האחרון גדול מ־1 — וזה תקין, מפני שדוגמה חדשה יכולה להיות מחוץ לטווח האימון. אסור להתאים את המינימום והמקסימום מחדש על נתוני Test: פעולה כזו תחשוף למערכת מידע שלא אמור היה להיות זמין בזמן האימון.

הבחנה מרכזית — מחלקת SimpleMinMax היא הדגמה לימודית של עקרון התאמת טרנספורמציה. היא אינה מחליפה את בדיקות הקלט, הטיפול בחסרים וההתנהגות המלאה של MinMaxScaler המקצועית.

2.14 בדיקות יחידה ואיכות קוד

מחלקה טובה אינה רק מחלקה שנראית מסודרת. צריך להראות שהמתודות שלה פועלות נכון על קלט רגיל וגם במקרי קצה. בדיקת יחידה מפעילה רכיב קטן ומוודאת שהתוצאה מתאימה למה שציפינו. כאשר מתכננים בדיקות מראש, ההסבר של המתודות נעשה מדויק יותר: התלמיד חייב לחשוב מה פירוש ״נכון״ לפני שהוא רואה את הפלט.



איור 2.8: בדיקה משווה תוצאה בפועל לתוצאה צפויה, כולל מצבים חריגים.

```
scaler = SimpleMinMax()
scaler.fit([10, 20, 30])
result = scaler.transform([10, 15, 30])
assert result == [0.0, 0.25, 1.0]
assert scaler.transform([40]) == [1.5]
print("Tests passed")
```

הפקודה assert בודקת תנאי; אם הוא אינו מתקיים, התוכנית תעלה AssertionError. בדיקות מסוג זה שימושיות ללמידה ולבדיקות קטנות. במערכת גדולה אפשר להשתמש גם במסגרת בדיקות כמו pytest. חובה לא להסתפק במבחן אחד: יש לבדוק רשימת אימון ריקה, מאפיין קבוע, ערכי קצה, וכמות מאפיינים לא צפויה.

בדיקה חזותית של שינוי מצב

בנוסף לבדיקות חישוביות, חשוב לוודא שהממשק הציבורי של העצם לא מאפשר מצב סותר. לדוגמה, לא אמור להחזיק ממוצע טמפרטורה שנשמר במשתנה אחד בזמן שרשימת המדידות מתעדכנת במקום אחר. לעיתים עדיף לחשב את הממוצע מתוך הרשימה בכל קריאה, ולא לשמור עותק נוסף שדורש סנכרון. זו דוגמה לחשיבה הנדסית המתאימה במיוחד לתחום הנתונים.

2.15 מיני־פרויקט מסכם: SmartLab OOP

במיני־פרויקט דודו מבקש לבנות מערכת לניהול מדידות ממחשבי המעבדה. המערכת אינה צריכה לאמן מודל עמוק — זה יגיע בהמשך — אבל המבנה שלה צריך לאפשר להחליף בעתיד את כלל הסף הפשוט במסווג למידת מכונה. לכן נפריד בין אחסון המדידות, ניהול המעבדה, והפקת החלטת החיזוי.

רכיב	דרישה בסיסית	הרחבה אפשרית
Measurement	שמור ערך וזמן מדידה	תקינות טווח
Device	שמור שם ורשימת מדידות	חישוב מינימום, מקסימום וממוצע
Lab	הוסף מחשבים; הצג דוח	חיפוש לפי מזהה
ThresholdPredictor	החזר 0/1 לפי סף מוגדר	סף נפרד לסוג מחשב
Report	הצג דוח קריא	ייצוא CSV

```
class ThresholdPredictor:
    def __init__(self, threshold):
        self.threshold = threshold

    def predict(self, value):
        return int(value >= self.threshold)

predictor = ThresholdPredictor(70)
print(predictor.predict(65))
print(predictor.predict(74))
```

החלפנו כלל if שנכתב ישירות בתוך הדוח בעצם Predictor בעל מתודה predict. זו עדיין אינה למידת מכונה: הסף נקבע בידי המתכנת ואינו נלמד מנתונים. אבל המבנה כבר דומה לממשק של מודל אמיתי, כך שבהמשך נוכל להשתמש במחלקה שמאמנת פרמטרים ובוחנת נתונים חדשים.

מבדק קבלה למיני־פרויקט

- ☐ המערכת מציגה דוח עבור שלושה מחשבים, שלכל אחד מספר שונה של מדידות.
- ☐ המערכת אינה קורסת כאשר אין מדידות; היא מחזירה ערך מוסכם ומוסבר במקום ממוצע מדומה.
- ☐ מחלקת הניבוי מקבלת קלט אחד ומחזירה החלטה עקבית, בלי להדפיס מידע שלא התבקשה להדפיס.
- ☐ התלמיד יכול להסביר באילו מקומות יש הכלה ובאילו מקומות, אם בכלל, יש צורך בירושה.
- ☐ כל רכיב נבדק לפחות פעם אחת באמצעות קלט רגיל ופעם אחת באמצעות מקרה קצה.

העמקה 2.15.1 — משתנה של מחלקה לעומת משתנה של מופע

עד כה לכל מחשב הייתה טמפרטורה משלו: self.temperature. אבל לפעמים צריך נתון משותף לכל המחלקה, למשל מספר המופעים שנוצרו. במקרה כזה אפשר להגדיר משתנה מחלקה בראש המחלקה, מחוץ לכל המתודות. ההבחנה חשובה מאוד: משתנה מופע מתאר עצם אחד; משתנה מחלקה שייך למחלקה עצמה, אף שניתן לראות אותו גם דרך מופעים מסוימים.

```
class Device:
    created_count = 0

    def __init__(self, name):
        self.name = name
        Device.created_count += 1

a = Device("PC-01")
b = Device("PC-02")
print(Device.created_count)
print(a.name, b.name)
```

הספירה היא 2 מפני שמתודת האתחול הופעלה פעמיים. לעומת זאת a.name ו־b.name שונים, משום שכל שם נשמר במופע המתאים. אם נגדיר measurements = [] כמשתנה מחלקה, כל המופעים עלולים לחלוק אותה רשימה. מבחינה לשונית שתי הגישות נראות דומות; מבחינת התנהגות הן שונות לגמרי.

מתי להשתמש?	משמעות	מקום ההגדרה
מדידה של מחשב מסוים	מצב נפרד לכל מופע	self.temperature בתוך __init__
ספירת מופעים לימודית	נתון משותף למחלקה	created_count בגוף המחלקה
היסטוריית מדידות נפרדת	רשימה חדשה לכל מופע	self.measurements = [] בתוך __init__
בדרך כלל לא מתאים להיסטוריה אישית	רשימה משותפת בין מופעים שלא הסתירו אותה	measurements = [] בגוף המחלקה

כדי לבחון את התוצאה, נסו לפני ההרצה לנבא: אם למחשב הראשון נוספה מדידה, האם היא אמורה להופיע במחשב השני? אם התשובה בפועל חיובית בלי שהתכוונו לכך, ככל הנראה נשמרה רשימה

משותפת במקום רשימת מופע. זהו באג חשוב במיוחד כאשר עובדים עם מערכי נתונים של כמה משתמשים או ניסויים.

העמקה 2.15.2 — מעקב ידני אחרי רצף הפעלת מתודות

היכולת החשובה ביותר במבחן על עצמים אינה לזכור מילים באנגלית, אלא לדעת לעקוב אחר מצב העצם. נציג מעקב מלא ללא מחשב: בתחילה דודו יוצר מחשב ללא מדידות, לאחר מכן מוסיף שתי מדידות, שומר הפניה נוספת ולבסוף מוסיף מדידה דרך ההפניה השנייה. נעקוב גם אחר הערך המוחזר וגם אחר השינוי בזיכרון.

```
class Device:
    def __init__(self, name):
        self.name = name
        self.values = []

    def add(self, value):
        self.values.append(value)

    def average(self):
        if not self.values:
            return None
        return sum(self.values) / len(self.values)

first = Device("PC-01")
first.add(60)
first.add(80)
second = first
second.add(70)
print(first.values)
print(first.average())
```

חשוב במיוחד: ההוראה second = first אינה יוצרת עותק חדש. לכן הוספת 70 דרך second משנה את הרשימה שמוחזקת באותו עצם שאליו מתייחס גם first. תוצאת הממוצע תהיה 70, אבל התלמיד צריך להראות כיצד הגיע אליה ולא להסתפק בכך שהמחשב הדפיס אותה.

שלב	אילו נתונים נשמרים בעצם?	מה מחזירה **average**?
מיד אחרי first = Device(…)	[]	None
לאחר first.add(60)	[60]	60
לאחר first.add(80)	[60,80]	70
לאחר second = first	[60,80]	70
לאחר second.add(70)	[60,80,70]	70

שימו לב גם למקרה שבו אין מדידות. בתוכנית שלנו מחזירים None ולא אפס. החזרת אפס תיצור רושם שממוצע הטמפרטורות הוא 0 מעלות, בעוד שבפועל אין נתונים כלל. בחירת ערך החזרה מתאים היא חלק מתכנון ממשק המחלקה, ולא רק שאלה של תחביר.

העמקה 2.15.3 — תכנון לפני קוד: איך מחלקים מערכת לאחריות?

תלמידים רבים מתחילים לכתוב מחלקות לפי שמות העצמים שהם רואים בסיפור, אבל שוכחים לתכנן אילו פעולות כל מחלקה אמורה לבצע. נתרגל תהליך של ארבעה צעדים: ראשית מזהים את הישויות המרכזיות; שנית מנסחים את אחריותה של כל ישות; שלישית בוחרים איזה מידע צריך להיות שמור בה; ולבסוף מגדירים מתודות שיאפשרו לרכיבים אחרים להשתמש בה בלי לדעת את כל פרטי המימוש.

צעד	שאלת תכנון	החלטה בדוגמה
1 — זיהוי	מהן הישויות שקיימות בסיפור?	מחשב, מדידה, מעבדה, מסווג
2 — אחריות	מה כל אחד צריך לדעת לעשות?	Device שומר מדידות; Lab מנהלת מחשבים

צעד	שאלת תכנון	החלטה בדוגמה
3 — מידע	מהו מצב העצם ומה מחושב מחדש?	רשימת מדידות נשמרת; ממוצע מחושב
4 — ממשק	אילו פעולות חלקים אחרים יפעילו?	add, average, predict, summary

נבחן שתי הצעות לתכנון. בהצעה הראשונה Device מדפיס את כל הדוח ומאמן גם מודל. בהצעה השנייה Device שומר מדידות, Report מטפל בתצוגה ו־Predictor מחזיר תחזית. ההצעה השנייה נוחה יותר להרחבה, אך גם היא אינה ערובה לאיכות — חלוקה למחלקות קטנות מדי עלולה לפזר את הלוגיקה ולהקשות על ההבנה. המטרה היא חלוקה הגיונית לפי אחריות, לא מספר גדול ככל האפשר של מחלקות.

בפרויקט למידת מכונה נוכל בעתיד להחליף את Predictor במודל CNN או ברגרסיה מבלי לשכתב את המחלקה ששומרת את נתוני המחשב. כאשר ממשק predict ברור ונשאר יציב, שינוי המימוש הפנימי מצריך פחות שינויים במקומות אחרים. זהו שימוש מעשי חשוב בעקרון ההפרדה בין הממשק לבין אופן ביצוע העבודה.

העמקה 2.15.4 — כתיבת קוד קריא עם טיפוסים ותיעוד

ב־Python אפשר להוסיף רמזי טיפוסים (Type Hints) כדי לתאר מה מתודה מקבלת ומה היא מחזירה. לדוגמה, אפשר לציין שהמדידה היא מספר עשרוני והפעולה מחזירה אמת או שקר. רמזי הטיפוסים אינם מבצעים אכיפה מלאה בזמן ריצה; יש להמשיך לבדוק קלט בפועל כאשר זה חשוב. יתרונם העיקרי בספר שלנו הוא שהם הופכים את הממשק לקל יותר להבנה ומאפשרים לכלי פיתוח להתריע על חלק מהטעויות.

```
class ThresholdPredictor:
    def __init__(self, threshold: float):
        self.threshold = threshold

    def predict(self, value: float) -> bool:
        """Return True when the value reaches the threshold."""
        return value >= self.threshold

predictor = ThresholdPredictor(70.0)
print(predictor.predict(75.0))
```

השורה בתוך שלוש המרכאות היא Docstring — תיאור קצר של הפעולה. ההסבר אינו מתאר כיצד מחשבים באמצעות כל שורת קוד, אלא מה הפעולה מבטיחה למשתמש בה. כאשר תלמיד יפתח בעתיד מודל באמצעות PyTorch, תיעוד כזה יסייע לו להסביר את תפקיד רכיבי המערכת בפני בוחן ויקל על בדיקת הקוד.

העמקה 2.15.5 — תכנית עבודה למיני־פרויקט בארבע אבני דרך

כדי לאפשר למידה עצמאית ולא רק העתקת דוגמה, מומלץ לפתח את SmartLab בהדרגה. בכל אבן דרך נדרשת הוכחה שהרכיב הקודם עובד לפני שמוסיפים עוד חלק. כך קל יותר לגלות היכן נוצרה טעות, והתלמיד נדרש להסביר בכל שלב מה חדש ומה עדיין חסר.

אבן דרך	מה בונים?	בדיקת הצלחה
א׳ — נתונים	Measurement ו־Device עם מדידות	לכל עצם נשמרת היסטוריה עצמאית
ב׳ — ניהול	Lab שמכילה כמה מחשבים	אפשר להוסיף, לאתר ולספור מחשבים
ג׳ — חיזוי	ThresholdPredictor בעל predict	סיווג עקבי לפי סף מוגדר
ד׳ — הנדסה	Report, בדיקות ותיעוד	המערכת פועלת מחדש

בדיקת הצלחה	מה בונים?	אבן דרך
ומסבירה תוצאות		

הערכת הביצוע אינה נשענת רק על מספר הפקודות שהתלמיד כתב. נבדוק אם יש חלוקת אחריות נכונה, אם האובייקטים אינם משתפים מידע בטעות, אם קיימות בדיקות למצבי קצה ואם ניתן להסביר כיצד יעבור הקוד לשימוש במודל נלמד. חשוב להדגיש: הקוד המצורף לספר מדגים עקרונות; הגשת המיני־פרויקט דורשת מהתלמיד לכתוב פתרון משלו ולתעד החלטות.

2.16 טעויות נפוצות — כיצד מזהים ומתקנים?

דרך חשיבה לתיקון	למה זו בעיה?	הטעות
שאלו היכן המידע צריך להישמר	הפרמטר המקומי אינו בהכרח קיים במתודה אחרת	כתיבת name במקום self.name
צור רשימה חדשה בתוך __init__	מדידות של מחשב אחד מופיעות אצל אחר	שימוש ברשימה משותפת לכל העצמים
צרו מופע חדש או העתיקו במודע	שתי הפניות לאותו עצם	כדי להעתיק b = a
בדקו משמעות: הוא סוג של או מכיל?	המחלקות אינן מייצגות יחס is-a	ירושה במקום הכלה
הפרידו חישוב מהצגה	קשה לבדוק ולעשות שימוש חוזר	כל המחלקה מדפיסה וגם מחשבת
התאימו עיבוד על אימון בלבד	דליפת מידע בהכנת נתונים	fit על כל הנתונים לפני Test
טפלו רק בשגיאות צפויות	באגים מתגלים מאוחר מדי	except הסתרת כל חריגה עם כללי

2.17 תרגול מדורג — ללא פתרונות

רמה א׳ — זיהוי והבנת מושגים

☐ הסבירו במילים שלכם את ההבדל בין מחלקה, מופע, תכונה ומתודה. תנו דוגמה חדשה שאינה מחשב.

☐ עבור Device("PC-07", 68), כתבו מה מתרחש בעת יצירת המופע ואיזה מידע נשמר בו.

☐ בתוך __init__ מופיע גם name וגם self.name. מדוע צריך את שניהם?

☐ בתוכנית b = a ולאחר מכן "b.name = "X: מה יהיה a.name? נמקו בלי להריץ קוד.

רמה ב׳ — חישוב, מעקב ותכנון

☐ בנו מחלקת Sensor המקבלת שם וערך מדידה ומחזירה True כאשר המדידה חריגה לפי סף שנמסר.

☐ בנו טבלת מעקב לשלוש קריאות ל־update_temperature; בכל שלב ציינו מצב קודם, קלט, פלט ומצב חדש.

☐ צרו מחלקת Lab שמכילה לפחות שלושה Device, וכתבו פעולה המונה כמה מהם דורשים בדיקה.

☐ כתבו בדיקות עבור SimpleMinMax, כולל דוגמה שבה כל ערכי האימון זהים ודוגמה חדשה החורגת מטווח האימון.

רמה ג׳ — חשיבה של מפתח AI

☐ סרטטו תרשים מחלקות עבור מערכת שמזהה תקלות על סמך מדידות: אילו רכיבים הם נתונים ואילו מבצעים חיזוי?

☐ השוו שתי הצעות: מחלקה אחת של 250 שורות לעומת ארבע מחלקות בעלות אחריות ברורה. הציגו יתרונות ומגבלות לשתי החלופות.

☐ דונו מדוע לא נכון לקרוא לכללים מבוססי threshold בשם מודל למידת מכונה אם הסף לא נאמד מנתונים.

☐ הציעו שני מקרי בדיקה שיגלו באגים בהכנת נתונים לפני predict, והסבירו מה יקרה אם לא נבדוק אותם.

2.18 שאלת הערכה מסכמת בסגנון בחינה

מערכת SmartLab מכילה אוסף של עצמי Device. כל Device שומר שם, רשימת מדידות וטמפרטורה אחרונה. המחלקה Lab מפיקה דוח על כל המחשבים, והמחלקה ThresholdPredictor מסווגת את המחשבים לפי סף. לא כל קטע קוד המופיע בהמשך הוא מימוש של למידת מכונה; בחלק מן הסעיפים מדובר בסיווג לפי כלל קבוע.

סעיף א׳: הסבירו את הקשר בין Lab ל־Device ואת הקשר בין Device ל־Measurement. בחרו עבור כל קשר הכלה או ירושה ונמקו. סעיף ב׳: נתון שלמשתנים a ו־b יש אותה הפניה ל־Device. לאחר הוספת מדידה דרך b, כיצד ישתנה הממוצע המחושב דרך a? סעיף ג׳: תכננו מתודה שמחזירה את הממוצע בלי לשנות את מצב העצם. סעיף ד׳: הסבירו כיצד אפשר להחליף את ThresholdPredictor במודל שנלמד מנתונים תוך שמירה על ממשק predict. סעיף ה׳: הציעו שני ניסויי בדיקה ואיום אחד על אמינות המערכת.

בעת פתרון נדרשת דרך חשיבה ולא רק תשובה סופית. כתבו אילו הנחות קיבלתם, מתי מתבצע שינוי מצב ומה ניתן להסיק מן הפלט. השאלה מקורית בסגנון הערכה לימודית, ולא שאלה רשמית משאלון בגרות.

2.19 סיכום מושגים ודף נוסחאות

מושג	פירוש קצר
Class	תבנית המגדירה נתונים ומתודות
Instance/Object	מופע יחיד שנוצר ממחלקה
Attribute	נתון השייך לעצם
Method	פעולה של עצם
self	הפניה למופע שעליו פועלת המתודה
__init__	מתודת אתחול בעת יצירת מופע
Encapsulation	ארגון מידע וגישה מבוקרת אליו
Composition	עצם המכיל עצמים אחרים
Inheritance	מחלקה מיוחדת היורשת התנהגות ממחלקת בסיס
Polymorphism	אותו ממשק מפעיל התנהגויות שונות
Identity	האם שתי הפניות מצביעות לאותו עצם
fit/transform	למידת פרמטרי הכנה והמרת נתונים
predict	הפקת תחזית ממידע קלט
Unit test	בדיקה של רכיב תוכנה קטן

אין נוסחה מתמטית יחידה ל־OOP: ההצלחה נמדדת ביכולת לתכנן אחריות, להסביר מצב של עצמים, לבצע בדיקות ולהשתמש בקוד באופן אמין. כאשר נרצה לבצע חישוב מספרי, נוכל ליישם אותו בתוך מתודה תוך שמירה על ההפרדה בין נתונים ללוגיקה.

2.20 מקורות והמשך הדרך

עוגן בתכנית: מפת התמחות למידת מכונה ולמידה עמוקה — פרק 2, תכנות מונחה עצמים, 30 שעות: מסלול . https://github.com/samiabo75/Data-Science-ML-Course/tree/main/03-Deep-Learning-5-Units ההכנה המלווה: https://github.com/samiabo75/Data-Science-ML-Course/tree/main/01-Python-Basics/ 03-Deep-Learning-Python

מקורות מקצועיים: The Python Tutorial — Classes (docs.python.org/3/tutorial/classes.html); Python Data Model (docs.python.org/3/reference/datamodel.html); Python documentation — copy, dataclasses; Effective Python — Brett Slatkin, מהדורות מעודכנות; Hands-On Machine Learning with Scikit-Learn and PyTorch — Aurélien Géron, O’Reilly 2025. הספרים והתיעוד משמשים להעמקה מושגית ולהשראה פדגוגית; דוגמאות SmartLab, ההסברים והתרגילים נכתבו באופן מקורי.

הפרק הבא בחוברת היסוד: פרק 3 — מבוא למדעי הנתונים ולמידת מכונה (10 שעות). בו נחבר את מחלקות הנתונים והתכנות אל שאלת מחקר, מאגר נתונים, מודל, אימון וחיזוי.

דף סיום אישי — האם אני מוכן לפרק הבא?

לפני המעבר למבוא למדעי הנתונים, דרגו את שליטתכם בכל מיומנות. בחרו דוגמת קוד משלכם שמוכיחה לפחות שני סעיפים. אם אחת המיומנויות אינה ברורה, חזרו לדוגמה ולתרגול המתאימים; עדיף לזהות פער עכשיו מאשר באמצע פרויקט גדול.

מיומנות נבדקת	שולט/ת	דורש/ת תרגול
אני יוצר מחלקה ומסביר את תפקיד __init__ ו־self	☐	☐
אני מבדיל בין משתנה מופע למשתנה מחלקה	☐	☐
אני יודע להסביר הפניה כפולה והעתקת עצם	☐	☐
אני מזהה קשר הכלה לעומת קשר ירושה	☐	☐
אני מממש מתודה שמשנה מצב ומחזירה ערך	☐	☐
אני מתכנן מחלקה בעלת אחריות מוגדרת	☐	☐
אני בודק קוד בקלט רגיל ובמקרה קצה	☐	☐
אני מבדיל בין כלל קבוע לבין מודל שלמד מנתונים	☐	☐

משימת רפלקציה קצרה: כתבו שני משפטים על החלטת תכנון אחת שתחסוך לכם עבודה במיזם AI בעתיד. הוסיפו דוגמה אחת לבאג שזיהיתם בעצמכם, כיצד מצאתם אותו ומה שיניתם כדי לתקן. המטרה אינה לכתוב תשובה ״יפה״ אלא להראות למידה עצמאית מתוך התנסות.

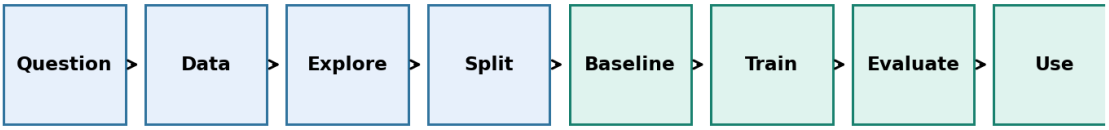
חוברת היסוד לספר הקורס המאוחד

פרק 3 — מבוא למדעי הנתונים וללמידת מכונה

כיצד הופכים נתונים לשאלה מחקרית, למודל, לתחזית ולהחלטה אחראית?

התמחות למידת מכונה ולמידה עמוקה, 5 יחידות לימוד | כיתות י״א–י״ב | מסגרת הפרק בתכנית: 10 שעות

מהדורת תלמיד · הסברים מלאים · דוגמאות מקוריות · חישובים ידניים · Python ו־Colab · משימות עצמאיות



מפת הפרק: לא מתחילים מבחירת אלגוריתם, אלא מהגדרת בעיה שאפשר לבדוק.

פתח דבר — מדוע אנחנו צריכים פרק מבוא?

בפרק הראשון למדנו לכתוב תוכניות ב־Python. בפרק השני למדנו כיצד לארגן קוד באמצעות מחלקות ועצמים. כעת נעמוד בפני שאלה חדשה: כיצד מחשב יכול להשתמש בדוגמאות מן העבר כדי לקבל תחזית לגבי מקרה שטרם פגש? זו כבר אינה רק שאלה בתכנות. זו שאלה של נתונים, מדידה, הנחות, ניסוי והערכת תוצאות.

המטרה אינה להכיר כמה שיותר אלגוריתמים בתוך עשר שעות. המטרה היא להבין את המילים, את דרך החשיבה ואת סדר הפעולות שילוו אותנו בכל פרקי ההמשך: רגרסיה, KNN, פרספטרון, CNN, רשתות חוזרות ופרויקט הגמר. לכן נשתמש בסיפור אחד שחוזר לאורך הפרק: דודו בונה מערכת המסייעת למעבדת המחשבים בבית הספר לקבוע אילו מחשבים כדאי לבדוק קודם.

מה תוכלו לעשות בסיום הפרק?

- להבחין בין נתון בודד, רשומה, מאפיין, תווית, מערך נתונים ומודל.
- להסביר את ההבדל בין AI, למידת מכונה ולמידה עמוקה, בלי להניח שכל מערכת חכמה לומדת.
- לזהות האם בעיה מתאימה לסיווג, לרגרסיה או לקיבוץ, ולנסח שאלת חקר ניתנת לבדיקה.
- להגדיר את מטריצת המאפיינים X ואת משתנה המטרה y, ולהסביר מדוע אסור להשתמש במידע שאינו זמין בזמן החיזוי.
- לקרוא קוד אימון בסיסי, להסביר את תפקידי fit, predict ו־score, ולהשוות למודל בסיס.
- להציג תוצאה מספרית בזהירות: מה נמדד, באילו נתונים, ומה איננו יכולים להסיק.

חלוקת שעות מוצעת לפרק

שעות	תוכן ועבודה	יחידת לימוד
2	הקשר לעולם ה־AI, שאלת חקר, טבלת תצפיות	א — בעיה ונתונים
2	סוגי למידה, X/y, סיווג מול רגרסיה וקיבוץ	ב — שפת המודלים

שעות	תוכן ועבודה	יחידת לימוד
2	הכנת נתונים, חלוקה, קו בסיס, טעויות והטיות	ג — ניסוי הוגן
2	חישוב ידני, קוד Colab, מדד הצלחה	ד — מודל ראשון
2	ניתוח מקרה, שאלות דמויות בחינה ותחילת פרויקט	ה — העברה והערכה

עוגן בתכנית: פרק 3 — "מבוא למדעי הנתונים ולמידת מכונה", 10 שעות, לפי מפת מסלול ההתמחות במאגר הקורס. החלוקה למפגשים והדוגמאות הן תכנון דידקטי שלנו. פרטי החובה המדויקים כפופים למסמך הרשמי התקף.

3.1 הסיפור: דודו רוצה למנוע השבתה של מחשבים

במעבדת המחשבים יש מחשבים רבים. חלקם פועלים באופן תקין, ואחרים נוטים להיתקע בדיוק כשמתחיל מבחן. דודו רוצה לקבל בכל בוקר רשימה של מחשבים שכדאי לבדוק לפני השיעורים. נניח שהוא מסוגל למדוד לכל מחשב טמפרטורה ממוצעת, מספר שגיאות בשבוע ושעות שימוש ביום. בנוסף, בסיום בדיקה מקצועית נרשמת תווית: האם המחשב נזקק לטיפול דחוף או לא.

חשוב להבדיל בין שתי פעולות. הראשונה היא כלל שכתב מתכנת: "אם הטמפרטורה גבוהה מ־70, הצג אזהרה". הכלל מבצע החלטה על פי סף קבוע מראש. הפעולה השנייה היא למידה מדוגמאות: המחשב מקבל נתוני עבר ותוצאות בדיקה, ומנסה לגלות קשר שעוזר לנבא טיפול דחוף במקרה חדש. בשלב זה עדיין אין הבטחה שהלמידה תהיה טובה יותר מכלל ידני.

ID	Temp C	Errors	Hours	Label y
PC-01	52	1	5	0
PC-02	67	2	8	1
PC-03	58	1	6	0
PC-04	74	4	10	1

X = three measurements; y = target label

דוגמת רשומות: שלוש עמודות מדידה ותווית אחת. הנתונים בספר מומצאים לצורכי הוראה בלבד.

3.2 נתונים, תצפיות, מאפיינים ותוויות

נתון אינו עדיין ידע

המספר 67, כשהוא עומד לבדו, כמעט אינו אומר דבר. האם זה גיל, מחיר או טמפרטורה? כדי להפוך אותו לנתון שימושי צריך לדעת מה נמדד, באיזו יחידה, מתי ובאיזו דרך. אם 67 הוא טמפרטורת המעבד בצֶלְסִיוּס שנמדדה אתמול, כבר אפשר לחקור אותו. משמעות הנתון נובעת מן ההקשר ומאיכות המדידה, לא מן המספר לבדו.

המונחים שנשתמש בהם

דוגמה מן המעבדה	הסבר לתלמיד	מונח
טבלת 12 מחשבים	אוסף דוגמאות עם מבנה ידוע	מערך נתונים — Dataset

מונח	הסבר לתלמיד	דוגמה מן המעבדה
תצפית — Observation / Sample	מקרה אחד שעליו נאספו נתונים	מחשב PC-04
מאפיין — Feature	פרט קלט שיש לנו בזמן הניבוי	טמפרטורה, מספר שגיאות
מטרה או תווית — Target / Label	מה שאנו מבקשים לנבא	האם נדרש טיפול דחוף?
מודל — Model	מנגנון שחושב קשר בין קלט לתוצאה	מסווג שמפיק 0 או 1
תחזית — Prediction	הפלט שמופק עבור מקרה חדש	"נדרש טיפול"

עמודת מזהה כגון PC-04 מסייעת לנו לעקוב אחר הרשומה, אך לרוב אינה תכונה טובה לחיזוי: היא לא מתארת את מצבו של המחשב. יתרה מכך, מזהה שמרמז על מקור הנתונים עלול ליצור דפוס מלאכותי. בחירת מאפיינים דורשת הבנה של הבעיה, לא רק בחירה של כל העמודות המספריות.

ייצוג מתמטי: X ו־y

נסמן את כל מאפייני הקלט באות X ואת הערך שאנחנו רוצים לנבא באות y. אם יש 12 מחשבים ושלושה מאפיינים לכל אחד, X תהיה מטריצה בעלת 12 שורות ו־3 עמודות. לעומתה y יהיה וקטור בן 12 תוויות. חייבת להיות התאמה מלאה: התווית במקום השביעי שייכת לתצפית השביעית בדיוק.

$$X.shape = (12, 3) \qquad y.shape = (12,)$$

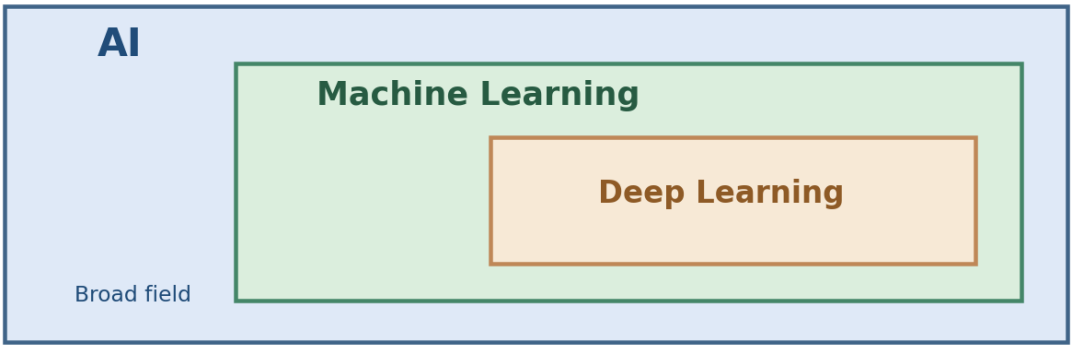
הסוגריים אינם מציינים שני משתנים: מדובר ב־shape, תיאור הממדים של מערך. כאשר עובדים עם pandas, שמות העמודות מסייעים לנו להבין מה מייצג כל רכיב. כאשר עוברים ל־NumPy, אין להניח שמערכת התכנות תזכור את משמעות היחידות עבורנו.

```
import pandas as pd

data = {"temperature": [52, 67, 58, 74],
        "errors": [1, 2, 1, 4],
        "hours": [5, 8, 6, 10],
        "urgent": [0, 1, 0, 1]}
df = pd.DataFrame(data)
X = df[["temperature", "errors", "hours"]]
y = df["urgent"]
print(X.shape, y.shape)
```

בדוגמה זו אנחנו משתמשים בארבע רשומות בלבד כדי לראות את המבנה. אין כאן מספיק דוגמאות כדי להוכיח שמודל יהיה שימושי בעולם האמיתי. כאשר יהיו לנו יותר נתונים, אותו מבנה קוד ימשיך לפעול.

3.3 כיצד AI, למידת מכונה ולמידה עמוקה קשורות זו לזו?



שלוש רמות של רעיונות: למידה עמוקה היא משפחה של שיטות בלמידת מכונה, והיא חלק מעולם ה־AI.

בינה מלאכותית (AI) היא שם רחב לתחום שמפתח מערכות המבצעות משימות הדורשות יכולות כגון הסקה, זיהוי דפוסים, תכנון או עבודה בשפה. חלק מן המערכות משתמשות בכללים ובחיפוש, ולא בכל מערכת כזו מתקיימת למידה מנתונים.

למידת מכונה (Machine Learning) מתמקדת בשיטות שמשתמשות בדוגמאות כדי להתאים מודל. לעיתים ההתאמה היא בחירת מקדמים ברגרסיה; לעיתים היא חיפוש קבוצות דומות; ולעיתים היא אימון מיליוני משקולות ברשת עצבית. המשמעות המשותפת היא שהמערכת מסתמכת על מבנה שנתגלה בנתונים ולא רק על אוסף כללים שנוסח מראש עבור כל מקרה.

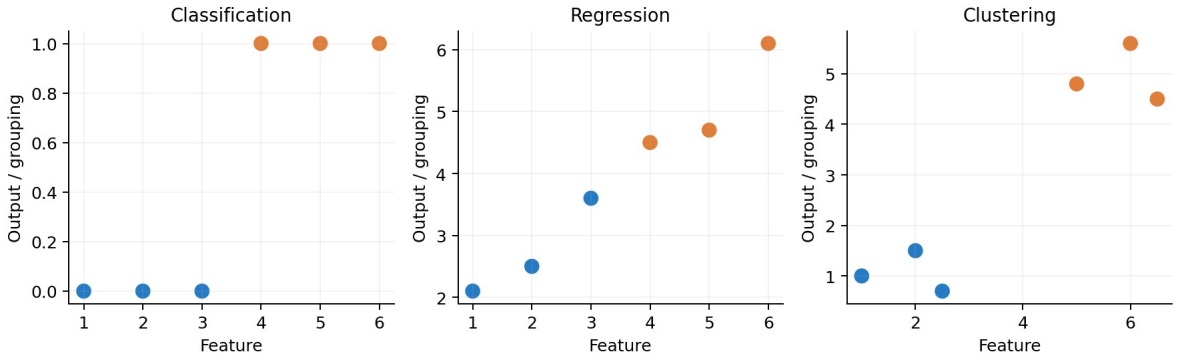
למידה עמוקה (Deep Learning) היא קבוצת שיטות המבוססת על רשתות עצביות עם כמה שכבות עיבוד. במקרים רבים היא יעילה לתמונות, צליל וטקסט, אך זה אינו אומר שהיא הבחירה המתאימה לכל בעיה. במיוחד כשיש מעט נתונים טבלאיים, מודל פשוט עשוי להיות קל יותר להסבר ואף מדויק יותר.

> שאלת הבנה: האם "תוכנה שמשתמשת ב־if" היא בהכרח מערכת AI? לא. האם כל מודל למידת מכונה הוא רשת עצבית? לא. האם תוצאת מודל היא אמת ודאית? גם לא.

3.4 מהי בכלל המשימה שלנו?

סיווג, רגרסיה וקיבוץ

לפני בחירת אלגוריתם צריך להגדיר מהו הפלט המבוקש. בסיווג (Classification) מבקשים לנבא קטגוריה — לדוגמה "טיפול דחוף" או "אין טיפול דחוף". ברגרסיה (Regression) מבקשים לנבא ערך כמותי — למשל מספר הדקות המשוער עד לסיום תיקון. בקיבוץ (Clustering) לא מספקים למערכת מראש תווית טיפול, אלא מבקשים ממנה לזהות דמיון בין רשומות.



שלוש משימות שונות: סיווג למחלקות, חיזוי ערך מספרי, וגילוי קבוצות ללא תוויות ידועות.

שאלת חקר	סוג משימה	הסיבה
האם מחשב זקוק לטיפול דחוף?	סיווג	התשובה היא קטגוריה
כמה דקות יימשך התיקון?	רגרסיה	התשובה היא מספר כמותי
אילו מחשבים פועלים בדפוסים דומים?	קיבוץ	אין תווית תשובה מראש
האם הציון עולה עם שעות למידה?	ניתוח קשרים	זו שאלת קשר; לא כל שאלה דורשת תחזית

יש להיזהר מן הטעות "אם התוויות כתובות במספרים, זו רגרסיה". התוויות 0 ו־1 בדוגמת טיפול הן שמות קוד למחלקות ולא כמות שיש לחזות. לעומת זאת ציון 85 או זמן תיקון 120 דקות הוא משתנה כמותי.

למידה מונחית, לא־מונחית ולמידת חיזוק

למידה מונחית (Supervised Learning) משתמשת בדוגמאות שיש להן תוצאה ידועה: X ו־y. כך אפשר ללמד מסווג או רגרסור. למידה לא־מונחית (Unsupervised Learning) מחפשת מבנה בנתונים ללא תוויות המטרה, למשל קבוצות בעלות דפוס שימוש דומה. למידת חיזוק (Reinforcement Learning) עוסקת בסוכן המקבל משוב על פעולות שביצע ומנסה לשפר את מדיניות הפעולה לאורך זמן. בפרק זה נכיר את ההבדלים באופן מושגי; את שיטות הסיווג והקיבוץ נלמד באופן שיטתי בפרקים המתקדמים.

3.5 לנסח שאלת חקר שהמחשב יכול לענות עליה

"אני רוצה לבנות AI חכם למחשבים" אינו ניסוח מחקרי: לא ברור מהו הקלט, מהי התשובה הרצויה ואיך נבדוק הצלחה. לעומתו "האם ניתן לנבא אם מחשב יזדקק לטיפול דחוף בשבוע הקרוב על סמך נתונים שהיו זמינים עד סוף השבוע הקודם?" הוא ניסוח שאפשר לבדוק. הוא קובע זמן חיזוי, מאפיינים אפשריים ותווית עתידית.

חמש שאלות לפני כתיבת קוד

- מהו המקרה היחיד שאנחנו מנתחים — מחשב, יום עבודה, תלמיד או תמונה?
- מהי התוצאה שנרצה לנבא או התופעה שנרצה לתאר?
- מאילו מקורות יגיעו הנתונים, והאם מותר להשתמש בהם?
- אילו מאפיינים באמת זמינים בזמן שבו המערכת אמורה לפעול?
- כיצד נדע שהפתרון מועיל בהשוואה לדרך פשוטה יותר?

דוגמת דליפת מידע: אם התווית היא "זקוק לטיפול בשבוע הבא", אסור לתת למודל מאפיין בשם "מספר תיקונים שבוצעו בשבוע הבא". המידע הזה מופיע רק לאחר הזמן שבו תופק התחזית. מודל שיקבל אותו אולי יציג ציון נפלא בבדיקה, אך לא יוכל להשתמש בו באמת.

3.6 מחקר נתונים אינו מתחיל באימון מודל

בדיקת איכות של מערך הנתונים

לפני אימון צריך להבין אם נתוני המקור אמינים. ייתכן שמדידות הטמפרטורה נלקחו מחיישנים שונים; ייתכן שעמודת שעות שימוש מכילה ערכים חסרים; וייתכן שמחשב אחד מופיע בטבלה כמה פעמים. בלי בדיקה אנחנו עלולים ללמד את המודל דפוסים הנובעים משגיאות הקלדה או מכפל רשומות.

לכל עמודה נבדוק: סוג הנתון, יחידת המידה, טווח אפשרי, מספר חסרים, חריגים ומקור. בנוסף נבדוק אם המחלקות מאוזנות, אם תוויות ניתנו באופן עקבי, ואם הנתונים מייצגים גם את הסיטואציה שבה ישתמשו במודל בעתיד.

```
print(df.head())
print(df.info())
print(df.isna().sum())
print(df["urgent"].value_counts())
```

אלה פעולות סקירה ולא תהליך למידה. הן עוזרות לנו לשאול שאלות על הנתונים. בפרק הבא — EDA — נעמיק בגרפים, סטטיסטיקה ובדיקה של קשרים בין מאפיינים. כאן נתרכז במשמעות השאלות ובצורך לתעד את ממצאי הבדיקה.

מדוע הטיות בנתונים חשובות?

דמיינו שמרבית נתוני המעבדה הגיעו ממחשבים ישנים, ובחודש הבא מתקינים מחשבים חדשים. מודל שהתאים את עצמו לדוגמאות הישנות עלול להצליח פחות בסביבה החדשה. גם אם קוד האימון תקין מבחינה טכנית, איכות התחזית תלויה במידת ההתאמה בין נתוני האימון לנתונים שהמודל יפגוש בפועל.

3.7 מאוסף דוגמאות לניסוי הוגן



Illustration only: proportions depend on dataset and evaluation plan

המחשה עקרונית של חלוקת נתונים. אין יחס אחיד שמתאים לכל מערך, ובנתוני זמן נדרשת חלוקה לפי סדר הזמן.

אם נשאל את המודל על אותן דוגמאות ששימשו לבנייתו, נקבל תשובה לשאלה "עד כמה המודל מתאים למה שכבר ראה?". אבל המטרה שלנו היא לבדוק נתונים חדשים. לכן נהוג להפריד בין Train, שממנו לומדים; Validation, שבעזרתו בוחרים בין גרסאות מודל; ו־Test, המיועד להערכה סופית לאחר סיום הבחירות.

אפשר גם להשתמש ב־Cross-Validation על קבוצת האימון במקום לשמור מראש קבוצת Validation קבועה. התלמיד אינו צריך לזכור כאן כל פרוטוקול אפשרי, אבל עליו להבין כלל עקרוני: אסור להשתמש בנתוני Test כדי לבחור מאפיינים, לכוונן פרמטרים או להחליט איזה מודל לפרסם.

מהי זליגת מידע (Data Leakage)?

זליגת מידע נוצרת כאשר תהליך האימון או קבלת ההחלטות משתמש במידע שאמור להיות מחוץ להישג ידו של המודל. דוגמה נוספת היא חישוב ממוצע לכל מערך הנתונים לפני חלוקה, ואז שימוש בממוצע הזה כדי לתקנן את נתוני האימון. במקרה זה נתוני הבדיקה כבר השפיעו על ההכנה, אפילו אם לא הוכנסו ישירות לפעולת fit של המסווג.

בפרקים המתקדמים נשתמש ב־Pipeline שמבטיח כי פעולות הכנה מתאימות את עצמן רק לנתוני האימון בכל פיצול. זו לא רק המלצה תכנותית: זו דרך להגן על אמינות המסקנה המדעית.

> בדיקת הבנה: אם בחרנו ערך k ב־KNN אחרי שבדקנו עשרה ערכים על Test, האם Test עדיין מספק הערכה בלתי תלויה? לא. אפשר ללמוד מהניסוי, אך אין להציג את אותו Test כהערכה סופית בלתי תלויה.

3.8 קודם קו בסיס, אחר כך מודל מסובך

קו בסיס (Baseline) הוא פתרון פשוט שמשמש נקודת השוואה. במקרה של חיזוי האם מחשב יזדקק לטיפול דחוף, אפשר להתחיל ממודל שתמיד מנבא את המחלקה הנפוצה ביותר. זו אינה שיטת חיזוי מתוחכמת, אבל היא עוזרת לברר האם המודל החדש מוסיף ערך מעבר לניחוש פשוט.

חישוב ידני: מדוע Accuracy של 90% עלול להטעות?

נניח שנבדקו 100 מחשבים. 90 לא היו זקוקים לטיפול דחוף, ו־10 כן. מודל שמנבא לכל המחשבים "לא דרוש טיפול" מסווג נכון 90 מקרים מתוך 100. לכאורה הוא מדויק ב־90%, אך הוא החמיץ את כל 10 המחשבים שבאמת נזקקו לטיפול דחוף.

$$\text{Accuracy} = 90 / 100 = 0.90 = 90\%$$

$$\text{Recall (urgent)} = 0 / 10 = 0$$

כאשר מחלקה אחת נפוצה מאוד, מדד הדיוק לבדו עלול להציג תמונה מעודדת אך מטעה.

המסקנה איננה ש־Accuracy חסר ערך. המסקנה היא שיש לבחור מדדים לפי הבעיה ולפי מחיר הטעות. במערכת בטיחותית החמצת מקרה חמור עשויה להיות יקרה במיוחד. בנושא זה נעמיק בפרק הערכת המודלים שכבר נכתב בספר המאוחד.

3.9 מודל ראשון: מחישוב ידני להרצת קוד

דוגמה ידנית קטנה — שש תחזיות

נניח שיש לנו שש דוגמאות בדיקה. התוויות האמיתיות והתחזיות של מודל ראשוני הן אלה:

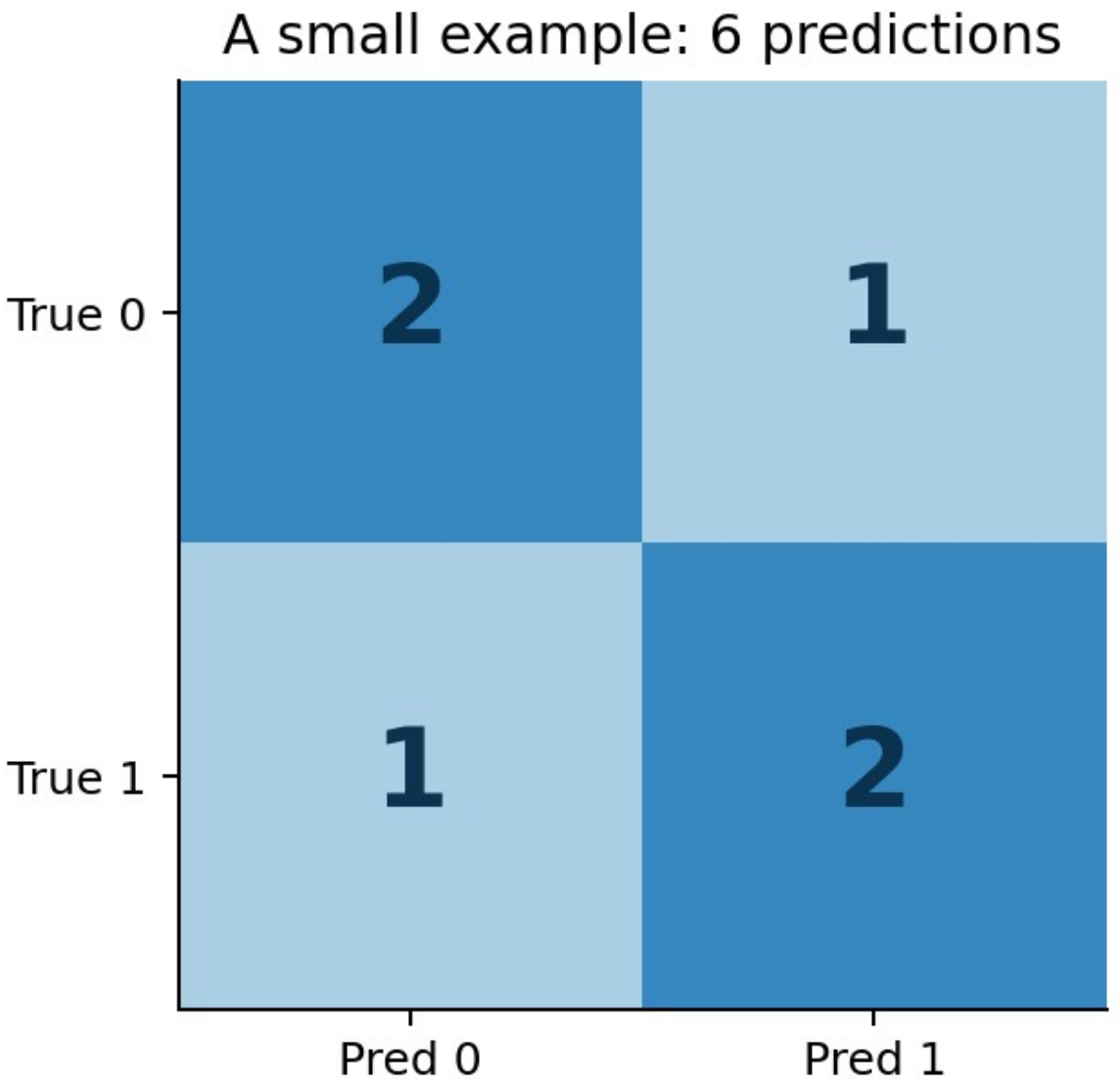
תצפית	תווית אמיתית y	תחזית $\hat{y}$	נכון?
1	0	0	כן
2	0	1	לא
3	1	1	כן
4	1	0	לא
5	0	0	כן
6	1	1	כן

ארבע תחזיות נכונות ושתי תחזיות שגויות. לכן Accuracy הוא ארבע שישיות, כלומר כשני שלישים. כעת נבנה מטריצת בלבול: שני מקרים של 0 זוהו נכון, שני מקרים של 1 זוהו נכון, מקרה אחד של 0 סווג בטעות כ־1, ומקרה אחד של 1 סווג בטעות כ־0.

$$\text{Accuracy} = (2 + 2) / 6 = 4/6 \approx 0.667$$

$$\text{Precision}(1) = 2 / (2 + 1) = 2/3$$

$$\text{Recall}(1) = 2 / (2 + 1) = 2/3$$



מטריצת בלבול עבור שש הדוגמאות: השורה היא אמת, העמודה היא תחזית.

אין להסיק משש דוגמאות זעירות שהמודל יהיה מדויק בעתיד. התרגיל נועד ללמד את משמעות המדד ואת הדרך לבדוק חיזוי. בדוגמה אמיתית נגדיר מראש כיצד מדגמים את נתוני הבדיקה, ומהי רמת אי־הוודאות של האומדן.

מודל ראשון עם נתוני Iris

במקום לאמן מסווג על ארבעת המחשבים שבטבלה, נשתמש כעת במאגר Iris המובנה בספריית scikit-learn. כל תצפית היא פרח, ובכל פרח ארבע מדידות. תווית המטרה מציינת את קבוצת הפרח. נשתמש בשיטה מוכרת, KNN, בלי להיכנס כרגע לכל המתמטיקה שלה: הוא בוחן דוגמאות אימון קרובות ומנבא בהתאם לתוויות שלהן. נלמד אותו לעומק בפרקים המתקדמים.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.dummy import DummyClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)
```

```
baseline = DummyClassifier(strategy="most_frequent")
baseline.fit(X_train, y_train)
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)

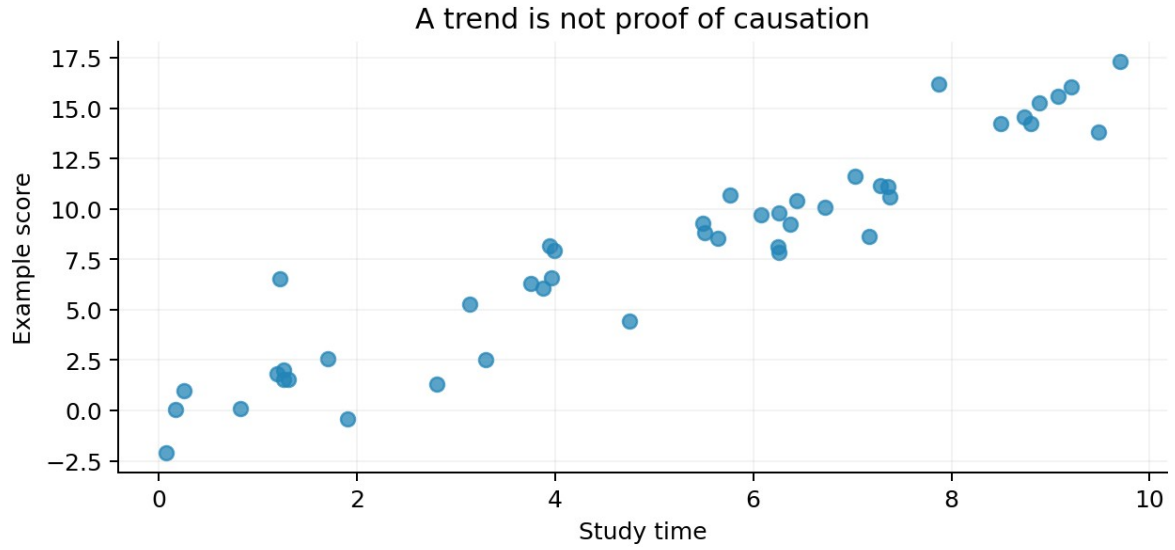
print("Baseline:", accuracy_score(y_test, baseline.predict(X_test)))
print("KNN:", accuracy_score(y_test, model.predict(X_test)))
```

בקריאת הקוד חשוב להבין את הרצף: load_iris טוענת נתונים; train_test_split מחלקת אותם; DummyClassifier מספק קו בסיס; fit משתמש בדוגמאות אימון; predict מפיק תחזיות; ו־accuracy_score משווה אותן לתוויות האמיתיות. גם אם נקבל ציון גבוה, זו תוצאה של חלוקה אחת, ולא הבטחה לביצועים בכל מצב.

חמש שאלות שכל תלמיד חייב לענות עליהן לאחר הרצה

- מה כל שורה ב־X מייצגת, וכמה מאפיינים יש בה?
- היכן בקוד בוצעה חלוקת הנתונים, ומה מטרתה?
- איזו פקודה "לומדת" מן הדוגמאות, ואיזו פקודה מחזירה תחזיות?
- איזה מודל שימש קו בסיס, ולמה כדאי להשוות אליו?
- מה אפשר להסיק ממדד הדיוק — ומה עדיין לא אפשר להסיק?

3.10 מתאם, סיבתיות והקשר האנושי



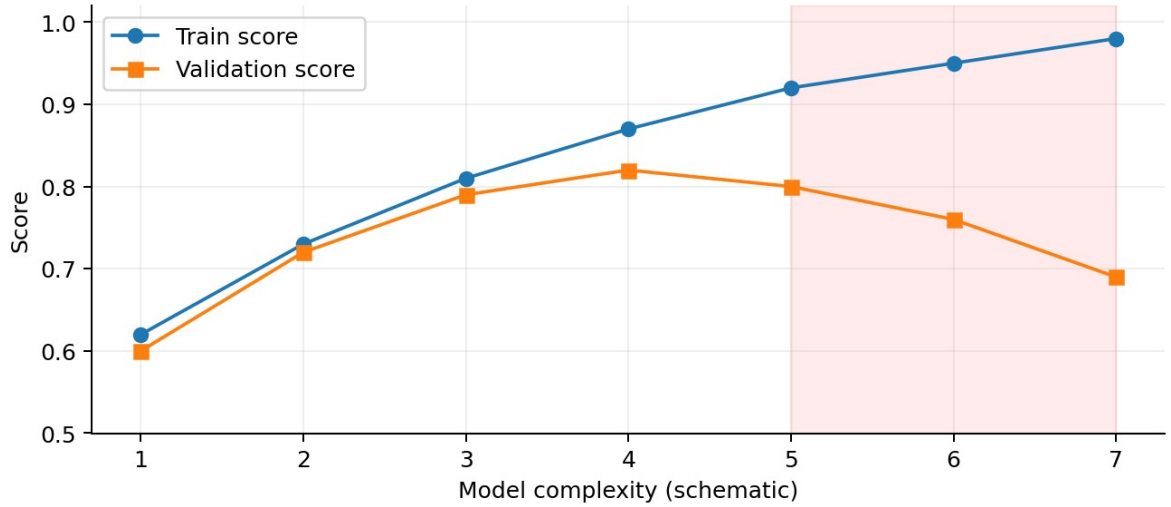
תרשים פיזור של נתונים מדומים: קו מגמה אינו הוכחה שהמשתנה האחד גרם לשני.

גם כאשר נזהה קשר בין מאפיין לתוצאה, אין להסיק אוטומטית קשר של סיבה ותוצאה. למשל, אפשר למצוא קשר בין שעות שימוש במחשב לבין מספר התקלות. אבל ייתכן שמחשבים ישנים גם נמצאים בשימוש רב יותר וגם מתקלקלים לעיתים קרובות. בלי לבחון את גורמי הרקע, הקשר שנמצא לא אומר ששעות השימוש לבדן גרמו לתקלה.

כאשר עוברים למערכות העוסקות באנשים, חובת הזהירות גדלה. מודל לחיזוי ציונים, לזיהוי קושי לימודי או למיון בקשות עשוי להשפיע על הזדמנויות של אנשים. לכן יש להתייחס לפרטיות, לאיכות תוויות, לייצוג הוגן של קבוצות וליכולת לערער על החלטה. ההבנה הזאת היא חלק מהמקצועיות של מפתח AI, לא נושא נפרד שאפשר לדלג עליו.

3.11 כיצד מזהים התאמת יתר?

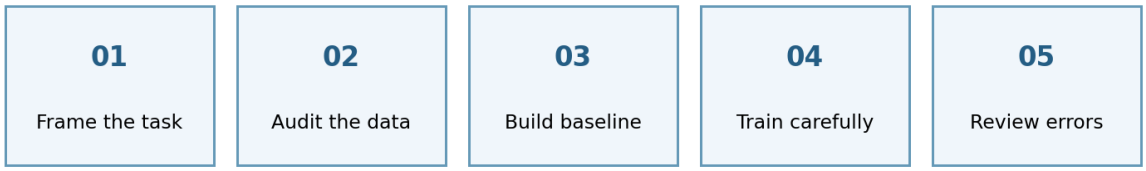
נניח שמודל מורכב מצליח לזכור כל פרט קטן בנתוני האימון. לפעמים הוא אף מצליח במבחן על דוגמאות אלו באופן מושלם, אבל נכשל כאשר מגיעות תצפיות חדשות. תופעה זו מכונה התאמת יתר (Overfitting). המטרה אינה לקבל את ציון האימון הגבוה ביותר, אלא לבחור מודל שמצליח להכליל.



עקומות סכמטיות: ציון האימון יכול להמשיך לעלות גם כאשר ציון הוולידציה מתחיל לרדת.

לכן פרק זה לא מסתיים בהדפסת Accuracy בלבד. בכל ניסוי נדרש לתעד את חלוקת הנתונים, את החלטות הפיתוח, את מגבלות המדידה ואת השגיאות שנמצאו. בהמשך הספר נראה כיצד להשתמש בוולידציה, ברגולריזציה ובעצירה מוקדמת, אך כאן חשוב להבין קודם את הבעיה.

3.12 יומן מדעי: מה על תלמיד להגיש בסוף ניסוי?



A research notebook should explain each decision, not just show a final score.

הסבר ניסוי מקצועי הוא סיפור החלטות: שאלה, נתונים, קו בסיס, מודל, בדיקת שגיאות.

מה כותבים	רכיב בדוח
מה התופעה שאותה מנסים לחזות או להבין?	שאלת חקר
מאין הנתונים, מהו היקפם ומהן מגבלותיהם?	מקור הנתונים
מה מייצגת רשומה? מה זמינות התכונות בעת ניבוי?	תצפיות ומאפיינים
כיצד נמנעה זליגת מידע?	חלוקה והכנה
מהי נקודת ההשוואה הפשוטה?	קו בסיס
איזו שיטה הופעלה, על אילו נתונים ואילו	מודל וניסוי

מה כותבים	רכיב בדוח
הגדרות נבחרו?	
מהו המדד, מהן השגיאות ומה עדיין לא ידוע?	תוצאה ומגבלה

3.12א חקר מקרה מפורט — מה באמת אומרת טבלת מחשבים?

נחזור למאגר שמופיע במחברת התלמיד. יש בו 12 מחשבים, ולכל מחשב שלוש מדידות ותווית טיפול. נניח שנבדקו שישה מחשבים שדרשו טיפול ושישה מחשבים שלא דרשו טיפול. מה אפשר להגיד על מערך כזה? ראשית, בדוגמה שלנו אין חוסר איזון בין המחלקות. אבל אין להסיק מכך שהנתונים מייצגים את בית הספר כולו: ייתכן שבחרנו בכוונה מספר שווה מכל קבוצה כדי להקל על התרגול. אם כך, שכיחות הצורך בטיפול בעולם האמיתי עשויה להיות אחרת לגמרי.

כעת נשאל על המאפיינים. טמפרטורה של 74 מעלות עשויה להיות משמעותית רק כאשר יודעים היכן נמדדה ובאילו תנאי עומס. גם מספר שגיאות 4 אינו מספר שאפשר לפרש לבד: האם מדובר בשגיאות מערכת חמורות, בניסיונות התחברות שגויים, או באירועים רגילים? לכן לפני אימון מודל אנחנו חייבים לכתוב "מילון נתונים" שבו לכל מאפיין הגדרה, יחידת מדידה ותנאי איסוף.

מאפיין	הגדרה ברורה שנדרשת בפרויקט	בדיקת איכות מוצעת
temperature	ממוצע מדידת חיישן המעבד במשך יום, במעלות צלזיוס	איתור קריאות חסרות או ערכים בלתי סבירים
errors	מספר שגיאות מערכת מסוג מוגדר במשך שבעה ימים	וידוא שכל המחשבים נספרו באותו חלון זמן
hours	ממוצע שעות שימוש יומי באותו שבוע	בדיקת ערכי 0 וערכים מעל 24
urgent	קביעה של גורם מקצועי לפי כלל אחיד	בדיקת עקביות בהגדרת "דחוף"

נניח שהמטרה היא תחזית למצב בשבוע הבא. במקרה כזה גם חלון הזמן של המאפיינים צריך להסתיים לפני תחילת השבוע הבא. אם בטעות נצרף מידע שנרשם לאחר תחילת התקלה, המודל עשוי לזהות את תוצאות התקלה במקום את סימני האזהרה שקדמו לה. כעת אפשר לראות מדוע הגדרת שאלת המחקר קובעת גם את אופן איסוף הנתונים, ולא רק את שם האלגוריתם.

> מסקנת ביניים: טבלה מסודרת אינה בהכרח מערך נתונים טוב. צריך לברר כיצד נוצרה כל שורה, כיצד נמדדה כל תכונה, ומתי התקבלה התווית.

3.12ב חישוב ידני של מודל רגרסיה וקו בסיס

עד כה חישבנו מדדי סיווג. כדי להבין כיצד אותן פעולות מחקריות משמשות גם ברגרסיה, נדמיין מודל שמנבא את משך תיקון המחשב בדקות. נציג ארבעה מקרים שלא שימשו לאימון. משכי התיקון האמיתיים הם 30, 40, 35 ו־45 דקות. תחזיות המודל הן 32, 39, 30 ו־42 דקות. כל תחזית היא ניסיון לנבא גודל מספרי, ולכן איננו מחשבים כאן Accuracy של סיווג.

שגיאה מוחלטת	תחזית	זמן אמיתי	מקרה
2	32	30	א
1	39	40	ב
5	30	35	ג
3	42	45	ד

שגיאה מוחלטת	תחזית	זמן אמיתי	מקרה

כדי לחשב MAE — ממוצע השגיאה המוחלטת — מחברים את כל ההפרשים המוחלטים ומחלקים במספר המקרים. מקבלים סכום 11 על ארבעה מקרים, כלומר 2.75 דקות. מדוע משתמשים בערך מוחלט? משום ששגיאה של מינוס חמש דקות אינה אמורה "לבטל" שגיאה של פלוס חמש דקות: שתיהן טעויות בחיזוי, למרות הכיוון השונה.

$$MAE = (2 + 1 + 5 + 3) / 4 = 2.75 \text{ minutes}$$

אבל עדיין אין לנו תשובה לשאלה אם המודל מועיל. נניח שבנתוני האימון זמן התיקון הממוצע היה 38 דקות. קו בסיס פשוט מנבא תמיד 38 דקות, ולכן על ארבעת המקרים החדשים נקבל שגיאות של 8, 2, 3 ו־7 דקות. הממוצע הוא 5 דקות. בדוגמה זו המודל המורכב משפר את MAE לעומת קו הבסיס. השוואה הוגנת מחייבת להשתמש באותם מקרי בדיקה ובאותו מדד.

$$\text{Baseline MAE} = (8 + 2 + 3 + 7) / 4 = 5 \text{ minutes}$$

שימו לב שהממוצע 38 התקבל מנתוני האימון בלבד. אילו היינו מחשבים את ממוצע זמני התיקון האמיתיים של קבוצת הבדיקה, היינו מאפשרים לתוצאות הבדיקה להשפיע על קו הבסיס. זו דוגמה נוספת לכך שלעיתים זליגת מידע מתרחשת בתוך חישוב שנראה תמים ביותר.

3.12ג אותה רשומה פעמיים — מדוע חלוקה אקראית אינה תמיד נכונה?

נניח שמחשב PC-07 נמדד בכל יום במשך שנה. אם נחלק באופן אקראי את כל המדידות, ייתכן שחלק מימי המחשב ישמשו לאימון ואחרים לבדיקה. במקרה זה המודל עלול לזהות תכונות קבועות של המחשב עצמו, ולכן ציון הבדיקה לא ישקף היטב את היכולת להתמודד עם מחשב חדש לגמרי. אם שאלת המחקר היא "כיצד ננבא מחשבים שלא הופיעו באימון?" כדאי לשקול חלוקה לפי מזהה מחשב או קבוצת מכשירים.

לעומת זאת, אם המטרה היא לחזות את השבוע הבא עבור אותם מחשבים שכבר קיימים, חשוב לשמור על הסדר הכרונולוגי. בשני המקרים צריך להתאים את שיטת ההערכה לשאלה המחקרית. אין חלוקת Train/Test אחת שהיא תמיד נכונה. תלמיד שמסביר את ההבדל בין שתי השאלות האלה מפגין הבנה של המחקר, ולא רק של פקודת Python.

טעות אפשרית	חלוקה הגיונית	מטרת הפרויקט
אותו מחשב גם באימון וגם בבדיקה	הפרדה לפי מזהה מחשב	חיזוי עבור מחשבים חדשים
שימוש בנתונים מן העתיד בזמן האימון	אימון על עבר ובדיקה על עתיד	חיזוי השבוע הבא
אי־שמירה על ייצוג המחלקות כאשר הן נדירות	חלוקה אקראית מבוקרת כשיש הצדקה	חיזוי על דוגמאות עצמאיות

3.12ד כיצד קוראים פלט של תוכנית בצורה מדעית?

נניח שקוד ההדגמה מדפיס שתי שורות: Baseline accuracy ואחריה KNN accuracy. לפני שמכריזים שהמודל השני "טוב יותר", עונים על סדרת שאלות. האם שני המודלים נבדקו על אותה קבוצת בדיקה? האם השתמשנו באותה הגדרת הצלחה? האם מספר הדוגמאות מספק כדי לקבל אומדן יציב? האם אותן קבוצות פרחים מופיעות באופן דומה בנתוני הבדיקה?

לאחר מכן מסבירים את התוצאה במילים מדויקות. ניסוח מקצועי הוא למשל: "בחלוקת הבדיקה הזאת, ובמדד Accuracy, המודל השיג תוצאה גבוהה מקו הבסיס. יש צורך בהערכה נוספת על דוגמאות אחרות כדי לקבוע עד כמה ההבדל יציב." לעומת זאת המשפט "המודל הבין את הפרחים ומעכשיו תמיד יצליח" חורג ממה שהניסוי מוכיח.

דרך טובה לבדוק הבנה היא להפוך את השאלה: מה ישתנה אם נחליף רק את random_state? החלוקה עשויה להשתנות, ולכן ייתכן שנקבל ציון אחר. מה ישתנה אם נשאיר את החלוקה קבועה ונחליף את המודל? כאן ההשוואה מתמקדת בשיטת החיזוי, אם יתר התנאים נשמרים. מה ישתנה אם נוסיף מאפיין שנמדד ביחידות גדולות מאוד? מודל המבוסס על מרחקים עשוי להגיב אחרת, ולכן נצטרך לבדוק את קנה המידה בפרקים הבאים.

3.12ה חיבור בין Python, OOP והמשך הלמידה העמוקה

בפרקי Python למדנו לכתוב פונקציות שמקבלות נתונים ומחזירות תוצאה. בפרק OOP למדנו להגדיר מחלקות בעלות אחריות ברורה. עכשיו אפשר להבין מדוע ספריית scikit-learn משתמשת בעצמים: אנו יוצרים אובייקט של מסווג, מאמנים אותו, ואז מבקשים ממנו תחזית. הפעולות fit ו־predict אינן מילים קסומות; הן שמות של פעולות בממשק תוכנתי, והן מחברות בין ארגון תוכנה לבין תהליך של למידת מכונה.

מאוחר יותר, בפרקי הרשתות העצביות, נשתמש ברעיון דומה אך עם חישובים מורכבים יותר: נגדיר ארכיטקטורה, נעביר נתונים קדימה, נחשב שגיאה, נעדכן משקולות ונעריך את התוצאה. מושגי היסוד שנלמד כאן — נתון, מאפיין, תווית, אימון, בדיקה ומדד — אינם נעלמים כאשר המודל הופך עמוק. הם נשארים הבסיס שעליו נבנה הפרויקט כולו.

3.13 תרגול מדורג — מן הידע להבנה עצמאית

רמה א — הבנת מושגים

- כתבו במילים שלכם מה ההבדל בין תצפית, מאפיין ומשתנה מטרה. הוסיפו דוגמה שאינה קשורה למחשבים.
- בחרו איזו משימה היא סיווג ואיזו רגרסיה: מחיר רכב, סוג ציפור, זמן משלוח, האם אימייל הוא דואר זבל.
- טבלה מכילה 200 תלמידים וחמישה מאפייני קלט. כתבו את גודל X ואת אורך y והסבירו את משמעותם.
- הסבירו מדוע עמודת מזהה אינה בהכרח מאפיין בעל ערך חיזויי.

רמה ב — חישוב, ניתוח קוד ופרשנות

- בדקו את שש התחזיות שבסעיף 3.9. חשבו מחדש Accuracy, Precision ו־Recall והסבירו איזו טעות מסוכנת יותר במערכת לתחזוקת מחשבים.
- במערך של 100 דוגמאות יש 90 דוגמאות ממחלקה אחת. הסבירו מדוע דיוק של 90% אינו בהכרח הצלחה.
- בקוד Iris מצאו את השורות של חלוקה, אימון, חיזוי והערכה. הסבירו מה עשויה לגרום השמטת random_state.
- נתונים נורמלו לפני החלוקה ל־Train ול־Test. האם יש כאן סיכון לזליגה? הסבירו בלי להשתמש רק במילה "אסור".

רמה ג — חשיבה והכנה לפרויקט גמר

- דודו מציע לחזות תקלה בשבוע הבא בעזרת "מספר תיקונים בשבוע הבא". נסחו במדויק מה שגוי במאפיין, וכתבו שני מאפיינים חלופיים.
- שני מודלים מציגים 92% ו־89% Accuracy. איזה מהם נבחר? נסחו שלוש שאלות שצריך לברר לפני החלטה.
- תכננו מחקר מקורי של 6–8 שורות: שאלת מחקר, סוג משימה, יחידת תצפית, שלושה מאפיינים, יעד ומדד הערכה.
- ציירו תרשים זרימה המציג את שלבי פרויקט AI, וכתבו מעל כל שלב סכנה אפשרית אחת ודרך לצמצם אותה.

3.14 משימת סיכום — כרטיס פרויקט ראשון

בחרו בעיה קטנה ומדידה מתוך עולם בית הספר, הספורט, התחבורה או המחשבים. הימנעו בשלב זה מנתונים אישיים רגישים. הציגו כרטיס פרויקט בן עמוד אחד: שם הפרויקט, מהי ההחלטה או התחזית, מיהו המשתמש, מהי תצפית, אילו מאפיינים ייאספו, מי יספק את תווית המטרה, מהו קו בסיס, וכיצד תיבחן הצלחה על נתונים שלא שימשו לאימון.

אין חובה לאמן עדיין מודל מורכב. תלמיד שמסוגל להגדיר היטב בעיה ולהסביר כיצד תיבדק כבר עשה צעד מקצועי משמעותי לקראת פרויקט הגמר.

3.15 מילון מושגים מסכם

מושג	הגדרה קצרה
Dataset	אוסף רשומות לצורך תיאור, חקר או למידה
Feature	מאפיין קלט המייצג מידע על תצפית
Target/Label	הערך שאותו מבקשים לנבא בלמידה מונחית
Model	מנגנון הממפה קלט לתחזית או לתוצאה
Train	דוגמאות המשמשות להתאמת המודל
Validation	דוגמאות לבחירת מודל ולכוונון תהליך
Test	נתונים להערכה סופית אחרי קבלת החלטות הפיתוח
Baseline	פתרון פשוט להשוואה
Generalization	היכולת להצליח על נתונים חדשים
Overfitting	התאמה חזקה מדי לפרטי האימון
Data Leakage	שימוש במידע שאינו אמור להיות זמין בתהליך
EDA	חקר נתונים לפני ובמהלך תהליך בניית המודל

3.16 קשרי המשך לשלוש תכניות הלימוד

היכן זה ממשיך	מה כבר למדנו / מה מוסיפים	מסלול
ספר יחידה 3 — EDA, סיווג, הערכה ופרויקט	תצפיות, מאפיינים, גרפים, KNN, מדדים וחקר	יחידה 3 — יסוד
פרקי נושאי ליבה 1–5 בספר	העמקת הסברים מתמטיים,	יחידה 5 — נושאי ליבה

היכן זה ממשיך	מה כבר למדנו / מה מוסיפים	מסלול
המאוחד	מרחקים, סטטיסטיקה ואלגוריתמים	
פרקים 4–16 בהמשך ספר ההתמחות	מחקר נתונים, מימוש, בחירה, תיעוד ואימון רשתות	התמחות Deep Learning

מעמד ההקשרים: מטרת פרק 3 בהתמחות אומתה לפי מפת הפרקים במאגר. ההפניות ליחידה 3 וליחידה 5 הן גשרים פדגוגיים ולא קביעה שכל אחד מן המושגים הוא דרישת בחינה חדשה במסלול האחר.

3.17 מקורות והכוונה לקריאה נוספת

- מפת תכנית ההתמחות במאגר הקורס: github.com/samiabo75/Data-Science-ML-Course — פרק 3, מבוא למדעי הנתונים ולמידת מכונה (10 שעות).
- Aurélien Géron, Hands-On Machine Learning with Scikit-Learn and PyTorch — גישת העבודה על נתונים, אימון והערכה.
- Luis G. Serrano, Grokking Machine Learning — הסברים אינטואיטיביים והתקדמות מדוגמאות למודלים.
- James et al., An Introduction to Statistical Learning with Applications in Python — הבנת קשר בין נתונים, מודל, דיוק והכללה.
- Google Machine Learning Crash Course: developers.google.com/machine-learning/crash-course
- scikit-learn, Common pitfalls: scikit-learn.org/stable/common_pitfalls.html
- התמונות, הנתונים המדומים, התרגילים והסיפורים בפרק נוצרו עבור תלמידי הקורס; אין בהם העתקה של קטעי ספר או תרשימים מוגנים.

3.18 דף משוב עצמי — מוכנות לקראת פרק 4 (EDA)

הפרק הבא יעמיק בבדיקת איכות נתונים, בסטטיסטיקה ובגרפים. לפני המעבר אליו מומלץ לענות על שאלות הבדיקה שלהלן ללא הסתכלות במחברת. כל שורה שבה אינכם בטוחים היא סימן לנושא שחשוב לחזור עליו. מדובר בכלי ללמידה עצמית, לא בציון.

מצב אישי	מה אני מסוגל להסביר ללא עזרה?	מיומנות
☐ יודע ☐ לתרגל	אני יכול לנסח שאלת חקר שיש לה קלט, יעד ומדד הצלחה.	הגדרת בעיה
☐ יודע ☐ לתרגל	אני יכול לזהות X, y ואת משמעות shape.	תצפיות ומאפיינים
☐ יודע ☐ לתרגל	אני מבחין בין סיווג, רגרסיה וקיבוץ ונותן דוגמה.	סוגי למידה
☐ יודע ☐ לתרגל	אני מסביר את ההפרדה בין Train, Validation ו־Test.	אימון והערכה
☐ יודע ☐ לתרגל	אני יודע לחשב מדד פשוט ולהסביר למה משווים אליו.	קו בסיס
☐ יודע ☐ לתרגל	אני יודע למה צריך לבדוק חסרים, יחידות וחריגים.	איכות נתונים
☐ יודע ☐ לתרגל	אני מסוגל לכתוב תוצאה ומגבלה בלי להבטיח דיוק בעתיד.	פרשנות

משימת כרטיס יציאה: בחרו אחת מן הבעיות שבהן עסקנו, והסבירו בחמש עד שבע שורות מה יידרש כדי להפוך אותה לפרויקט AI מקצועי. בתשובתכם השתמשו לפחות במושגים תצפית, מאפיין, תווית, קו בסיס, אימון ובדיקה. יש לנסח את התשובה במילים שלכם, לא רק להעתיק הגדרה מן הטבלה.

> נקודת המעבר לפרק 4: שם נלמד להפוך את הבדיקה הראשונית של מערך הנתונים לחקר שיטתי — זיהוי שאלות, תיאורים סטטיסטיים, גרפים, איכות נתונים ומסקנות מבוססות. ההחלטה אם לאמן מודל תתקבל רק אחרי שנבין מה הנתונים יכולים לספר לנו.

פרק 4 — חקר נתונים (EDA)

לשאול, לחקור, לבדוק ולנמק: כיצד הופכים טבלת נתונים להבנה שמקדמת מודל AI?

ספר הלימוד המאוחד — חוברת יסוד, פרק רביעי | התמחות למידת מכונה ולמידה עמוקה | 15 שעות לימוד



איור — המסלול החשיבתי בחקר נתונים: מתחילים בשאלה ולא בגרף.

הסיפור: דודו מרכז מעבדת מחשבים בבית הספר. לפני שהוא מפתח מודל שיחזה תקלות, הוא מקבל טבלה עם נתוני מחשבים: גיל, טמפרטורה, מספר תקלות וזמן תיקון. מנהל המעבדה שואל: ״איזה חדר סובל מהתחממות גבוהה יותר, והאם גיל המחשב קשור לתקלות?״

התשובות אינן מצויות בפקודת Python אחת. על דודו לבדוק מה כל שורה מייצגת, אילו ערכים חסרים, האם קיימות מדידות חריגות, איזה גרף מתאים לשאלה ומה ניתן להסיק בלי לחרוג מן הנתונים.

מטרות למידה והידע הנדרש

בסיום הפרק תוכלו: לטעון ולבדוק DataFrame; להסביר תצפית, מאפיין ומשתנה מטרה; לחקור מבנה וטיפוסים; לסנן ולמיין; לזהות חסרים וכפילויות; לחשב מדדי מרכז ופיזור; להשתמש ב־groupby; לשרטט גרפים; לזהות קשרים וחריגים; ולהציג ממצא, מגבלה ושאלה להמשך.

ידע קודם: משתנים, רשימות, פונקציות, יסודות NumPy ומושגי הממוצע והחציון מפרקי היסוד הקודמים. כל קטעי הקוד המודרכים נועדו להרצה ברצף באותה מחברת Colab.

תוצר	מטרת ביצוע	חלוקת 15 שעות מוצעת
כרטיס מאגר וסקירה	שאלה, מבנה נתונים, CSV ו־DataFrame	שעות 1–3
יומן איכות	סינון, מיון, איכות נתונים וחסרים	שעות 4–6
טבלה מנומקת	סטטיסטיקה, groupby והשוואת קבוצות	שעות 7–9
שלוש המחשות ופירוש	גרפים, קשרים וערכים חריגים	שעות 10–12
דוח קצר ומעבדת תלמיד	דוח EDA, מגבלות והכנה ל־ML	שעות 13–15

4.1 מהו EDA ומדוע הוא קודם לאימון מודל?

EDA הוא קיצור של Exploratory Data Analysis — חקר נתונים ראשוני. מטרתו להבין את מבנה המאגר, איכות המדידות, דפוסים וקשרים אפשריים, וגם לגלות שגיאות ושאלות שלא ידענו לשאול לפני שפתחנו את הנתונים. חקר נתונים אינו שלב קוסמטי של ״בניית גרפים יפים״; הוא חלק מתהליך קבלת החלטות מבוסס ראיות.

יש הבדל בין ממצא לבין פירוש. ״בחדר ב׳ ממוצע הטמפרטורה גבוה יותר״ הוא משפט תיאורי שניתן לבדוק בטבלה. ״בחדר ב׳ מערכת המיזוג מקולקלת״ הוא הסבר סיבתי אפשרי, אך לא הוכחה. ייתכנו הבדלים בגיל המחשבים, בשעת המדידה, בתנאי העומס או במד תקול.

שאלת חקר	נתונים שצריך	מה לא ניתן לקבוע לבדו
איזה חדר חם יותר?	טמפרטורה וחדר	מדוע הטמפרטורה שונה
האם גיל קשור לתקלות?	גיל ומספר תקלות	שגיל בהכרח גורם לכל תקלה
כמה מחשבים דורשים טיפול?	הגדרת ״דורש טיפול״ ומדידות	שההגדרה תתאים לעולם האמיתי

תרגול 1 | נסחו שאלה שאפשר לענות עליה מנתוני המעבדה, ושאלה שאי אפשר לענות עליה בלי נתון נוסף. הסבירו מדוע.

4.2 היכרות עם המאגר: תצפית, מאפיין וטבלת DataFrame

במאגר שלנו, כל שורה מתארת מחשב אחד. זו יחידת התצפית. כל עמודה היא תכונה: למשל גיל המחשב בשנים או טמפרטורת עבודה במעלות צלזיוס. זיהוי מחשב (`computer_id`) הוא מזהה ולא בהכרח מאפיין המתאים לאימון מודל. בעת חיזוי זמן תיקון, `repair_hours` עשוי להיות משתנה המטרה; בעת חיזוי תקלה אפשר שהיעד יהיה שדה אחר.

חשוב לשמור על יחידות: 55 מעלות צלזיוס אינן שקולות ל־55 דקות. מחשב מייצג עצם בעולם האמיתי, אך טבלת הנתונים היא ייצוג חלקי שלו, ולא תיאור של כל מה שמשפיע עליו.

computer_id	room	age_years	temp_c	failures
101	A	1	49.0	0
102	A	2	51.0	0
103	A	3	53.0	1
104	A	4	55.0	1
105	A	5	57.0	1
106	A	6	60.0	2

איור — חמש עמודות מתוך המאגר: שורה אחת מתארת מחשב אחד; לעמודות יחידות ומשמעויות שונות.

```
import pandas as pd
import numpy as np

data = {
   "computer_id": list(range(101, 113)),
   "room": ["A"] * 6 + ["B"] * 6,
   "age_years": [1, 2, 3, 4, 5, 6, 2, 4, 3, 7, 8, 1],
   "temp_c": [49, 51, 53, 55, 57, 60, 50, 58, 54, 91, None, 48],
   "failures": [0, 0, 1, 1, 1, 2, 0, 1, 1, 3, 2, 0],
   "repair_hours": [1, 1, 2, 2, 3, 4, 1, 2, 3, 8, 5, 1]
}
df = pd.DataFrame(data)
print(df.shape)
print(df.head())
```

שורת `data` מגדירה מילון: לכל שם עמודה מוצמדת רשימת ערכים באורך זהה. `pd.DataFrame` הופכת את המילון לטבלה. `shape` מחזירה זוג: מספר שורות ומספר עמודות — כאן 12 שורות ו־6 עמודות. `head()` מציגה תצוגה מקדימה; היא אינה משנה את הנתונים.

```
df.to_csv("computers_demo.csv", index=False)
loaded = pd.read_csv("computers_demo.csv")
print(loaded.head(3))
```

קובץ CSV הוא קובץ טקסט פשוט לייצוג טבלה. הכתיבה `index=False` מונעת שמירת אינדקס Pandas כעמודה נוספת. `read_csv` קוראת אותו בחזרה. בקובץ חיצוני חשוב לתעד מקור, תאריך, רישוי והגדרות של כל עמודה; כאן המאגר סינתטי ונוצר ללמידה בלבד.

4.3 סקירה ראשונית: head, info, describe ו־dtypes

לפני בניית גרף, כדאי להתבונן במאגר בכמה רמות. `head` עונה ״מה רואים ברשומות הראשונות?״; `shape` עונה ״מהו גודל הטבלה?״; `dtypes` עונה ״באיזה טיפוס נשמר כל שדה?״; `info` מסכמת טיפוסים ומספר ערכים שאינם חסרים; `describe` מסכמת ערכים מספריים, בדרך כלל בלי למחוק חסרים.

שימו לב: שדה גיל עשוי להיות מסוג `int64`, ואילו שדה טמפרטורה עם `None` ייקלט בדוגמה כסוג מספרי עשרוני (`float64`), משום ש־NaN משתלב בטיפוס זה. טיפוס שנשמר במחשב אינו בהכרח מפרש נכון את משמעות השדה — מיקוד ביקורתי צריך להתייחס גם לשמות וליחידות.

```
print(df.columns.tolist())
print(df.dtypes)
df.info()
print(df.select_dtypes(include="number").describe())
print(df["room"].value_counts())
```

המספר `count` בעמודת `temp_c` ב־`describe` יהיה 11, אף שבמאגר 12 מחשבים. הסיבה היא שקיימת מדידה חסרה אחת. זו הבחנה יסודית: 0 הוא ערך מספרי, אך חוסר מדידה הוא סימון של היעדר מידע.

פעולה	שאלה שהיא עוזרת לברר	זהירות
head(5)	איך נראות חמש השורות הראשונות?	לא בהכרח מייצגות את כל המאגר
info()	איזה טיפוס וכמה ערכים זמינים?	הטיפוס לא מבטיח משמעות נכונה
describe()	ממוצע, חציון, פיזור וטווח	סיכומים אינם חושפים כל דפוס
value_counts()	כמה מחשבים בכל חדר?	קטגוריות חסרות עשויות לא להיספר כברירת מחדל

תרגול 2 | אם יש 12 רשומות ורק 11 טמפרטורות ידועות, האם נכון לומר שהטמפרטורה החסרה היא אפס? נמקו.

4.4 בחירת נתונים, סינון ומיון: לשאול את הטבלה שאלה

השוואת קבוצות דורשת בחירה מודעת של שורות ועמודות. `df["temp_c"]` מחזירה Series, כלומר עמודה אחת; `df[["room", "temp_c"]]` מחזירה DataFrame, כלומר טבלה של שתי עמודות. ההבדל חשוב כאשר פעולות בהמשך מצפות לטבלה.

תנאי בוליאני ב־Pandas מחזיר עבור כל שורה אמת או שקר. בעזרת המסכה בוחרים את הרשומות המתאימות. אם יש כמה תנאים, נכתוב כל תנאי בסוגריים ונשתמש ב־`&` (וגם) או `|` (או) — ולא ב־`and` או `or` הפועלים על ערכי אמת יחידים.

```
temperatures = df["temp_c"]
room_and_temp = df[["room", "temp_c"]]
room_b = df.loc[df["room"] == "B", ["computer_id", "temp_c"]]
needs_review = df.loc[
   (df["temp_c"] > 60) | (df["failures"] >= 3),
   ["computer_id", "temp_c", "failures"]
]
print(needs_review)
print(df.sort_values("failures", ascending=False).head(4))
```

`loc` בוחרת לפי תוויות ושמות עמודות; `iloc` לפי מיקומים מספריים. `sort_values` מסדרת רשומות להצגה ואינה משנה את הטבלה המקורית ללא השמה או שימוש מפורש באפשרות מתאימה. סינון שורות ל״מחשבים שדורשים בדיקה״ הוא כלל פשוט, ולא מודל AI מאומן.

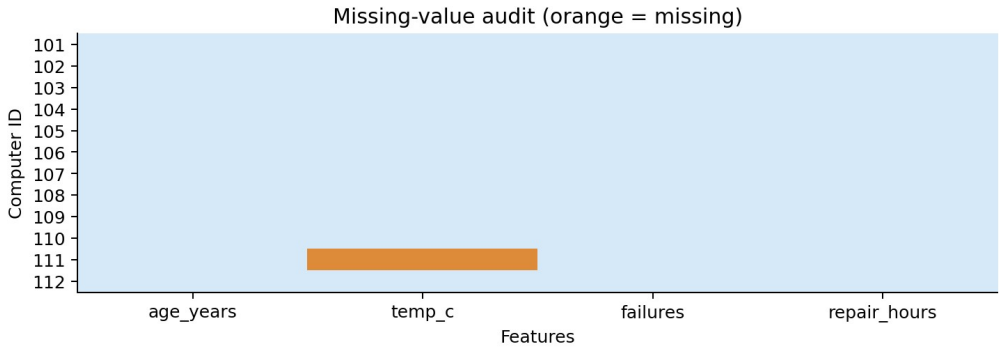
תרגול 3 | בנו מסכה לבחירת מחשבים שגילם ארבע שנים לפחות ושנרשמה אצלם לפחות תקלה אחת. לפני הרצה נסו לחזות כמה שורות יתקבלו.

4.5 איכות נתונים: ערכים חסרים, כפילויות, טווחים ושגיאות מדידה

נתוני העולם אינם תמיד שלמים. חוסר יכול להיווצר ממד שלא עבד, רישום שדולג או בעיית העברה. כפילות עשויה להופיע כאשר אותה רשומה נשמרה פעמיים. ייתכן גם ערך מחוץ לטווח סביר, כגון טמפרטורה גבוהה במיוחד. EDA צריכה להציג את הבעיה ולתעד את ההחלטה, לא לתקן בשקט.

נבדוק קודם את שיעור החסרים בכל עמודה. מאגר קטן בעל חסר אחד עלול לאבד חלק ניכר מהמידע אם נמחק שורות ללא הבחנה. גם מילוי חסרים בממוצע עלול לעוות את הפיזור. צריך לשאול האם החוסר אקראי, קשור לחדר מסוים או נובע מכשל מדידה.

```
missing_counts = df.isna().sum()
missing_percent = 100 * df.isna().mean()
print(pd.DataFrame({"missing": missing_counts,
                    "percent": missing_percent.round(1)}))
print("Exact duplicate rows:", df.duplicated().sum())
print("Duplicate computer IDs:", df.duplicated(subset=["computer_id"]).sum())
```



איור — כתום מסמן מדידה חסרה. גילוי מקום החוסר חשוב יותר ממחיקה אוטומטית.

הפקודה `duplicated()` בודקת שורות זהות, ואילו `duplicated(subset=["computer_id"])` בודקת חזרה על מזהה מחשב. בשני המקרים נקבל אפס בדוגמה, אך אם יופיע אותו מזהה פעמיים, נצטרך לבדוק האם מדובר בכפילות או בשתי מדידות בזמנים שונים. במקרה האחרון יש להוסיף זמן מדידה למפתח המתאר את הרשומה.

```
print(df.loc[df["temp_c"].isna(), ["computer_id", "room"]])
print(df.loc[df["temp_c"] > 75, ["computer_id", "temp_c"]])
clean_view = df.dropna(subset=["temp_c"]).copy()
print("Rows with known temperature:", len(clean_view))
```

בדוגמה נשמרות בטבלה המקורית 12 הרשומות. `clean_view` הוא עותק זמני המיועד לניתוח טמפרטורות בלבד, הכולל 11 רשומות. אין בכך הצדקה למחוק את המחשב בעל המדידה החסרה מכל יתר הניתוחים. אי אפשר ללמוד מטמפרטורה חסרה על זמן התיקון או על גיל המחשב.

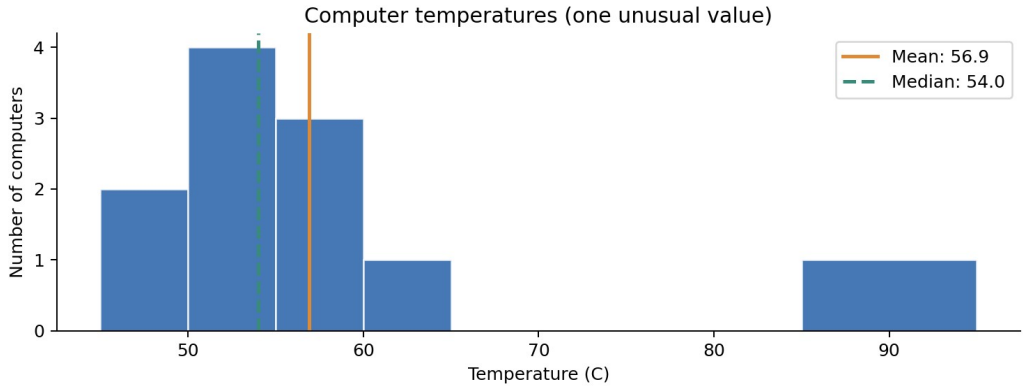
תרגול 4 | הציעו שתי אפשרויות לטיפול בטמפרטורה החסרה וציינו יתרון ומגבלה של כל אחת. החליטו מה תעשו בדוח **EDA**, בלי לפגוע במקור.

4.6 מדדי מרכז ופיזור: מה מסתתר מאחורי הממוצע?

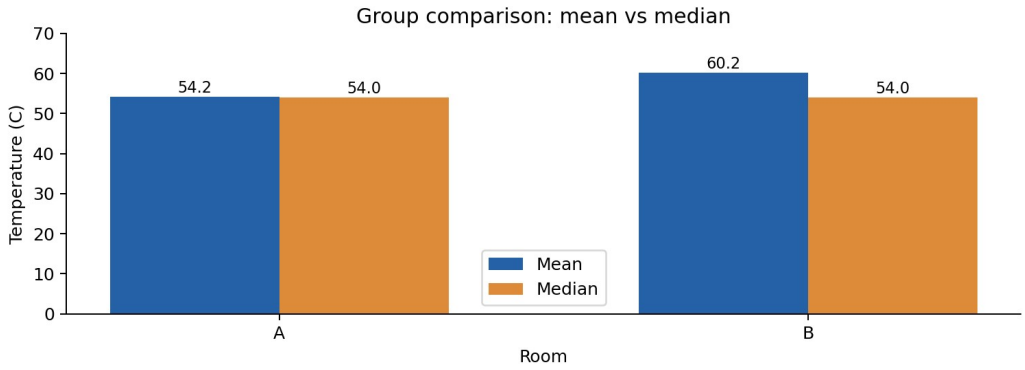
נחשב את טמפרטורת העבודה הממוצעת של החדרים. בחדר A הנתונים הם 49, 51, 53, 55, 57, 60. סכומם 325 ולכן הממוצע הוא 325 חלקי 6, כלומר כ־54.17 מעלות. בחדר B ידועות חמש מדידות בלבד: 50, 58, 54, 91, 48. סכומן 301 ולכן הממוצע הוא 301 חלקי 5, כלומר 60.2 מעלות.

אם מסתפקים בממוצע בלבד, עלולים לקבוע שחדר B חם בהרבה. אולם יש בו מדידה חריגה של 91 מעלות. החציון של חמש המדידות בחדר B הוא 54 מעלות — נמוך מהממוצע. לכן יש להציג גם חציון, התפלגות והחריג, ולשאול אם המדידה נכונה.

```
summary = df.groupby("room")["temp_c"].agg(
   count="count", mean="mean", median="median",
   std="std", minimum="min", maximum="max"
)
print(summary.round(2))
```



איור — היסטוגרמה של הטמפרטורות ומיקומי הממוצע והחציון. הערך החריג משפיע על הממוצע.



איור — ממוצע מול חציון לפי חדר. תמיד יש לבדוק מה מסתתר מאחורי ההבדל.

סטיית תקן מתארת פיזור סביב הממוצע. כברירת מחדל, `Series.std()` של Pandas מחשבת סטיית תקן מדגמית באמצעות חלוקה ב־n−1; לעומת זאת `np.std` כברירת מחדל מחלקת ב־n. אין להציג את שתי התוצאות כטעות — צריך לבחור הגדרה מתאימה ולציין אותה.

רבעונים וטווח בין־רבעוני (IQR) מסייעים לבדוק ערכים חריגים. ב־Pandas הפונקציה `quantile` משתמשת כברירת מחדל באינטרפולציה ליניארית; שיטות חלוקה ידניות אחרות עשויות לתת ערכי רבעונים מעט שונים במדגם קטן. חשוב להסביר מהי השיטה שנבחרה, ולא לערבב שיטות באמצע פתרון.

```
known = df["temp_c"].dropna()
q1 = known.quantile(0.25)
q3 = known.quantile(0.75)
iqr = q3 - q1
upper_fence = q3 + 1.5 * iqr
print(f"Q1={q1}, Q3={q3}, IQR={iqr}, upper={upper_fence}")
print(known[known > upper_fence])
```

בדוגמה זו Pandas מחזירה בקירוב Q1=50.5, Q3=57.5 וגבול עליון 68. לכן הערך 91 מסומן כחריג לפי כלל IQR×1.5. סימון חריג הוא אות לחקירה נוספת — לא הוכחה שהמדידה שגויה ולא הוראה למחוק אותה.

תרגול 5 | הסבירו מדוע לחדר **B** ממוצע גבוה יותר אף שהחציון בו דומה לזה של חדר **A**. איזו בדיקה הייתם עורכים לפני כתיבת המלצה להנהלת בית הספר?

4.7 GroupBy: להשוות בין קבוצות בלי לאבד את ההקשר

פעולת `groupby` מחלקת את הרשומות לפי ערך משותף, מפעילה פעולה על כל קבוצה ומחזירה סיכום. הרעיון נקרא לעיתים Split–Apply–Combine: מחלקים, מחשבים, מחברים את התוצאות. למשל, לכל חדר נחשב את מספר המחשבים, הממוצע של מספר התקלות וסך שעות התיקון.

חשוב להבחין בין `size`, הסופרת את כל השורות בקבוצה, לבין `count`, הסופרת רק ערכים שאינם חסרים בעמודה מסוימת. במקרה שלנו בחדר B יש שישה מחשבים, אך רק חמש מדידות טמפרטורה זמינות.

```
room_report = df.groupby("room").agg(
    computers=("computer_id", "size"),
    known_temperatures=("temp_c", "count"),
    average_failures=("failures", "mean"),
    total_repair_hours=("repair_hours", "sum")
)
print(room_report.round(2))
```

נניח שחדר A מכיל 6 מחשבים וחדר B גם 6. אם נגלה שבחדר B סך שעות התיקון גבוה יותר, נצטרך לבדוק האם התקלות קשות יותר, המחשבים ישנים יותר, או שתועדו אחרת. טבלת קיבוץ מספקת תיאור, לא תשובה סיבתית. בניתוח אמיתי נרצה להוסיף תאריכים, שעות שימוש ומאפיינים נוספים.

תרגול 6 | האם מספר המחשבים בחדר בהכרח שווה למספר טמפרטורות ידועות? תנו דוגמה מתוך הנתונים והסבירו את משמעותה.

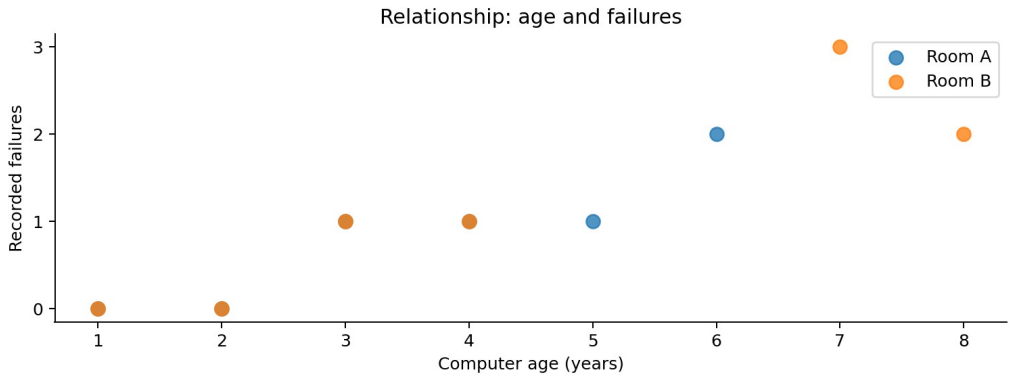
4.8 בחירת גרף לפי השאלה ולא לפי ההרגל

תרשים עמודות מתאים להשוואת קטגוריות — למשל מספר מחשבים בכל חדר. היסטוגרמה מתארת התפלגות של משתנה מספרי, כגון טמפרטורה. תרשים פיזור מתאים לשני משתנים מספריים, כגון גיל ומספר תקלות. תרשים קופסה מסכם חציון, רבעונים וערכים חריגים אפשריים.

לפני גרף נשאל: איזה משתנה מוצג בכל ציר? מה היחידות? האם יש חסרים? האם גודל המדגם מספיק? האם הצירים התחילו בנקודה שאינה אפס באופן שמגזים את ההבדל? מהי השאלה שהגרף אמור לעזור לענות עליה?

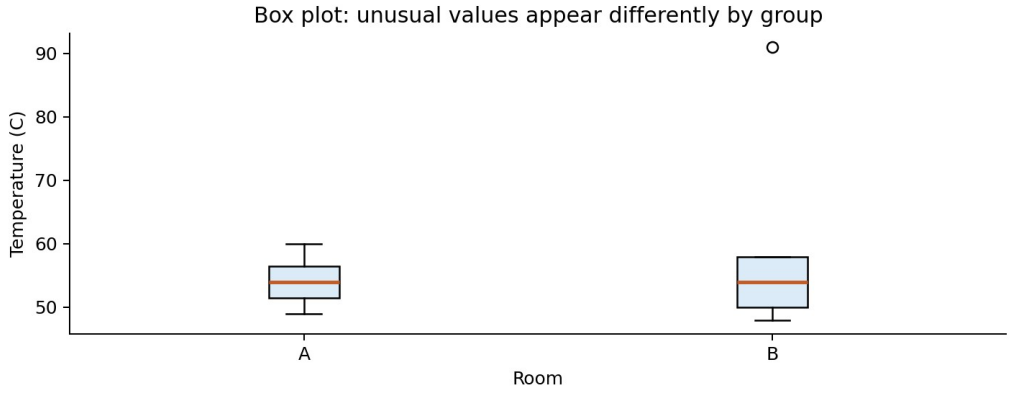
```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(6, 4))
ax.scatter(df["age_years"], df["failures"])
ax.set_xlabel("Computer age (years)")
ax.set_ylabel("Recorded failures")
ax.set_title("Age and failures")
plt.show()
```



איור — תרשים פיזור: קשר אפשרי בין גיל לתקלות; אין להסיק סיבתיות מתרשים.

```
fig, ax = plt.subplots(figsize=(6, 4))
room_a = df.loc[df["room"] == "A", "temp_c"].dropna()
room_b = df.loc[df["room"] == "B", "temp_c"].dropna()
ax.boxplot([room_a, room_b], labels=["A", "B"])
ax.set_ylabel("Temperature (C)")
ax.set_title("Temperatures by room")
plt.show()
```



איור — תרשים קופסה מציג את מאפייני ההתפלגות גם כאשר הממוצע מטעה.

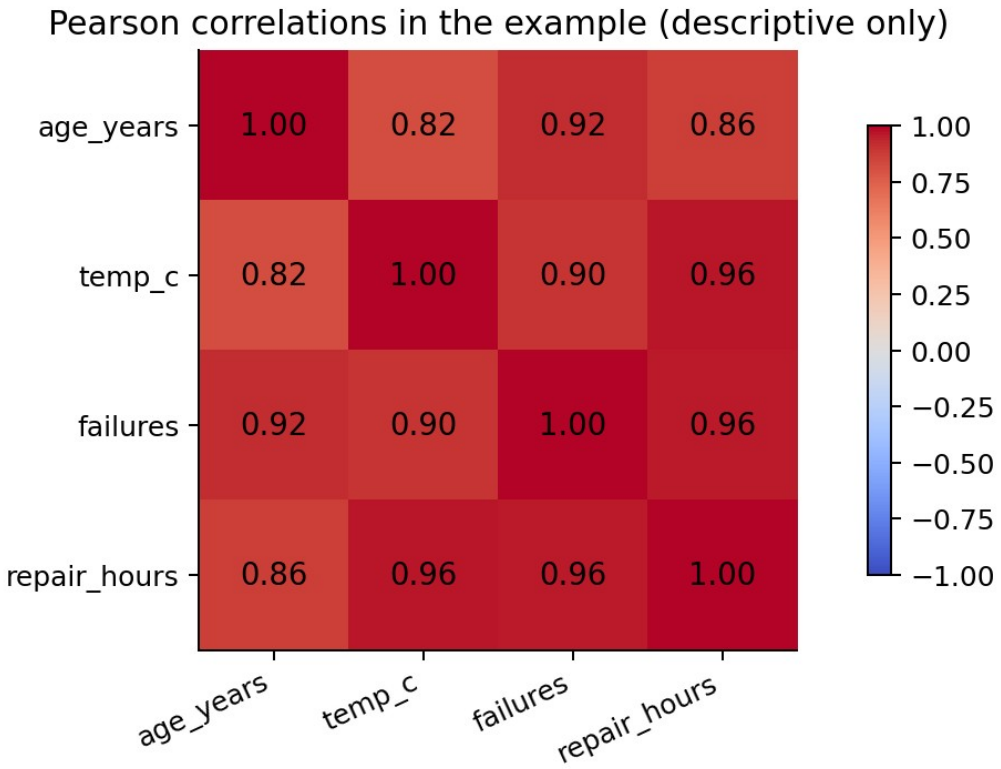
מטרת המחקר	הגרף המתאים	למה?
השוואת מספר מחשבים לפי חדר	עמודות	קטגוריות ומניינים
התפלגות הטמפרטורות	היסטוגרמה	התפלגות של משתנה מספרי
קשר גיל–תקלות	פיזור	שני משתנים מספריים
חריגים לפי חדר	קופסה	חציון, רבעונים וחריגים

4.9 קשר בין משתנים: מתאם אינו הסבר סיבתי

מתאם פירסון מודד עוצמה וכיוון של קשר ליניארי בין שני משתנים כמותיים, כאשר הוא מוגדר. ערכו בין מינוס אחת לאחת. ערך גבוה וחיובי מצביע על קשר ליניארי חיובי במדגם, אך אינו מוכיח שמשתנה אחד גורם לשינוי בשני. קשר יכול לנבוע ממשתנה שלישי, ממבנה הדגימה או מצירוף נסיבות.

במאגר שלנו ייתכן קשר חיובי בין מספר התקלות לשעות התיקון, בין השאר משום ששני המשתנים מתארים היבטים של בעיות במחשבים. עדיין ייתכן שזמן התיקון מושפע מחלקי חילוף, זמינות טכנאי ומועד התקלה — נתונים שאינם נמצאים בטבלה.

```
numeric = df[["age_years", "temp_c", "failures", "repair_hours"]]
correlations = numeric.corr(numeric_only=True)
print(correlations.round(2))
# The correlation matrix is descriptive, not a causal model.
```



איור — מפת חום של מתאמי פירסון מתוך הנתונים הסינתטיים בלבד.

קיימת מגבלה נוספת: ערכי המתאם מחושבים על זוגות זמינים של מדידות, ולכן מספר הרשומות המעורבות בכל תא במטריצה עשוי להשתנות כשיש חסרים. כמו כן, ערך חריג יכול להשפיע מאוד על פירסון; במידת הצורך יש לבדוק גם תרשים פיזור ומדד חלופי כגון Spearman. איננו מפעילים חישוב נוסף רק כדי לקבל מספר גבוה — מטרתנו להבין את השאלה.

תרגול 7 | כתבו שני משפטים: משפט אחד שמתאר ממצא שיכול להתקבל מתרשים פיזור, ומשפט אחד שמציג הסבר אפשרי שעדיין טעון בדיקה.

4.10 מה־EDA למודל AI: הבחירה והבדיקה חייבות להיות הוגנות

אם EDA מיועדת לתיאור מאגר בלבד, אפשר לסקור את הנתונים במלואם — בתנאי שלא נטען בהמשך שהערכת המודל עליהם בלתי תלויה. אך כאשר חקר הנתונים משמש לבחירת מאפיינים, זיהוי משתנים חשובים, קביעת ספי ניקוי או התאמת טרנספורמציות למודל, יש לשמור מראש קבוצת בדיקה עצמאית.

לכן בפרויקט למידת מכונה נפריד תחילה בין נתוני אימון לבדיקה (לפי מבנה הבעיה: חלוקה אקראית, לפי זמן או לפי קבוצות). החלטות שמכוונות את המודל נלמדות מנתוני האימון בלבד, או מתהליך ולידציה פנימי. לעולם אין להשתמש בקבוצת Test שוב ושוב כדי לבחור מודל ואז לדווח על הציון כאילו זו הערכה עצמאית.



איור — נתוני בדיקה נפרדים אינם משמשים לבחירת מאפיינים או להתאמת שלבי עיבוד.

```
from sklearn.model_selection import train_test_split

train_df, test_df = train_test_split(
    df, test_size=0.25, random_state=42
)
# Exploratory choices for modeling are made on train_df.
print("Train:", len(train_df), "Test:", len(test_df))
```

החלוקה כאן נועדה להמחיש רעיון בלבד: המאגר מכיל 12 מחשבים סינתטיים, מעט מדי כדי להסיק על מודל אמיתי. בפרויקט עם נתוני זמן או כמה מדידות לאותו מחשב נצטרך חלוקה מתאימה, המונעת מעבר מידע בין אימון לבדיקה. למשל, שתי מדידות של אותו מחשב עלולות להיות תלויות זו בזו, ולכן חלוקה אקראית של שורות אינה תמיד מספיקה.

תרגול 8 | מדוע אסור לחשב ממוצע לצורך מילוי חסרים על כל נתוני הפרויקט ואז להפריד לאימון ולבדיקה? הסבירו במונחי מידע שהיה זמין בזמן האימון.

4.11 חקר מקרה מסכם — דוח EDA שלא מסתפק בהדפסת גרפים

דודו נדרש להגיש להנהלת בית הספר דוח קצר. הוא לא כותב ״חדר B חם יותר ולכן צריך להחליף מזגן״. במקום זאת הוא מתאר את מקור הנתונים, טבלת המאפיינים, היקף החסרים, המדדים שנבחרו והחשש שמדידה אחת משפיעה על הממוצע. הוא מציג כמה הסברים אפשריים ומציע מדידות נוספות.

דוח מקצועי כולל לפחות ארבעה מרכיבים: מה בדקנו; מה ראינו בפועל; מה מגביל את הפרשנות; ומה נדרש כדי לקבל החלטה טובה יותר. כמו כן יש לשמור קוד שניתן להרצה מחדש, לציין אילו שורות נכללו בחישובים ולשמור עותק של נתוני המקור.

מרכיב בדוח	דוגמה מנוסחת
שאלת המחקר	האם התפלגות הטמפרטורות שונה בין חדרים A ו־B?
ממצא	הממוצע בחדר B הוא 60.2 מעלות, לעומת כ־54.17 בחדר A.
מגבלה	בחדר B חסרה מדידה אחת וקיימת מדידה גבוהה במיוחד של 91.
המשך בדיקה	בדיקת חיישן, שעות שימוש ורישום מדידות חוזרות לפני קביעה סיבתית.

כאשר עובדים עם תלמידים, נתוני בריאות או ציונים, חשוב במיוחד לשקול פרטיות, הוגנות והשפעה על אנשים. דוגמאות הספר סינתטיות; בעבודה עם מידע אמיתי יש לפעול לפי מדיניות בית הספר, רישיונות השימוש והוראות החוק הרלוונטיות.

4.12 מטלה מסכמת לתלמיד — חוקר נתונים עצמאי

במחברת התרגול המצורפת תעבדו על מאגר שונה מן הדוגמה שבספר: נתוני השאלת ציוד בית־ספרי שנוצרו לצורכי לימוד. תגדירו שאלת חקר, תבדקו את מבנה הטבלה ואת איכות הנתונים, תבצעו לפחות שני סיכומי קבוצות, תיצרו שלושה גרפים ותכתבו מסקנה מסויגת. קטעי הקוד החסרים במחברת הם מטלות לביצוע — לא בעיה בקובץ.

תרגול 9 | הגישו דוח קצר ובו כרטיס מאגר, שתי שאלות חקר, טבלת איכות נתונים, השוואת שתי קבוצות, שלושה גרפים עם פרשנות, מסקנה מוגבלת והמלצה להמשך איסוף נתונים.

4.13 ממוצע של ממוצעים: מדוע חשוב לדעת מה גודל כל קבוצה?

נניח שמנהלת המעבדה משווה בין שתי קבוצות תלמידים. בקבוצה הראשונה יש שני מחשבים, שממוצע זמן התיקון שלהם הוא 4 שעות. בקבוצה השנייה יש שמונה מחשבים, שממוצע זמן התיקון שלהם הוא 2 שעות. אם נחשב (4+2)/2 נקבל 3 שעות — אבל זהו ממוצע הממוצעים של הקבוצות, המייחס משקל שווה לשתי קבוצות בגדלים שונים. הוא אינו ממוצע זמן התיקון של כל עשרת המחשבים.

כדי לחשב את הממוצע המשוקלל לפי מספר הרשומות, נכפיל את ממוצע הקבוצה במספר המחשבים שבה: בקבוצה הראשונה סך השעות הוא 2×4=8; בשנייה 8×2=16. ביחד 24 שעות לעשרה מחשבים, ולכן ממוצע כלל המחשבים הוא 24/10=2.4 שעות.

כלומר, כשמחברים נתונים מקבוצות שונות יש לשאול באיזה מדד משתמשים, ומהו המשקל הראוי לכל קבוצה. ממוצע הממוצעים יכול להיות מוצדק אם השאלה היא ״מהו ממוצע הביצועים בין חדרים כשכל חדר מקבל משקל שווה?״ — אך אינו תשובה לשאלה ״מהו ממוצע כלל המחשבים?״.

קבוצה	מספר מחשבים	ממוצע שעות למחשב	סך שעות
חדר 1	2	4	8
חדר 2	8	2	16
ביחד	10	2.4 (משוקלל)	24

```
small_groups = pd.DataFrame({
    "group": ["A", "B"],
    "computers": [2, 8],
    "mean_hours": [4.0, 2.0]
})
weighted_mean = (
    (small_groups["computers"] * small_groups["mean_hours"]).sum()
    / small_groups["computers"].sum()
)
print(weighted_mean)  # 2.4 hours
```

תרגול 10 | כתבו דוגמה שבה ממוצע פשוט של ממוצעי שתי קבוצות יהיה נמוך מהממוצע המשוקלל. הסבירו איזו שאלה כל חישוב עונה עליה.

4.14 טיפול בחסרים: לא כל חסר דורש את אותו הפתרון

יש כמה דרכים אפשריות לטיפול בחסר: להשאיר אותו כחסר בזמן התיאור; לא לכלול את הרשומה בחישוב המתייחס לעמודה החסרה; למלא ערך לפי שיטת השלמה שהוגדרה מראש; או לחזור למקור הנתונים כדי לתקן את האיסוף. אין פתרון אחד שנכון לכל מאגר.

אם חסרות טמפרטורות דווקא כאשר המחשבים חמים מאוד והחיישן קורס, מחיקה של אותן מדידות עלולה לגרום להערכת חסר של הטמפרטורה הממוצעת. אם החסרים קשורים לחדר, לשעה או לגיל המחשב, גם

ההשוואה בין קבוצות עשויה להשתנות. לכן יש לבדוק היכן מופיעים החסרים ולשקול את מנגנון ההיעדרות, ולא רק לספור אותם.

בתהליך פיתוח מודל חשוב להבחין בין סיכום תיאורי לבין השלמת נתונים לצורך חיזוי. למשל, השלמה בחציון שנלמד מנתוני האימון יכולה להיות חלק מ־Pipeline; את אותו חציון מפעילים על קבוצת הבדיקה בלי לחשב אותו מחדש. חציון שחושב גם על הבדיקה חושף מידע שלא אמור להשפיע על ההחלטות בזמן אימון.

```
# Descriptive comparison only: keep the original intact.
analysis_a = df.copy()
analysis_b = df.copy()
analysis_b["temp_c"] = analysis_b["temp_c"].fillna(
   analysis_b["temp_c"].median()
)
print("Original known temperatures:", df["temp_c"].count())
print("Filled series count:", analysis_b["temp_c"].count())
# Do not use full-data imputation when preparing an independent ML test.
```

ההדגמה האחרונה ממלאת את המדידה החסרה לצורך השוואה לימודית בלבד, ולא לצורך הערכה של מודל. שימו לב שהעמודה החדשה כוללת ערך שהוסק מן הנתונים הקיימים, ולא מדידה אמיתית. לכן יש לסמן בתיעוד מה הושלם, מה נמדד בפועל ובאיזו שיטה הושלם.

4.15 ייצוג נתונים וגרפים: לזהות גם הצגה מטעה

כאשר מנהלת רואה תרשים עמודות שבו ערכי הטמפרטורה מתחילים בציר האנכי ב־50 מעלות, הבדל של חמש מעלות עשוי להיראות כמו פי שניים. לעיתים חיתוך ציר לגיטימי לצורך הצגת שינוי קטן, אך הוא מחייב סימון ברור של הטווח. בתרשימי עמודות של כמויות עדיף בדרך כלל להתחיל מאפס; בתרשימי קו המתארים מדידה לאורך זמן ייתכן שטווח מצומצם מועיל, בתנאי שמציגים אותו במפורש.

חשוב גם להבחין בין מספרים מוחלטים לאחוזים. בכיתה אחת היו שתי תקלות מתוך עשרה מחשבים ובכיתה אחרת שלוש תקלות מתוך חמישים. מספר התקלות גדול יותר בשנייה, אך שיעור התקלות דווקא נמוך יותר. כשעובדים עם קבוצות בגדלים שונים יש להציג לפעמים גם מונה וגם מכנה.

קבוצה	תקלות	מספר מחשבים	שיעור תקלות
קבוצה א׳	2	10	20%
קבוצה ב׳	3	50	6%

הטבלה מדגימה עיקרון של שיעורים: היחס 2/10 גדול מהיחס 3/50. אבל גם שיעור אינו בהכרח השוואה הוגנת אם משכי שימוש שונים, אם למחשבים בוצעו טיפולים שונים או אם אופן רישום התקלה אינו אחיד. השוואה אמינה דורשת הגדרות שוות ובדיקה של גורמים מערפלים.

תרגול 11 | מנהל אמר: ״בחדר ב׳ היו יותר תקלות, ולכן הוא מתוחזק פחות טוב״. הסבירו מה חסר בטענה, ואילו שני מדדים נוספים כדאי לבקש.

4.16 מילון נתונים (Data Dictionary): לדעת מה פירוש כל עמודה

טבלה בלי מילון שדות דומה למפת דרכים בלי שמות הרחובות. לפני חקר הנתונים יש להבין מי יצר את השדה, מה היחידות שלו, אילו ערכים חוקיים ומה משמעות החסר. לדוגמה, `age_years` מתאר את גיל המחשב בשנים; אם אחד המחשבים חדש וחלקי שנה עוגלו למטה, גיל 0 אינו בהכרח טעות.

ננסח מילון נתונים בסיסי למאגר התרגול, ונשמור אותו לצד ה־CSV והמחברת. אם בהמשך נשנה את סדר העמודות או ניצור עמודה חדשה, נעדכן גם את המילון. תיעוד כזה מצמצם שגיאות כאשר כמה תלמידים עובדים יחד על אותו פרויקט.

שדה	טיפוס ומשמעות	יחידה או תחום	אזהרת פירוש

computer_id	מזהה ייחודי למחשב	מזהה, לא מדידה	מספר גדול אינו מחשב חזק יותר
room	מיקום פיזי	A או B	קטגוריה, לא סדר מספרי
age_years	גיל המחשב	שנים	ייתכן עיגול
temp_c	מדידת חום	מעלות צלזיוס	חסר אחד; מדידה 91 דורשת בדיקה
failures	מספר תקלות מתועדות	ספירה	תלוי במשך זמן המעקב
repair_hours	סך שעות תיקון	שעות	עשוי להיות יעד של מודל חיזוי

```
data_dictionary = pd.DataFrame([
    ["computer_id", "identifier", "not a numeric feature"],
    ["room", "category", "A or B"],
    ["age_years", "numeric", "years"],
    ["temp_c", "numeric", "degrees Celsius"],
    ["failures", "count", "recorded cases"],
    ["repair_hours", "numeric", "hours"]
], columns=["field", "kind", "description"])
data_dictionary.to_csv("computers_dictionary.csv", index=False)
```

4.17 כיצד כותבים דוח EDA כראיה ולא כטענה כללית?

דוח טוב אינו אוסף של תרשימים. לכל תרשים צריכים להיות כותרת ותיאור שאלה, צירים ויחידות, ממצא מדויק, פירוש מוגבל והצעה להמשך. אפשר להשתמש במבנה הבא: ״בדקנו ... באמצעות ...; ראינו כי ...; יש להביא בחשבון ש ...; לפני הסקת מסקנה על ... נדרשת בדיקה של ...״.

למשל: ״בדקנו את הטמפרטורה בחדרים A ו־B בעזרת ממוצע, חציון ותרשים קופסה. ממוצע חדר B היה 60.2 מעלות לעומת כ־54.17 בחדר A, אך בחדר B נרשמה מדידה אחת של 91 מעלות ומדידה חסרה נוספת. לכן נדרש בירור מקור המדידות והרחבת תקופת האיסוף לפני קביעה שחדר אחד חם יותר באופן עקבי״.

ממצא אינו חייב לתמוך בהשערה המקורית. אם הגרף אינו מראה קשר ברור, מסקנה מתאימה היא ״בנתונים הקיימים לא מצאנו עדות מספקת לקשר ברור״, בתוספת מגבלות הדגימה. אין לשנות מאפיינים או מדדים שוב ושוב עד שמתקבלת תוצאה ״יפה״ ואז להציג זאת כניסוי בלתי תלוי.

תרגול 12 | נסחו פסקת ממצא חדשה מתוך אחד הגרפים בחוברת. פסקתכם צריכה לכלול נתון מספרי, פירוש, מגבלה ופעולת המשך אפשרית.

שאלות הבנה והעברה — שלוש רמות

משימה	רמה
פרשו את ההבדל בין 12 תצפיות ל־11 טמפרטורות ידועות. מהן היחידות?	א — בסיס
הבחינו בין Series ל־DataFrame ובין loc ל־iloc.	א — בסיס
חשבו ביד את הממוצע בחדר B והסבירו את	ב — חישוב

משימה	רמה
השפעת 91.	
בחרו גרף מתאים להשוואת התפלגויות ונמקו.	ב — ניתוח
תלמיד מחק את כל השורות החסרות. אילו הטיות עלולות להיווצר?	ג — ביקורת
כיצד הייתם מבצעים EDA בלי לחשוף קבוצת Test לבחירת מאפיינים?	ג — מחקר

בדיקת מוכנות לפרויקט גמר

לפני המעבר לפרק הרגרסיה הליניארית התלמיד צריך להראות שהוא יודע להגדיר משתנה מטרה, להסביר מהו מקור הנתונים, להפעיל בדיקות חסרים וכפילויות, לחשב סיכומים ו־groupby, לבחור גרף לפי סוג משתנה, לפרש מתאם בזהירות ולשמור על הערכת Test עצמאית. קוד שרץ ללא הסבר אינו מספיק.

מפת הקשרים בין התכניות

שימוש בהמשך	מוקד ההוראה	מסלול
שאלת חקר ותלקיט אישי	היכרות עם DataFrame, גרפים, סיכומים ומסקנות	יחידה 3 — מדעי הנתונים
העמקה מתמטית ופירוש	מדדי מרכז ופיזור, יחסים בין משתנים, קריאת גרפים	יחידה 5 — נושאי ליבה
בחירת מאפיינים, ניסויים ופרויקט גמר	תהליך EDA עצמאי לקראת פיתוח מודל	התמחות למידה עמוקה — פרק 4

מילון מושגים

משמעות	מושג
אוסף תצפיות ומאפיינים שנאספו או נוצרו למטרה מסוימת	Dataset
יחידת תצפית — במקרה שלנו מחשב	Observation
מאפיין המתאר תצפית	Feature
משתנה שאנחנו מנסים לחזות במודל מונחה	Target
טבלה דו־ממדית ב־Pandas	DataFrame
עמודה או סדרת ערכים עם אינדקס	Series
ערך חסר, שאינו זהה לאפס	Missing value
חלוקת רשומות לקבוצות וחישוב סיכומים	GroupBy
תצפית חריגה ביחס להתפלגות או לכלל שנבחר	Outlier
שימוש לא תקין במידע שלא אמור להיות זמין בזמן בניית המודל	Data Leakage

מקורות והערות עריכה

עוגן תכנית: מפת התמחות למידת מכונה ולמידה עמוקה במאגר GitHub של הקורס, פרק 4 — חקר נתונים, 15 שעות. חלוקת השעות, הדוגמאות והעומק המפורט הם הצעה דידקטית שלנו; יש לאמת כל חובת בחינה מול הנוסח הרשמי המעודכן.

חומרי הוראה משלימים במאגר: סיכום ההשתלמות על Pandas, Iris ו־Penguins, והנחיות המורה למפגש CSV. מקורות מקצועיים: An Introduction to Statistical Learning with Applications in Python; Hands-On Machine Learning; Pandas User Guide; scikit-learn — Common Pitfalls. הדוגמאות והאיורים בחוברת מקוריים; אין הם העתקה מספרי המקור.

תיעוד מקצועי: https://pandas.pydata.org/docs/user_guide/ ; https://pandas.pydata.org/docs/user_guide/groupby.html ; https://scikit-learn.org/stable/common_pitfalls.html

דף עזר — איזו פקודה מתאימה לשאלה שלי?

שאלה	פקודת **Pandas** מומלצת
כמה רשומות ועמודות יש?	`df.shape`
אילו טיפוסים וחסרים יש?	`df.info(); df.isna().sum()`
מהי שכיחות קטגוריה?	`df["room"].value_counts()`
איך בוחרים שורות ועמודות?	`df.loc[mask, ["col1", "col2"]]`
איך מסכמים לפי חדר?	`df.groupby("room").agg(...)`
איך מוצאים חציון או רבעון?	`df["temp_c"].median(); .quantile(0.25)`
מה הקשר הליניארי בין מאפיינים?	`df[["x", "y"]].corr()`
כיצד שומרים תוצאות בלי אינדקס?	`df.to_csv("file.csv", index=False)`

זכרו: בחירת הפקודה היא רק התחלה. בכל משימת חקר יש להסביר גם את משמעות הפלט, את היחידות ואת מגבלת המסקנה.